



A Technical Report Of CAA900NCE – Capstone Project On Secure Cloud-Based E-Commerce Platform: SOS (SEA of Style)

Submitted By:

Group 8

Date: 12 April 2026

Team Member and their Responsibilities

Shirish Anand Joshi : Infrastructure Engineer

Eldar Azimov : Lead DevOps Engineer

Amir Bhandari : Cloud Solutions Architect

Submitted To:

Professor: Mehdi Akbari

Cloud Architecture and Administration

Table of Contents

Introduction.....	7
System Overview	7
SOS-Sea of Style Infrastructure Diagram.....	8
Explanation of All Services	9
Frontend Layer.....	9
Authentication Layer	9
API & Compute Layer.....	9
Data Layer.....	10
Notification Layer	10
Observability & Monitoring	10
Security & IAM	10
DevOps & Infrastructure Management (CI/CD + Terraform).....	10
Workflow of SOS Web Application.....	11
Customer Workflow (End-User Journey).....	11
Admin Workflow (Management Operations).....	25
AWS Services Used.....	33
Core Application Services	33
Integration and Communication Services.....	46
Monitoring, Logging and Alerting Services	49
Security Services.....	51
DevOps and Infrastructure Management	54
Terraform Implementation.....	54
API Gateway Module	54
CloudFront Module: Frontend Hosting	55
Cognito Module: Authentication	56
DynamoDB Module: Application Tables	57
IAM Module: Execution Role and Policies	57
Lambda Module: Serverless Functions.....	58
Monitoring Module: CloudTrail and Audit Logging.....	59
Notifications Module	59
S3 Module: Data Buckets	60
Secrets Module: KMS and Secrets Manager	61
AWS Well-Architected Framework Compliance	62

Operational Excellence	62
Security	63
Reliability.....	63
Performance Efficiency	63
Cost Optimization	63
Sustainability.....	63
Cost Estimation.....	64
Cost Analysis	64
Conclusion	65
Questions and Answers:.....	66

Table of Figures

Figure 1: Architecture Diagram.....	8
Figure 2: Home Page	11
Figure 3: Shop Page.....	12
Figure 4: About Page	12
Figure 5: Contact Page.....	13
Figure 6: Login Page.....	13
Figure 7: Sign up Page.....	14
Figure 8: Verification Code	14
Figure 9: Account Confirmed	15
Figure 10: User Profile Dashboard	15
Figure 11: Order History.....	16
Figure 12: Change Password	16
Figure 13: Product Information	17
Figure 14: Add to Cart	17
Figure 15: Shopping Cart.....	18
Figure 16: Shipping Page.....	18
Figure 17: Payment with Stripe	19
Figure 18: Payment Successful.....	19
Figure 19: Order History.....	20
Figure 20: Telegram Notification	21
Figure 21: Amazon Lex - User Interaction	21
Figure 22: Forget Password	22
Figure 23: Verification Code for forgetting the password.....	23
Figure 24: Password Reset.....	23
Figure 25: Invalid Username or Password.....	24
Figure 26: Password Policy Unmatched	24
Figure 27: Admin Dashboard.....	25
Figure 28: Add Product.....	25
Figure 29: Edit Product.....	26
Figure 30: Delete the Product	26
Figure 31: Delete Successful	27
Figure 32: Modify User's Role.....	27
Figure 33: User status on Admin Panel	28
Figure 34: Admin's Order Page.....	28
Figure 35: Order Status	29
Figure 36: Order Confirmed	29
Figure 37: Order Delivered.....	30
Figure 38: Order Cancelled.....	30
Figure 39: Order status confirmed on both Admin and User side	31
Figure 40: Password Length Error.....	31
Figure 41: Update Password with valid.....	32
Figure 42: Verifying with the updated password.....	32
Figure 43: S3 Bucket List	34
Figure 44: S3 Bucket (Frontend Hosting).....	34
Figure 45: CloudFront Distribution	35

Figure 46: Amazon Cognito Overview	35
Figure 47: Amazon Cognito - App Client.....	36
Figure 48: Amazon Cognito – Users	36
Figure 49: Amazon Cognito - Groups	37
Figure 50: Amazon API Gateway	37
Figure 51: API Gateway - Resources.....	38
Figure 52: API Gateway – Stages.....	38
Figure 53: API Gateway - Dashboard.....	39
Figure 54: AWS Lambda Functions List.....	39
Figure 55: AWS Lambda Functions (sos-products-handler).....	40
Figure 56: AWS Lambda Functions (sos-cart-handler).....	40
Figure 57: AWS Lambda Functions (sos-checkout-handler)	41
Figure 58: AWS Lambda Functions (sos-users-handler).....	41
Figure 59: AWS Lambda Functions (sos-notifications-handler).....	42
Figure 60: AWS Lambda Functions (sos-admin-handler).....	42
Figure 61: AWS Lambda Functions (sos-stripe-webhook)	43
Figure 62: AWS Lambda Functions (sos-orders-handler).....	43
Figure 63: Amazon DynamoDB Tables	44
Figure 64: Amazon DynamoDB (sos-orders).....	44
Figure 65: Amazon DynamoDB (sos-products)	45
Figure 66: Amazon DynamoDB (sos-orders-explore-item)	45
Figure 67: Amazon DynamoDB (sos-products-explore-items).....	46
Figure 68: Amazon SNS Topic.....	46
Figure 69: Amazon SES Identities.....	47
Figure 70: Amazon Lex - Overview	47
Figure 71: Amazon Lex - Intents	48
Figure 72: Amazon Lex – Conversation Flows	48
Figure 73: Amazon Lex – Conversations	49
Figure 74: CloudWatch Overview	49
Figure 75: CloudWatch - Log Management	50
Figure 76: CloudTrail Event History	50
Figure 77: Telegram Notification Output	51
Figure 78: AWS IAM - Users.....	52
Figure 79: AWS IAM Roles (sos-lambda-execution-role).....	52
Figure 80: Amazon KMS – AWS Managed Keys.....	53
Figure 81: Amazon KMS – Customer Managed Keys	53
Figure 82: AWS Secrets Manager - Stored Secrets	53
Figure 83: Terraform plan output	54
Figure 84: api-gateway - main.tf file	55
Figure 85: CloudFront Module - S3 Bucket, Versioning, and Encryption.....	55
Figure 86: CloudFront Module with Plan Output.....	56
Figure 87: Cognito Module - User Pool Configuration.....	56
Figure 88: DynamoDB Module - Products Table with GSIs.....	57
Figure 89: IAM Module - Lambda Role and Policy.....	58
Figure 90: Lambda Module - Shared Config and Function Definition	58
Figure 91: Monitoring Module - CloudTrail S3 Bucket and Policy	59

Figure 92: Notifications Module - SNS Topic and SES Identities	60
Figure 93: S3 Module - Assets and Uploads Buckets.....	60
Figure 94: Secrets Module - KMS Key and Policy	61
Figure 95: GitHub Actions Workflow	62

Introduction

The SEA of Style (SOS) project is a secure, cloud-based e-commerce platform developed to simulate a real-world online shopping system. The application enables users to browse products, create accounts, place orders, and receive notifications, while maintaining a strong focus on security, scalability, and performance. The system is built using Amazon Web Services (AWS) and follows a serverless architecture approach, eliminating the need for traditional server management. By leveraging managed services such as Amazon S3, Amazon CloudFront, Amazon API Gateway, AWS Lambda, and Amazon DynamoDB, the platform can automatically scale based on user demand while reducing operational complexity.

The architecture is structured into multiple layers, each responsible for a specific function, including frontend delivery, authentication, API processing, data storage, notification services, and monitoring. This separation of concerns improves maintainability and allows each component to operate independently while contributing to the overall system. Security is integrated throughout the design using Amazon Cognito for user authentication and AWS Identity and Access Management (IAM) for controlled access between services. In addition, AWS Key Management Service (KMS) is used to encrypt sensitive data at rest, ensuring data protection across the platform.

The SOS platform also incorporates additional features to enhance both functionality and user experience. Amazon Lex is integrated to provide a conversational chatbot interface, allowing users to interact with the system using natural language. Furthermore, the project demonstrates modern DevOps practices using Terraform and GitHub Actions for infrastructure automation. This Infrastructure as Code (IaC) approach enables consistent, repeatable, and secure deployment of cloud resources without relying on manual configuration.

Overall, the project highlights how a modern cloud-native application can be designed using serverless and event-driven principles. It demonstrates the practical implementation of scalable architecture, secure authentication mechanisms, efficient data management, and automated infrastructure deployment, reflecting real-world industry practices.

System Overview

The SOS platform is designed as a multi-layered cloud application, where each component is responsible for handling specific aspects of user interaction and system operations. When a user accesses the website, the request is first routed through Amazon CloudFront, which delivers static frontend content stored in Amazon S3. By using a global content delivery network (CDN), the platform ensures low latency, faster load times, and consistent user experience across different geographic locations.

User authentication is managed using Amazon Cognito, which handles user registration, login, and identity verification. Once authenticated, users receive a secure JSON Web Token (JWT) that is used to authorize all subsequent API requests. All communication between the frontend and backend services is routed through Amazon API Gateway, which acts as the central entry point for the application. The API Gateway validates incoming requests and forwards them to AWS Lambda functions, where the core business logic of the application is executed.

In addition to standard API interactions, the system also integrates Amazon Lex to provide a chatbot-based interface. This allows users to interact with the application using conversational inputs, which are processed and routed to AWS Lambda functions for further handling. This feature enhances usability and introduces an alternative interaction method beyond traditional web navigation.

The application data, including product details and customer orders, is stored in Amazon DynamoDB, which provides a highly scalable and low-latency NoSQL database solution. For event-driven communication and notifications, the platform uses Amazon SNS together with a Telegram bot integration to deliver real-time purchase alerts to administrators. Monitoring and logging are handled by Amazon CloudWatch, which collects system logs and performance metrics, while AWS CloudTrail records control plane activities for auditing and security purposes.

In addition to the application runtime components, the platform also incorporates an infrastructure automation layer using Terraform. This is integrated with a CI/CD pipeline powered by GitHub Actions, which uses OpenID Connect (OIDC) to securely assume AWS IAM roles without storing credentials. Terraform manages infrastructure provisioning, with its state stored in Amazon S3 and state locking handled by DynamoDB. This setup enables automated, consistent, and secure deployment of resources, supporting modern DevOps practices. Together, these components form a cohesive and scalable system that supports the complete lifecycle of the application, from user interaction and data processing to monitoring, security, and infrastructure management.

SOS-Sea of Style Infrastructure Diagram

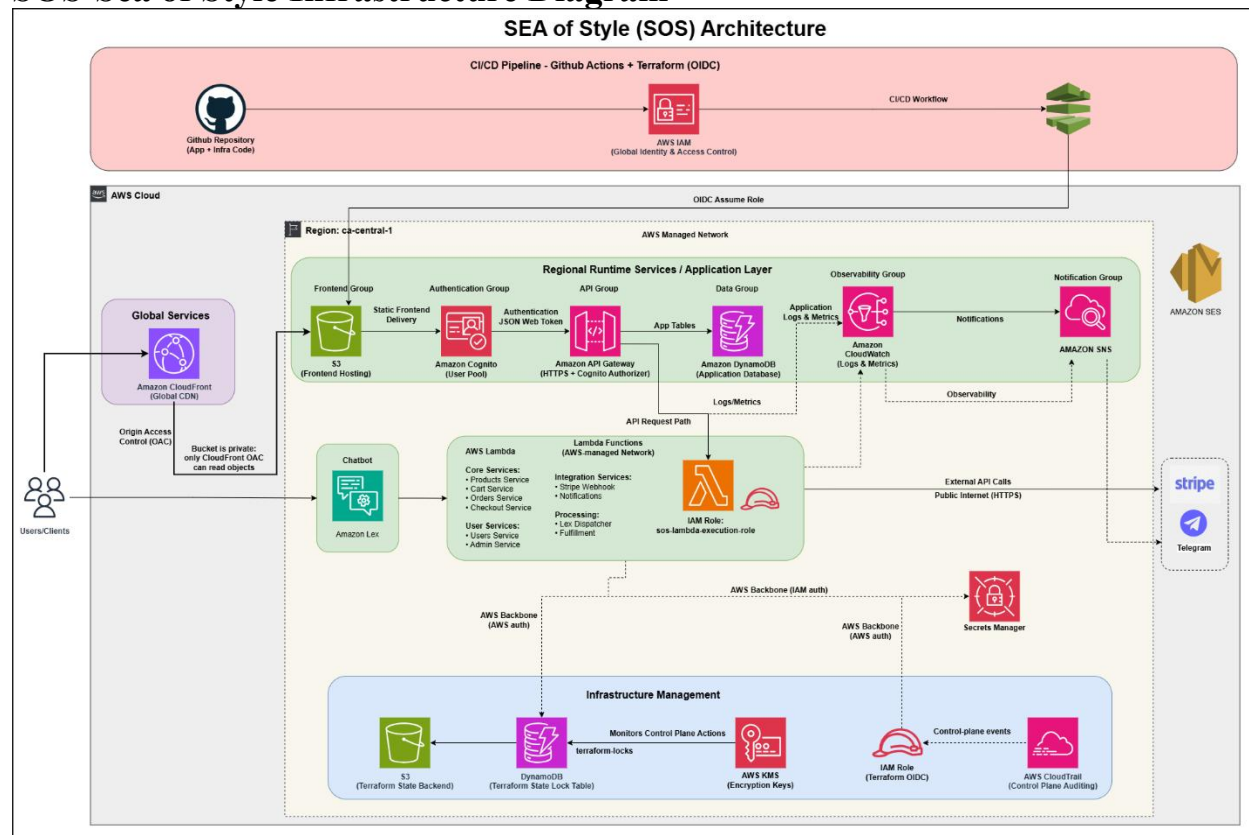


Figure 1: Architecture Diagram.

Figure 1 presents the overall architecture of the SOS platform and illustrates how various AWS services interact to deliver a secure, scalable, and serverless e-commerce solution. The system is organized into multiple logical layers, including frontend delivery, authentication, API processing, data storage, observability, notification services, and infrastructure management.

User requests are first handled by Amazon CloudFront, which delivers static frontend content from a private Amazon S3 bucket. Authentication is managed through Amazon Cognito, while backend communication is routed through Amazon API Gateway to AWS Lambda functions that execute the core application logic. The system also integrates Amazon Lex to provide a chatbot interface, enabling users to interact with the platform using natural language.

Application data is stored in Amazon DynamoDB, and notifications are handled through Amazon SNS and Telegram bot integration for real-time admin alerting. Monitoring and logging are managed by Amazon CloudWatch, with AWS CloudTrail providing auditing capabilities. In addition, the architecture includes a CI/CD pipeline using GitHub Actions and Terraform, which automates infrastructure provisioning through secure IAM role assumptions using OIDC. Overall, the diagram highlights a fully serverless, event-driven design that emphasizes scalability, security, and operational efficiency.

Explanation of All Services

Frontend Layer

The frontend of the SOS application is hosted on Amazon S3, where all static assets such as HTML, CSS, and JavaScript files are stored. To improve performance and user experience, Amazon CloudFront is used as a global content delivery network (CDN). CloudFront caches content at edge locations, reducing latency and ensuring faster access for users across different regions. For security purposes, the S3 bucket is configured as private, and access is restricted using Origin Access Control (OAC). This ensures that users cannot directly access the storage layer, and all requests must pass through CloudFront, adding an additional layer of protection.

Authentication Layer

User authentication and identity management are handled using Amazon Cognito. It provides secure user registration, login, and session management without requiring custom authentication logic. Once a user successfully authenticates, Cognito generates a JSON Web Token (JWT), which is used to authorize API requests. Amazon API Gateway integrates with Cognito through a built-in authorizer, ensuring that only authenticated users can access protected backend resources. This approach enhances security by validating user identity before allowing access to application services.

API & Compute Layer

Amazon API Gateway serves as the central entry point for all backend requests. It receives incoming API calls from the frontend and validates them using Cognito authentication. After validation, API Gateway routes the requests to AWS Lambda functions, where the business logic of the application is executed. The application follows a microservices-based approach using multiple Lambda functions. These functions handle core operations such as product management, cart processing, order handling, user services, and administrative tasks. In addition, integration services such as Stripe webhooks and notification processing are also managed through Lambda

functions. Amazon Lex is also integrated within this layer to provide a conversational chatbot interface. User inputs from the chatbot are processed by Lex and forwarded to Lambda functions, enabling dynamic and interactive communication with the system

Data Layer

Amazon DynamoDB is used as the primary database for the SOS platform. It stores application data such as product information, customer orders, and user-related data. DynamoDB is a fully managed NoSQL database that offers high availability, low latency, and automatic scaling. This makes it well-suited for handling dynamic workloads and real-time application requirements, ensuring that the system can efficiently manage increasing user traffic without performance degradation.

Notification Layer

The SOS platform uses Amazon SNS to handle event-driven notifications within the system. When a user completes a purchase, an event is generated and published through SNS, which is then processed by the notification workflow and forwarded to a Telegram bot. This allows administrators to receive real-time purchase alerts in the admin Telegram group. This event-driven approach ensures reliable and scalable communication.

Observability & Monitoring

Amazon CloudWatch is used to monitor system performance and collect logs from various services such as Lambda and API Gateway. It provides insights into application behavior, helping identify issues and track performance metrics. AWS CloudTrail complements this by recording all control plane activities, such as resource creation and configuration changes. This provides an audit trail that supports security, compliance, and troubleshooting

Security & IAM

Security within the SOS platform is implemented using AWS Identity and Access Management (IAM), which enforces role-based access control across services. Each component is assigned only to the permissions it requires, following the principle of least privilege. AWS Key Management Service (KMS) is used to encrypt sensitive data at rest, ensuring that stored information remains protected. Additionally, AWS Secrets Manager is used to securely store and manage sensitive credentials such as API keys, eliminating the need for hardcoded secrets in the application.

DevOps & Infrastructure Management (CI/CD + Terraform)

The SOS platform incorporates modern DevOps practices using Terraform and GitHub Actions. Infrastructure is defined and managed using Terraform, enabling consistent and repeatable deployment of cloud resources. The CI/CD pipeline is implemented using GitHub Actions, which uses OpenID Connect (OIDC) to securely assume AWS IAM roles without storing access credentials. Terraform state is stored in Amazon S3, while DynamoDB is used for state locking to prevent conflicts during deployment. This setup ensures automated, secure, and efficient infrastructure management, aligning with industry best practices for Infrastructure as Code (IaC).

Workflow of SOS Web Application

The SOS platform follows a complete end-to-end workflow that simulates a real-world e-commerce system. The system supports two main operational flows: the customer workflow, which covers the user journey from accessing the website to completing a purchase, and the admin workflow, which handles product and user management operations.

Customer Workflow (End-User Journey)

User Access and Website Loading

When a user enters the website URL, the request is first routed to Amazon CloudFront. CloudFront serves the frontend content from edge locations, ensuring fast and reliable access. The static files, including HTML, CSS, and JavaScript, are stored in a private Amazon S3 bucket and are accessed securely using Origin Access Control (OAC). This prevents direct access to the storage layer and ensures that all content delivery is handled through CloudFront.

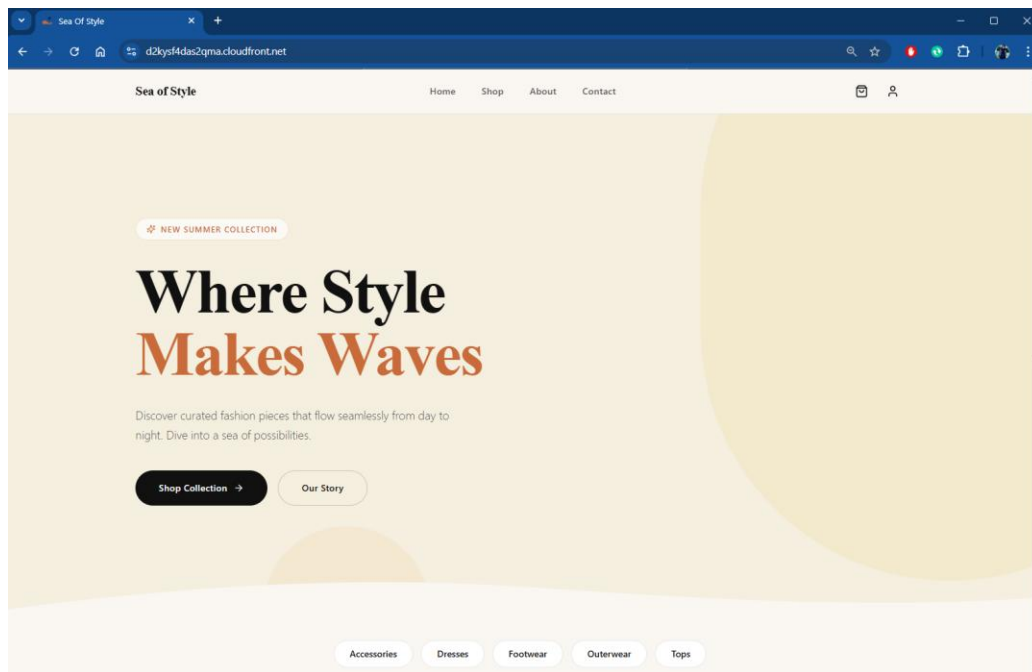


Figure 2: Home Page

Product Browsing

Once the website is loaded, users can browse available products. When product data is required, the frontend sends a request to Amazon API Gateway. The request is forwarded to AWS Lambda functions, which retrieve product information from Amazon DynamoDB and return it to the frontend.

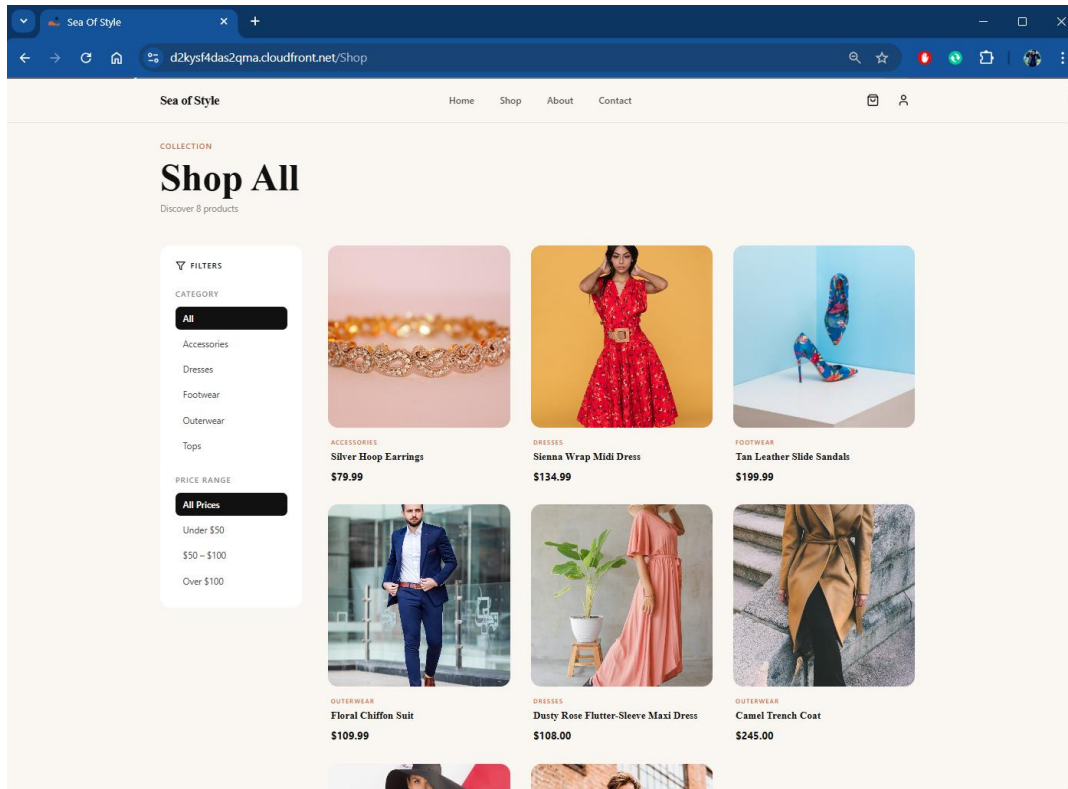


Figure 3: Shop Page

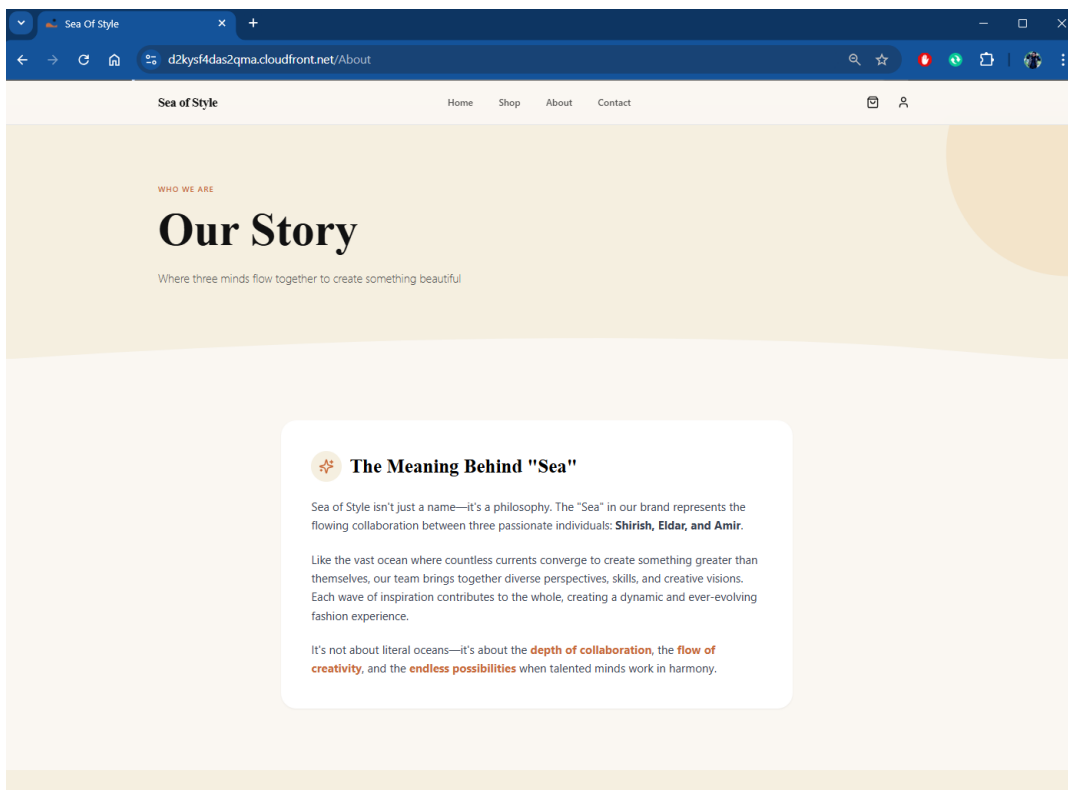
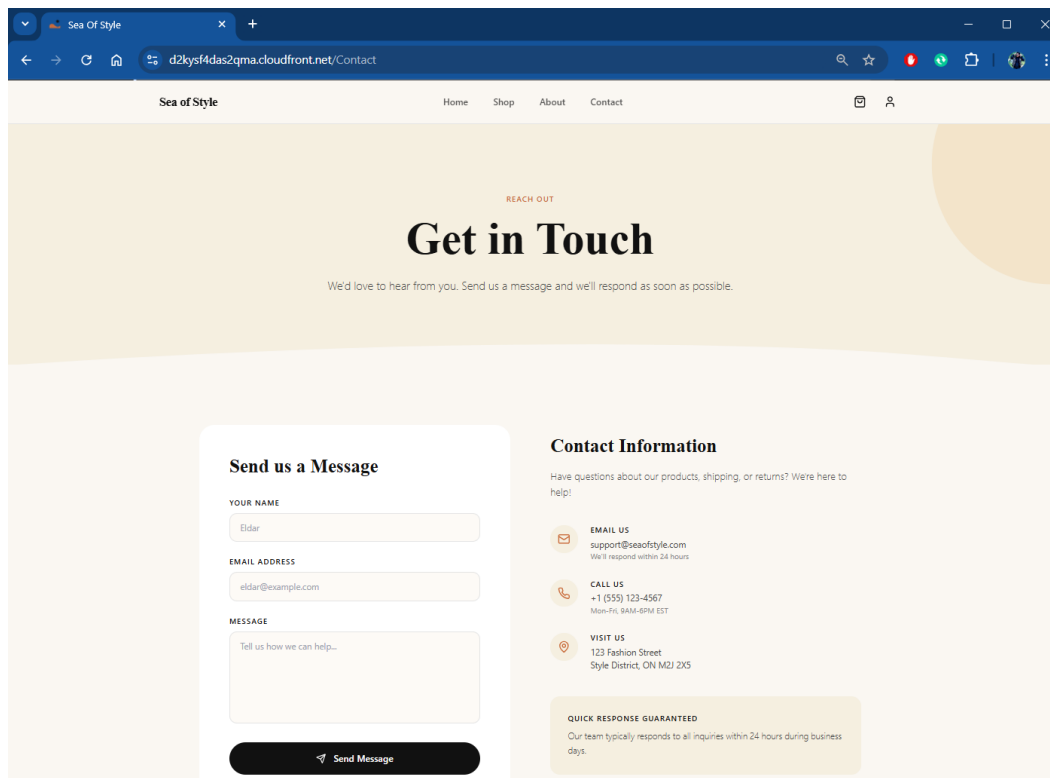


Figure 4: About Page

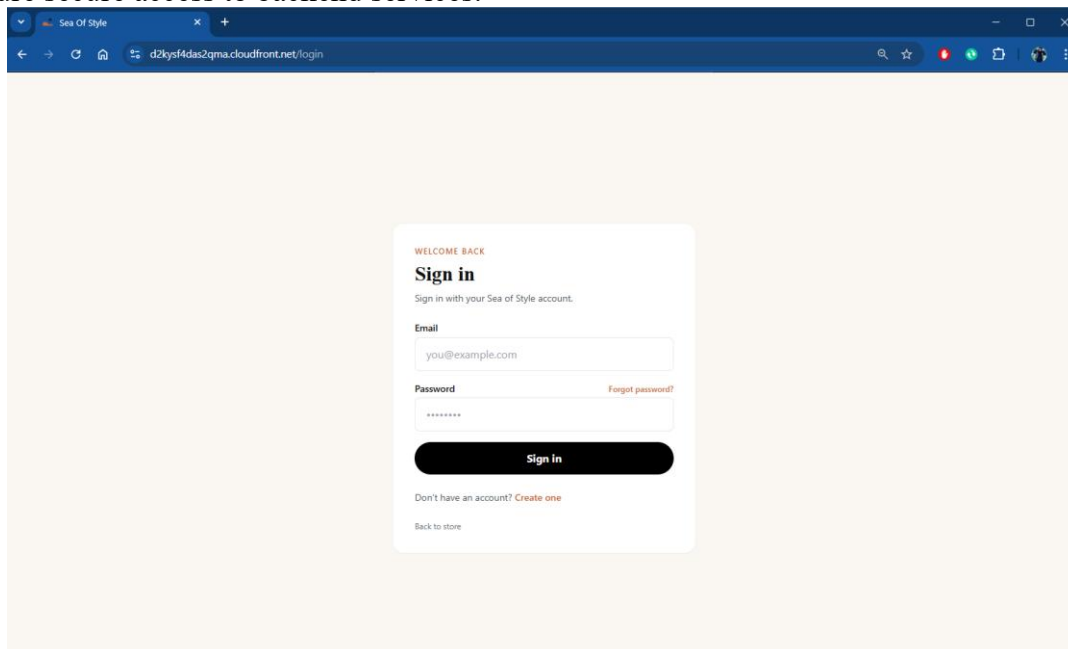


The screenshot shows a web browser window with the URL `d2kysf4das2qma.cloudfront.net/Contact`. The page is titled "Sea of Style" and has a navigation bar with links for Home, Shop, About, and Contact. The main heading is "Get in Touch" with the subtext "REACH OUT" and "We'd love to hear from you. Send us a message and we'll respond as soon as possible." Below this, there are two main sections: "Send us a Message" and "Contact Information". The "Send us a Message" section contains a form with fields for "YOUR NAME" (filled with "Eldar"), "EMAIL ADDRESS" (filled with "eldar@example.com"), and "MESSAGE" (with placeholder text "Tell us how we can help..."). A "Send Message" button is at the bottom of the form. The "Contact Information" section includes a heading, a brief description, and three contact methods: "EMAIL US" (support@seaofstyle.com), "CALL US" (+1 (555) 123-4567), and "VISIT US" (123 Fashion Street). A "QUICK RESPONSE GUARANTEED" note is also present.

Figure 5: Contact Page

User Registration and Authentication

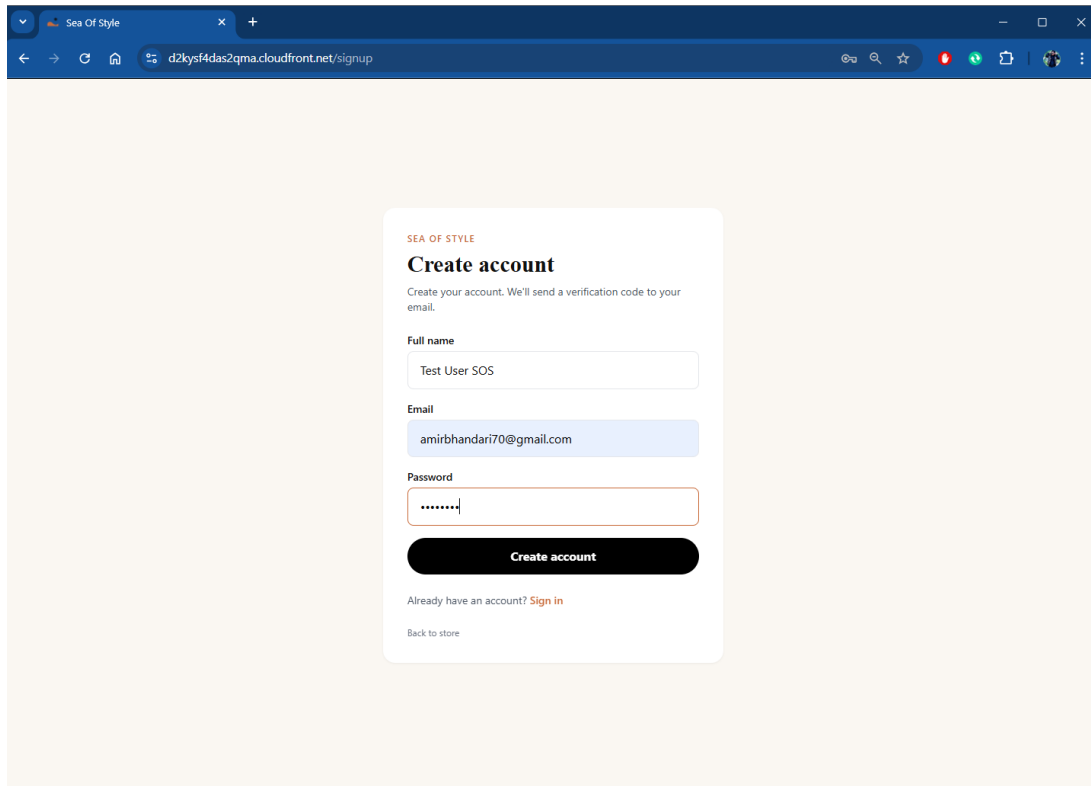
To access protected features such as placing an order, users must create an account or log in. Amazon Cognito manages the authentication process, including user registration, login, and verification. After successful authentication, a JSON Web Token (JWT) is generated and returned to the frontend. This token is included in subsequent API requests and is validated by API Gateway to ensure secure access to backend services.



The screenshot shows a web browser window with the URL `d2kysf4das2qma.cloudfront.net/login`. The page is titled "Sea of Style" and has a navigation bar with links for Home, Shop, About, and Contact. The main heading is "Sign in" with the subtext "WELCOME BACK" and "Sign in with your Sea of Style account." Below this, there is a form with fields for "Email" (filled with "you@example.com") and "Password" (filled with "*****"). A "Forgot password?" link is next to the password field. A "Sign in" button is at the bottom of the form. Below the button, there is a link for "Don't have an account? Create one" and a link for "Back to store".

Figure 6: Login Page

Create an Account



SEA OF STYLE

Create account

Create your account. We'll send a verification code to your email.

Full name

Email

Password

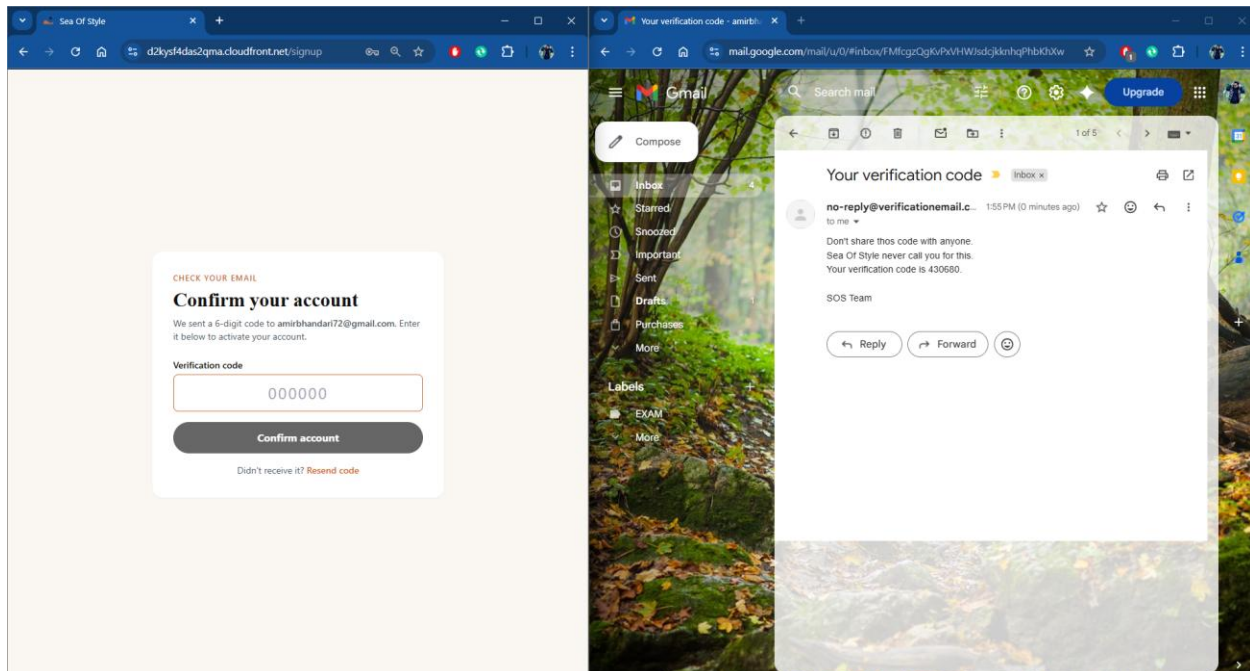
Create account

Already have an account? [Sign in](#)

[Back to store](#)

Figure 7: Sign up Page

Verification Code



CHECK YOUR EMAIL

Confirm your account

We sent a 6-digit code to amirbhandari72@gmail.com. Enter it below to activate your account.

Verification code

Confirm account

Didn't receive it? [Resend code](#)

Gmail

Compose

Inbox

Starred

Snoozed

Important

Sent

Drafts

Purchases

More

Labels

EXAM

More

Your verification code

no-reply@verificationemail.com 1:55 PM (3 minutes ago)

to me

Don't share this code with anyone. Sea Of Style never call you for this. Your verification code is 430680.

SOS Team

[Reply](#) [Forward](#) [More](#)

Figure 8: Verification Code

Account Confirmed

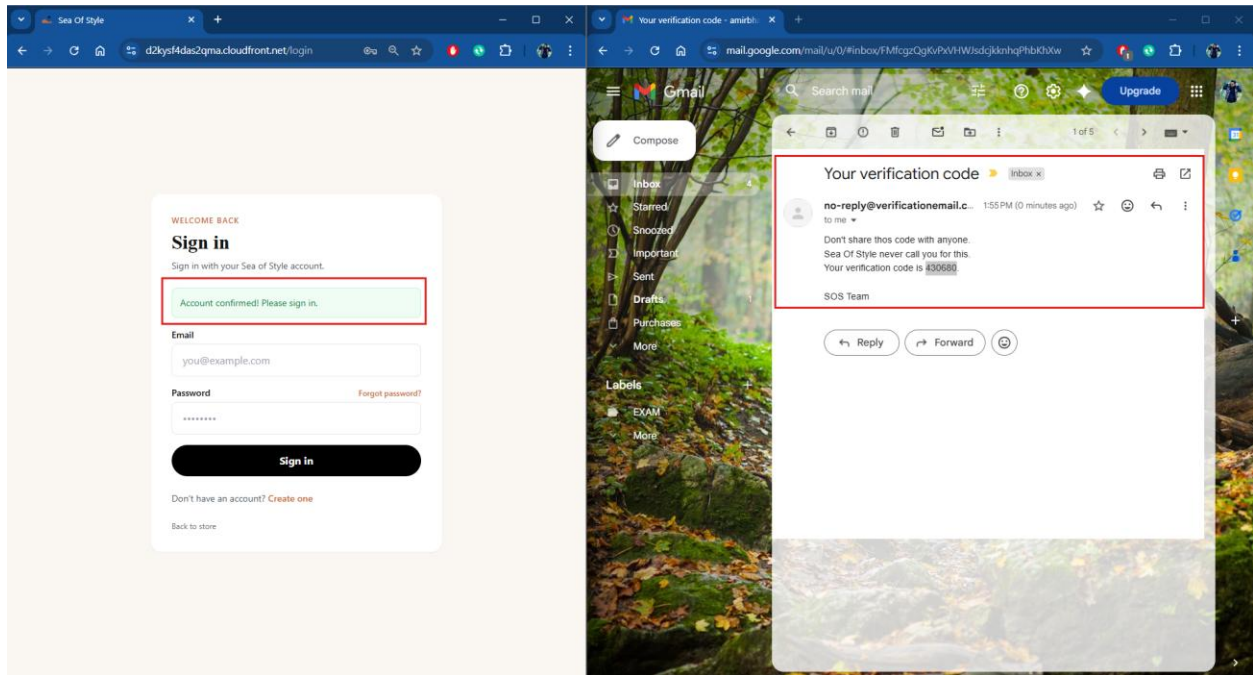


Figure 9: Account Confirmed

User Profile

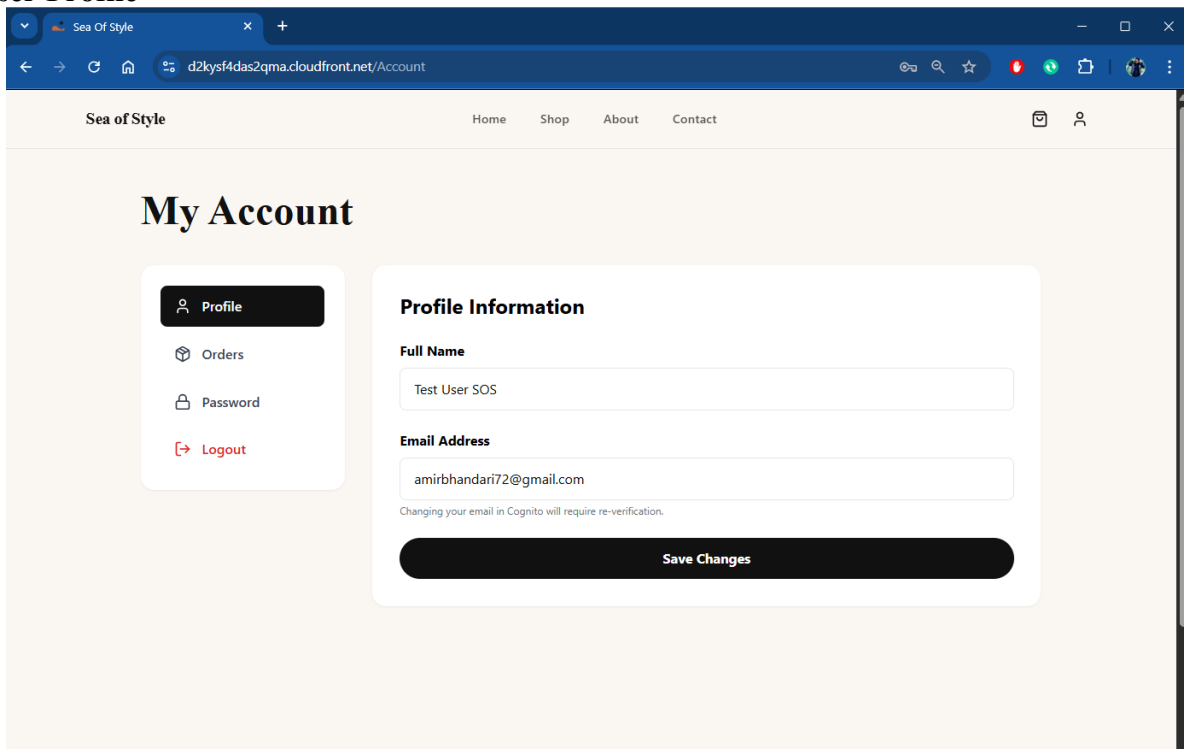


Figure 10: User Profile Dashboard

User Actions and Cart

Authenticated users can perform actions such as adding items to the cart, viewing their profile, and managing account settings. Each request includes the JWT token, which is validated before being processed by Lambda functions. These functions handle business logic such as updating cart data and retrieving user-specific information from DynamoDB.

Order History

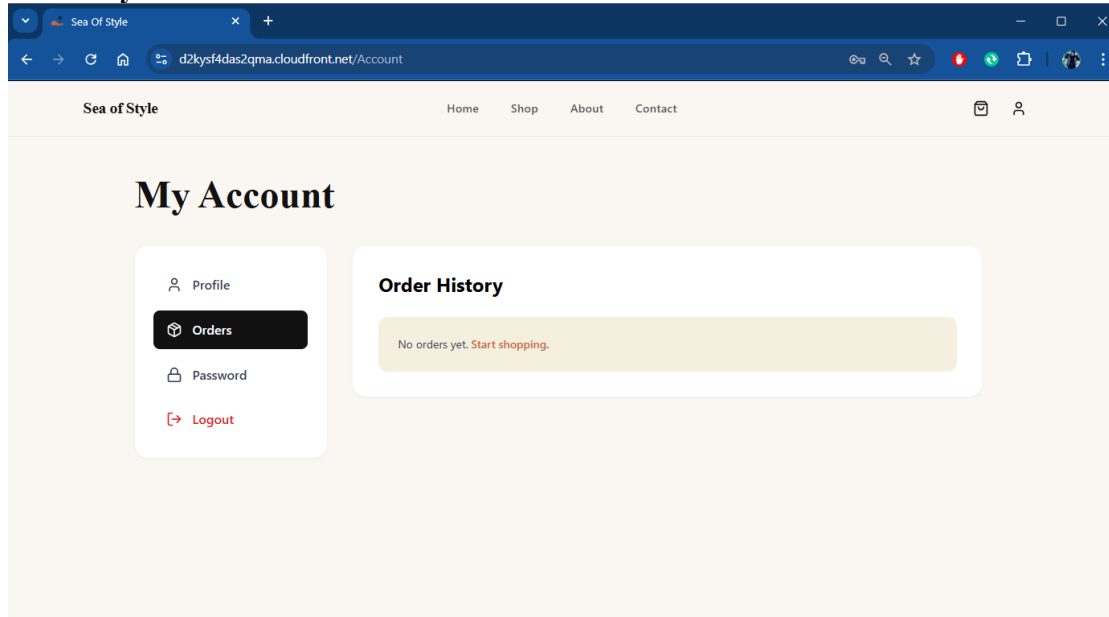


Figure 11: Order History

Change Password

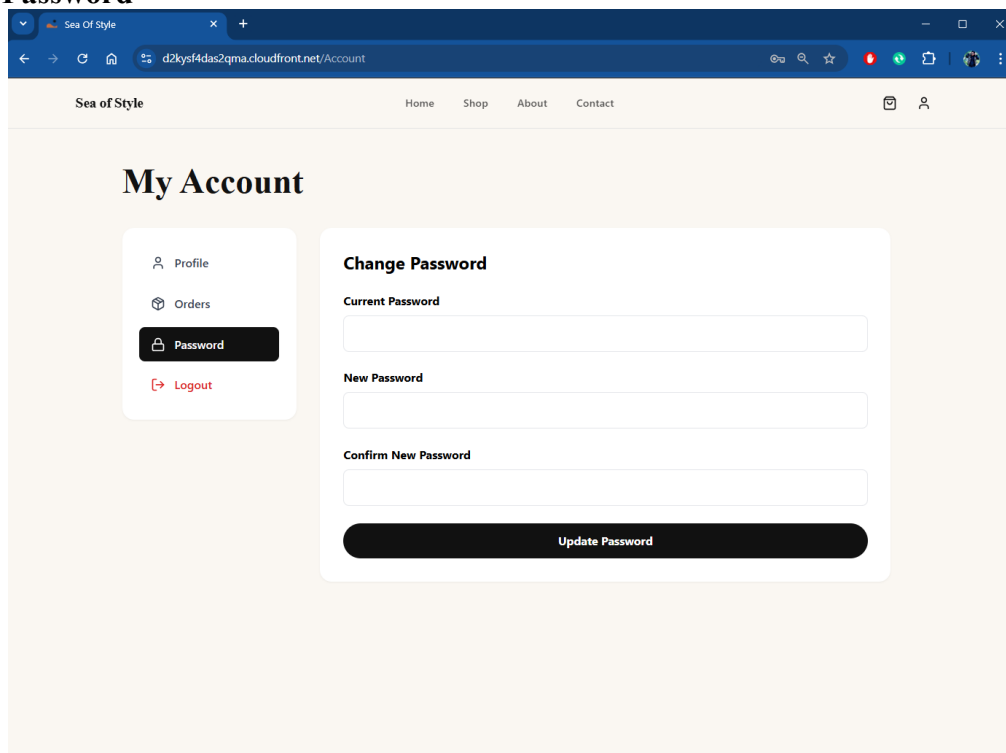


Figure 12: Change Password

Product Details from user side

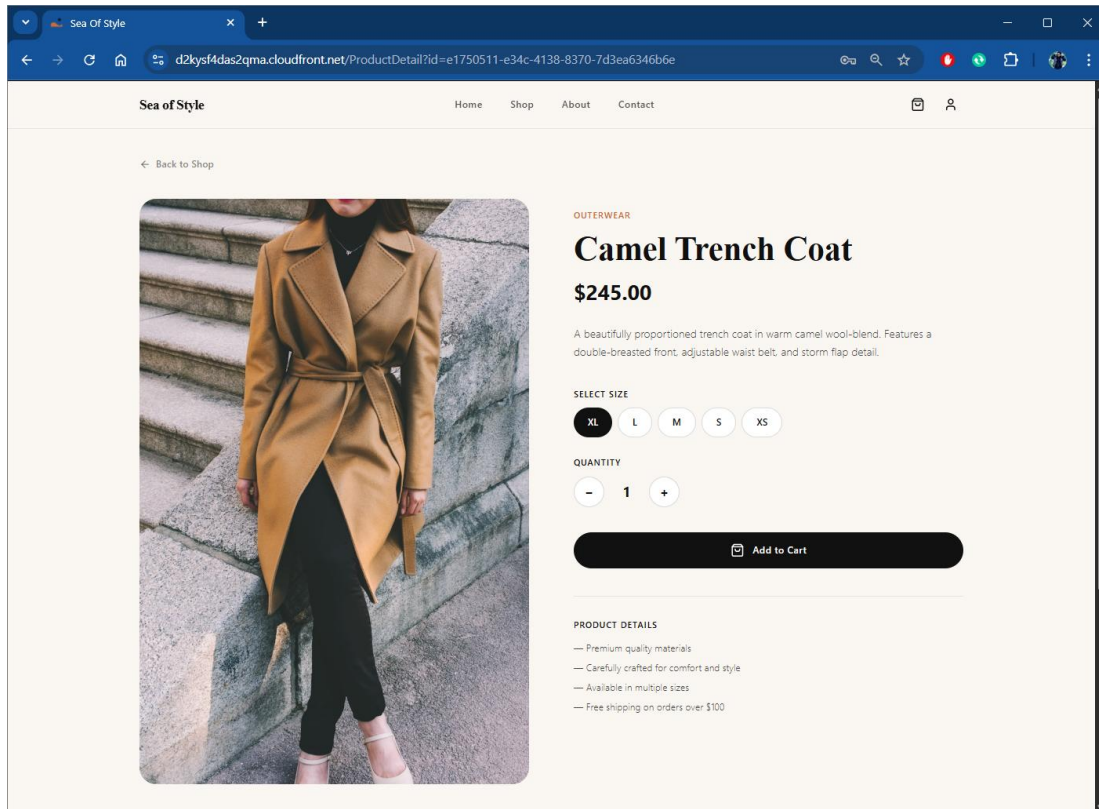


Figure 13: Product Information

Add to Cart

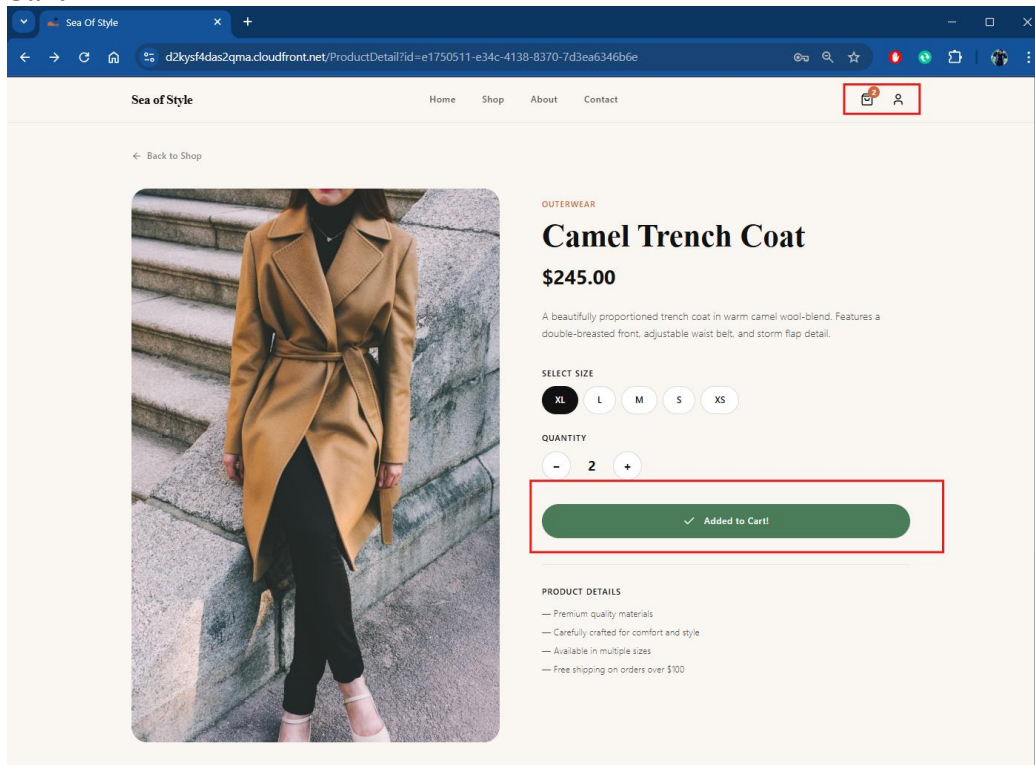


Figure 14: Add to Cart

Shopping Cart Page

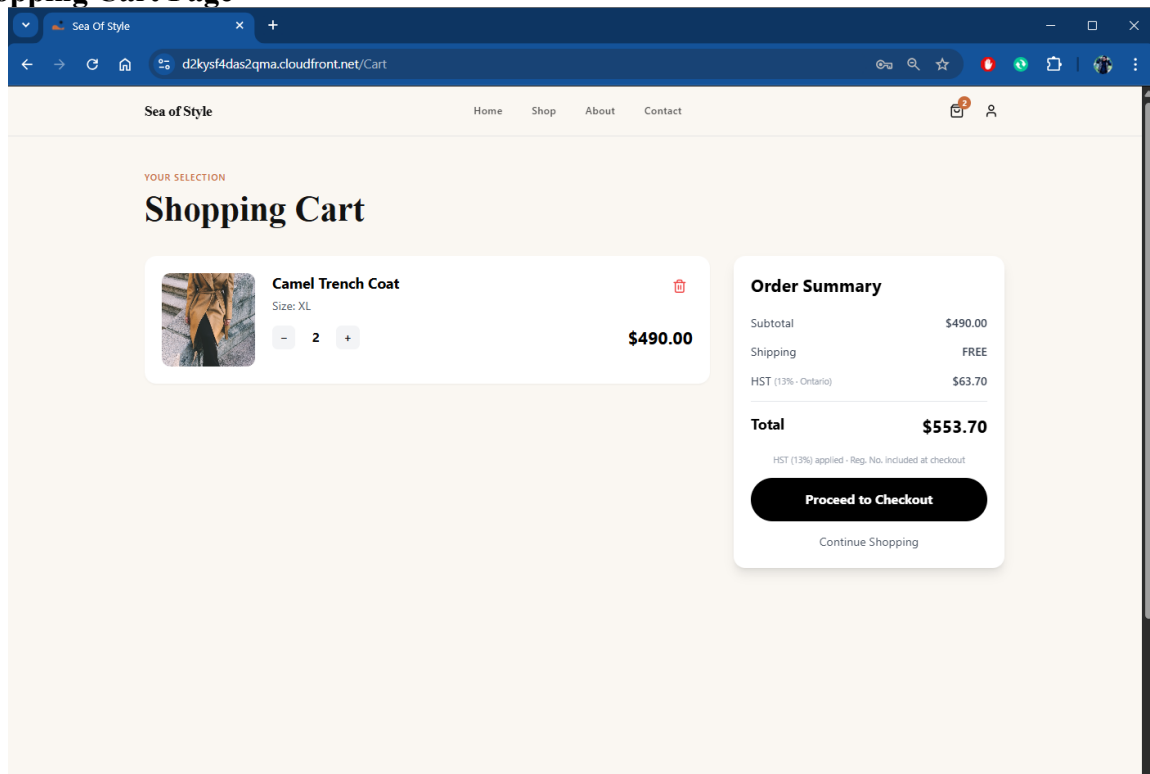


Figure 15: Shopping Cart

Finalizing the order with details

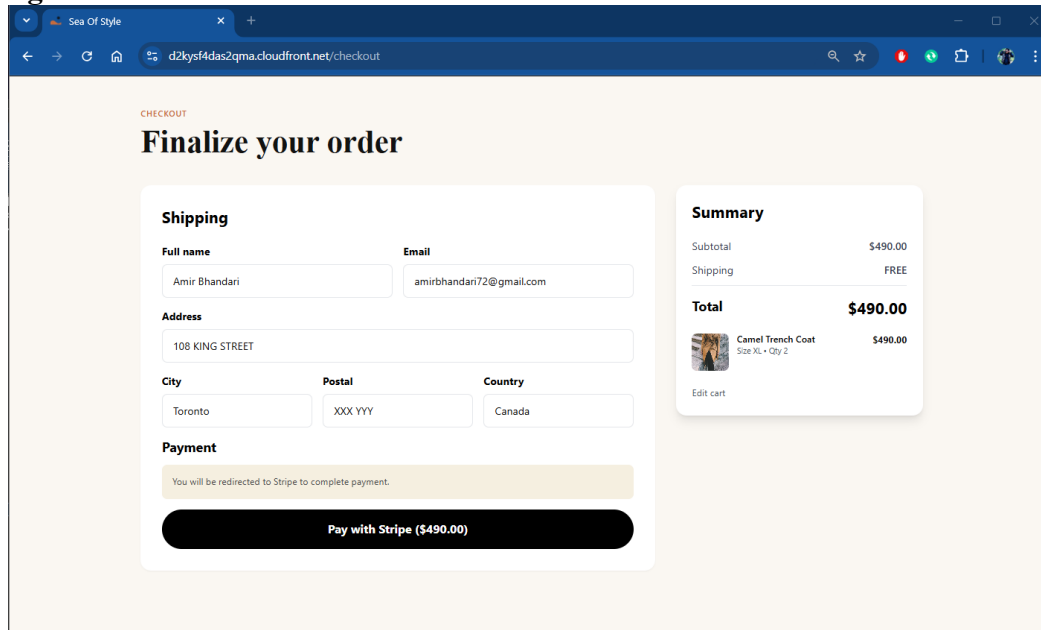
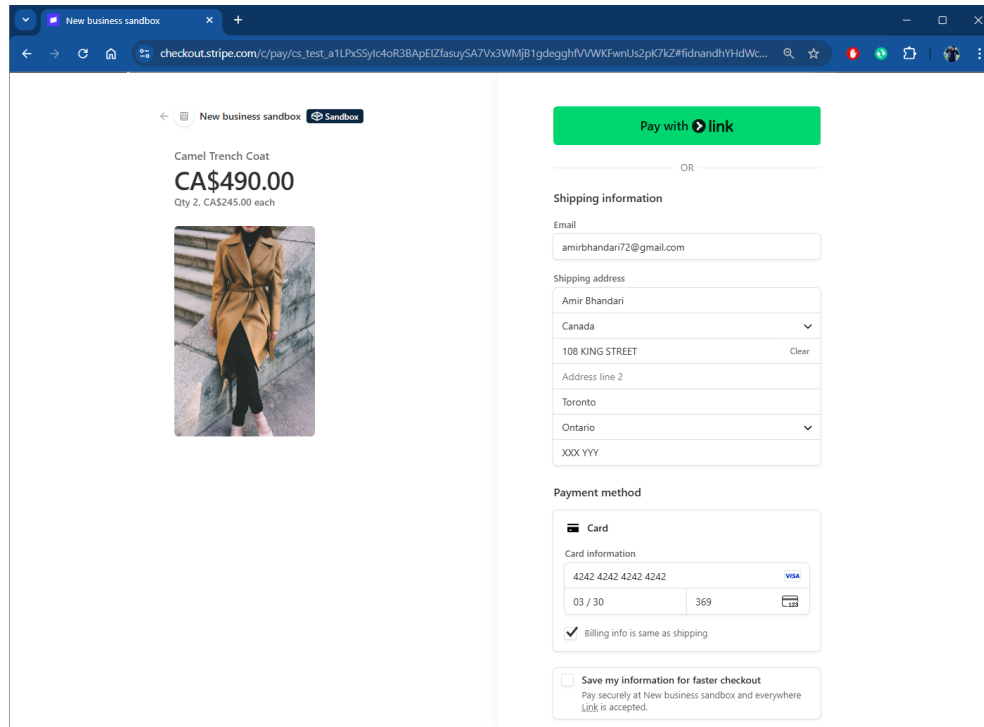


Figure 16: Shipping Page

Checkout and Payment Processing (Stripe Integration)

When the user proceeds to checkout, the application integrates with Stripe to handle payment processing. The user is redirected to Stripe's secure payment page, ensuring that sensitive financial information is not handled or stored within the SOS system. After the payment is completed, Stripe generates a webhook event and sends it back to the application.

Payment with STRIPE



The screenshot shows the Stripe checkout interface. On the left, the product is a 'Camel Trench Coat' priced at 'CA\$490.00' (Qty 2, CA\$245.00 each). A photo of the coat is displayed. On the right, there's a 'Pay with link' button. Below it, the 'Shipping information' section includes fields for Email (amirbhandari72@gmail.com), Shipping address (Amir Bhandari, Canada, 108 KING STREET, Toronto, Ontario), and a 'Clear' button. The 'Payment method' section shows a 'Card' option with card information (4242 4242 4242 4242, 03 / 30, 369) and a 'Billing info is same as shipping' checkbox. At the bottom, there's a checkbox to 'Save my information for faster checkout'.

Figure 17: Payment with Stripe

Payment Successful

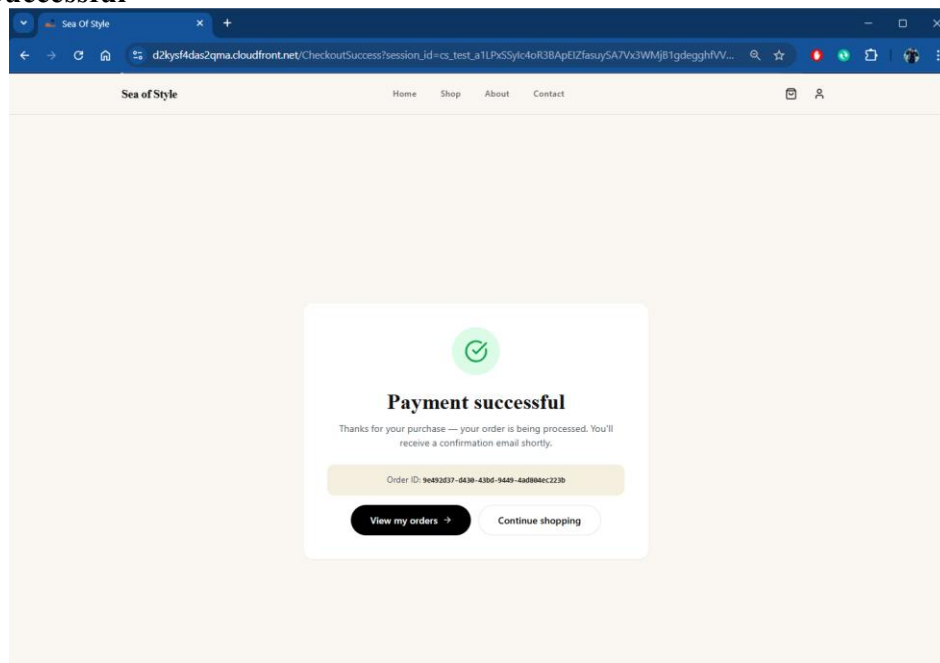


Figure 18: Payment Successful

Order Processing via Webhook

The Stripe webhook is received through Amazon API Gateway, which triggers a Lambda function. This function verifies the payment status and processes the order. Once confirmed, order details such as user information, purchased products, and transaction status are stored in DynamoDB. This ensures that only successful transactions are recorded in the system.

Order History Details

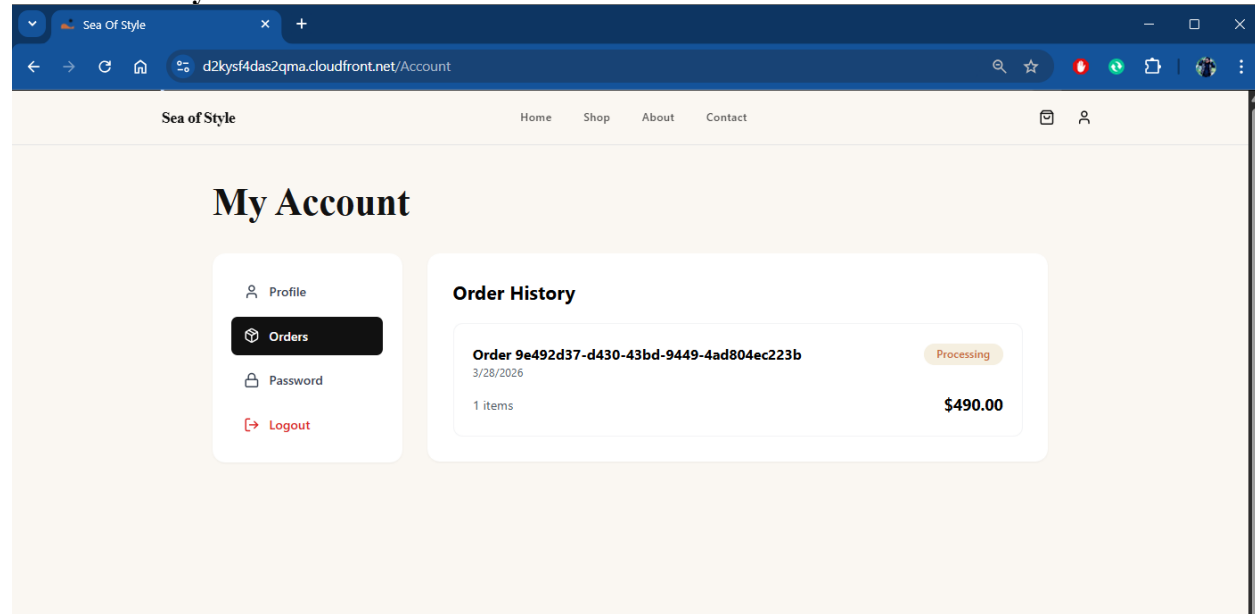


Figure 19: Order History

Purchase Notification (Admin Alerting)

After a user successfully completes a purchase, the system triggers a real-time notification workflow to inform administrators. Instead of using email-based notifications, the SOS platform sends alerts to a Telegram group for faster and more direct communication. Once the payment is completed through Stripe, a webhook event is sent to Amazon API Gateway, which triggers an AWS Lambda function to process the order details. The event is then published via Amazon SNS and forwarded to a Telegram bot using webhook integration, delivering the notification to the admin group. The message includes key information such as order ID, payment status, total amount, and customer details, allowing administrators to monitor transactions instantly.

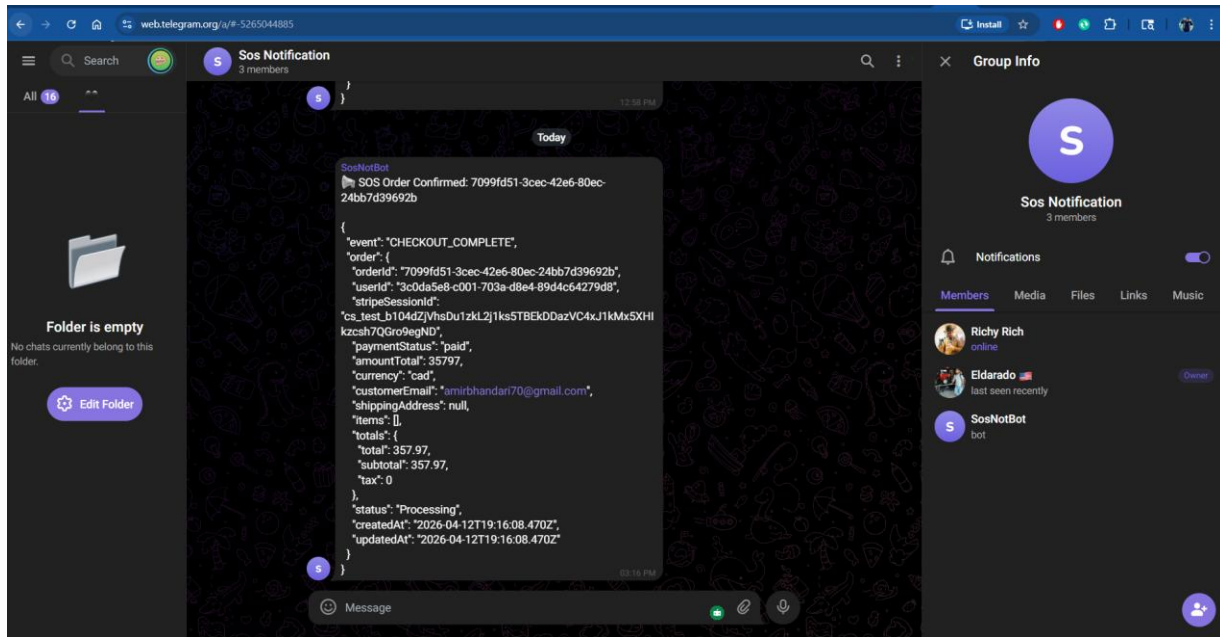


Figure 20: Telegram Notification

Chatbot Interaction (Amazon Lex)

In addition to traditional navigation, users can interact with the system using a chatbot powered by Amazon Lex. User queries are processed through Lex and forwarded to AWS Lambda functions, which generate appropriate responses. This provides an alternative and interactive way for users to access information and perform actions within the application.

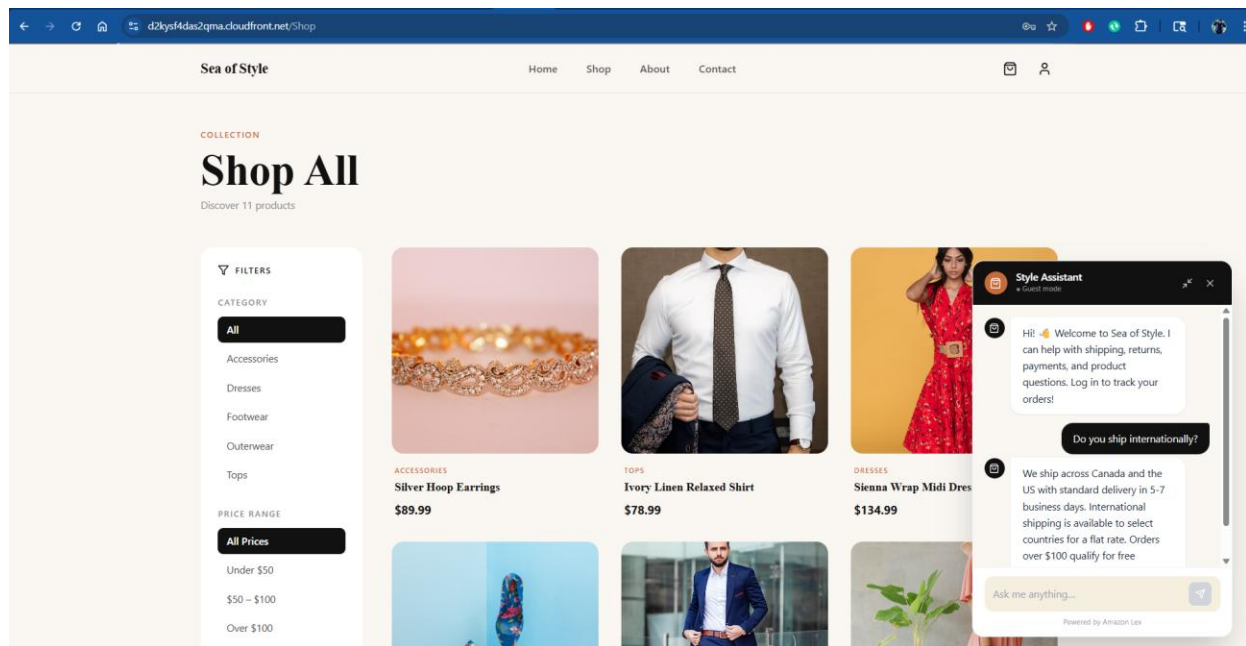


Figure 21: Amazon Lex - User Interaction

Logging, Monitoring and Auditing

Throughout the entire workflow, system activities are continuously monitored. Amazon CloudWatch collects logs and performance metrics from services such as Lambda and API Gateway, enabling real-time monitoring and troubleshooting. Additionally, AWS CloudTrail records all control plane operations, providing an audit trail for security and compliance purposes.

User's Password Recovery

If a user forgets their password, the system provides a secure password recovery mechanism through Amazon Cognito. From the login page, the user can select the “Forgot Password” option, which initiates the password reset process. The user is prompted to enter their registered email address, after which Cognito sends a verification code via email. The user then enters the received code along with a new password that meets the defined password policy. Once verified, the password is successfully updated, and the user can log in using the new credentials.

This process ensures secure account recovery without exposing sensitive information and maintains the integrity of user authentication.

Forget Password

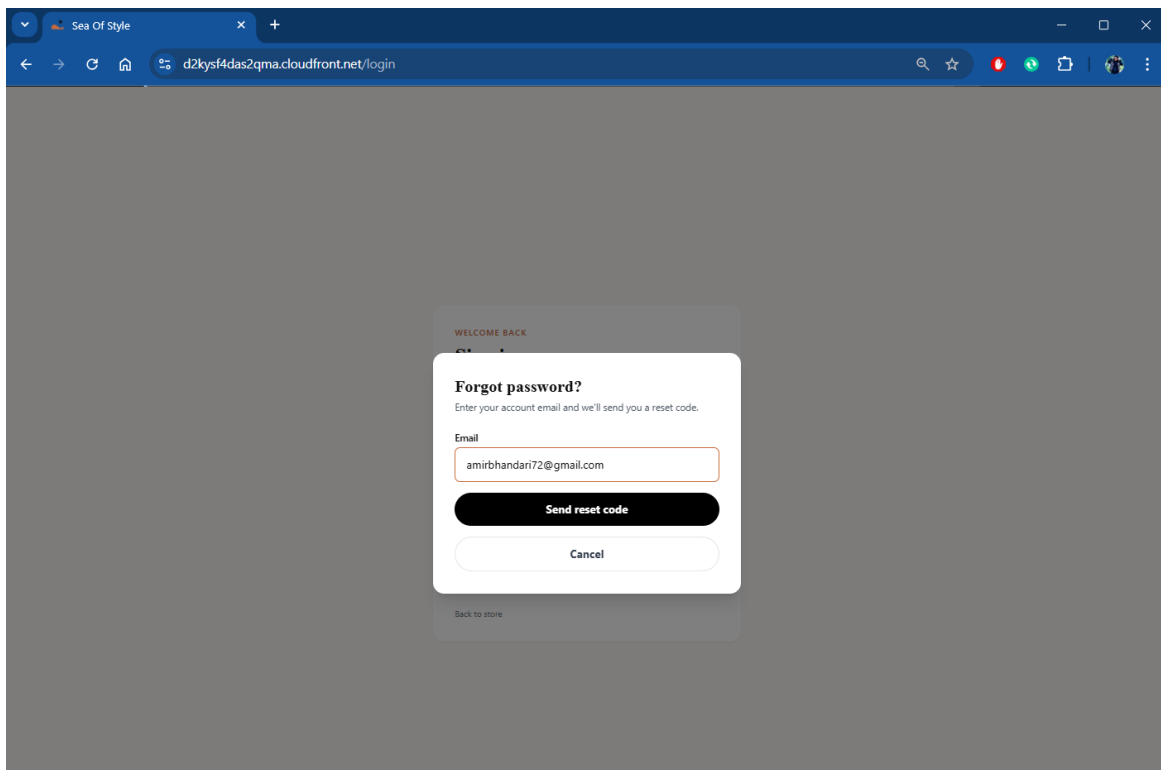


Figure 22: Forget Password

Received an email while forgetting the password

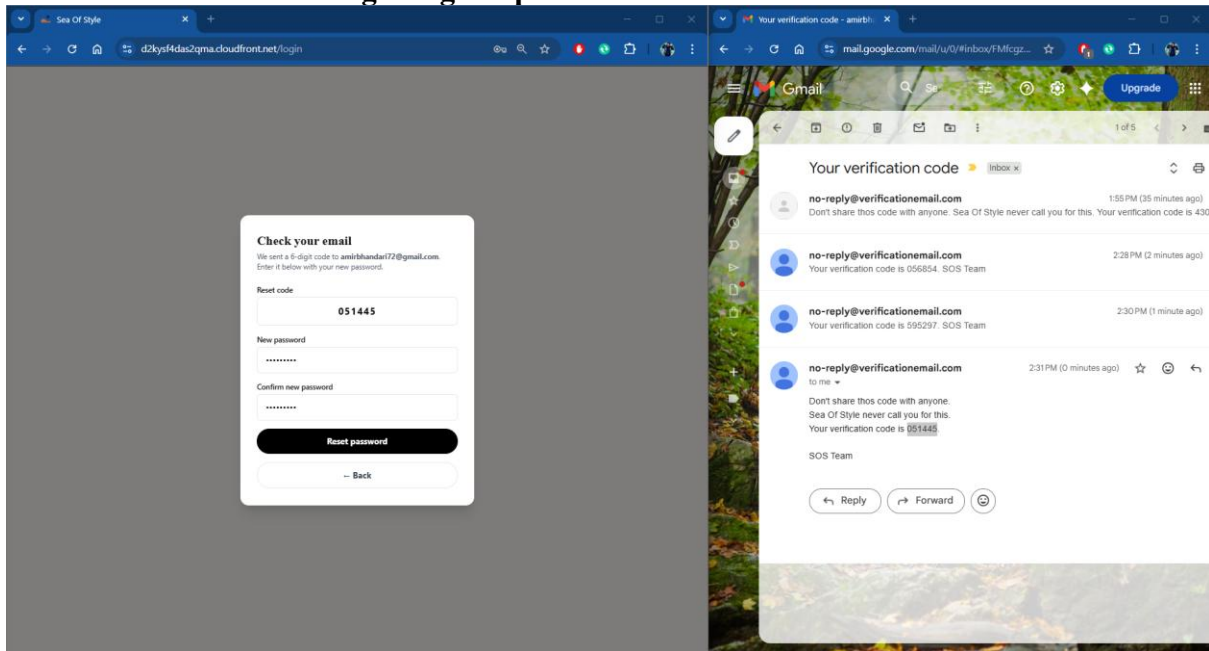


Figure 23: Verification Code for forgetting the password

Password Reset

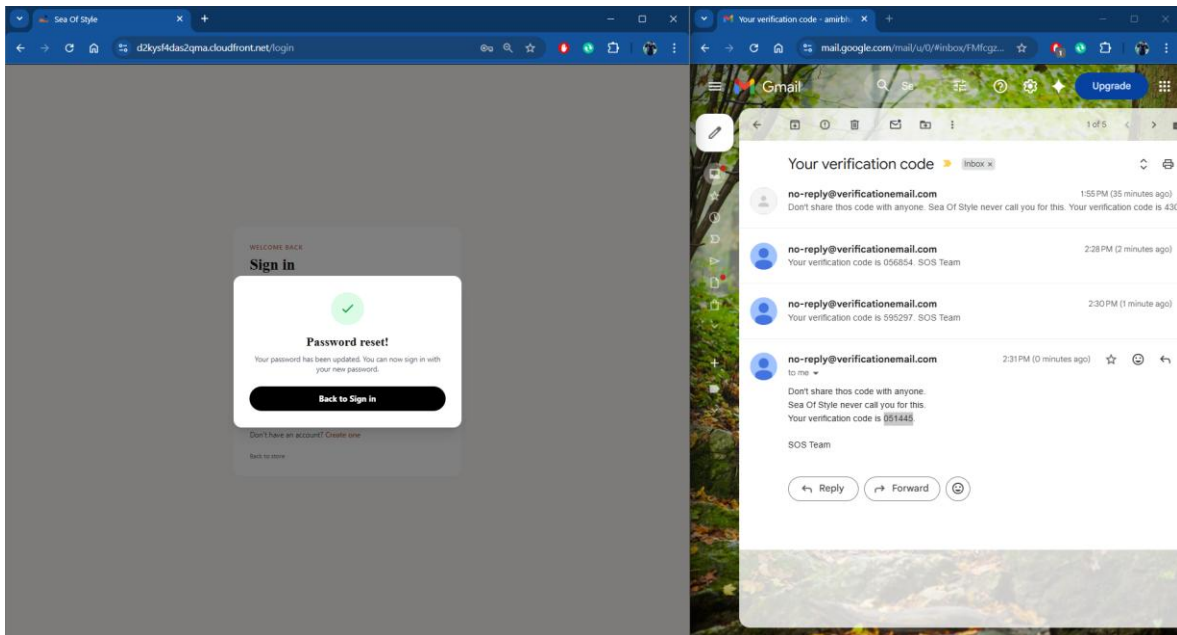


Figure 24: Password Reset

Tried with previous password

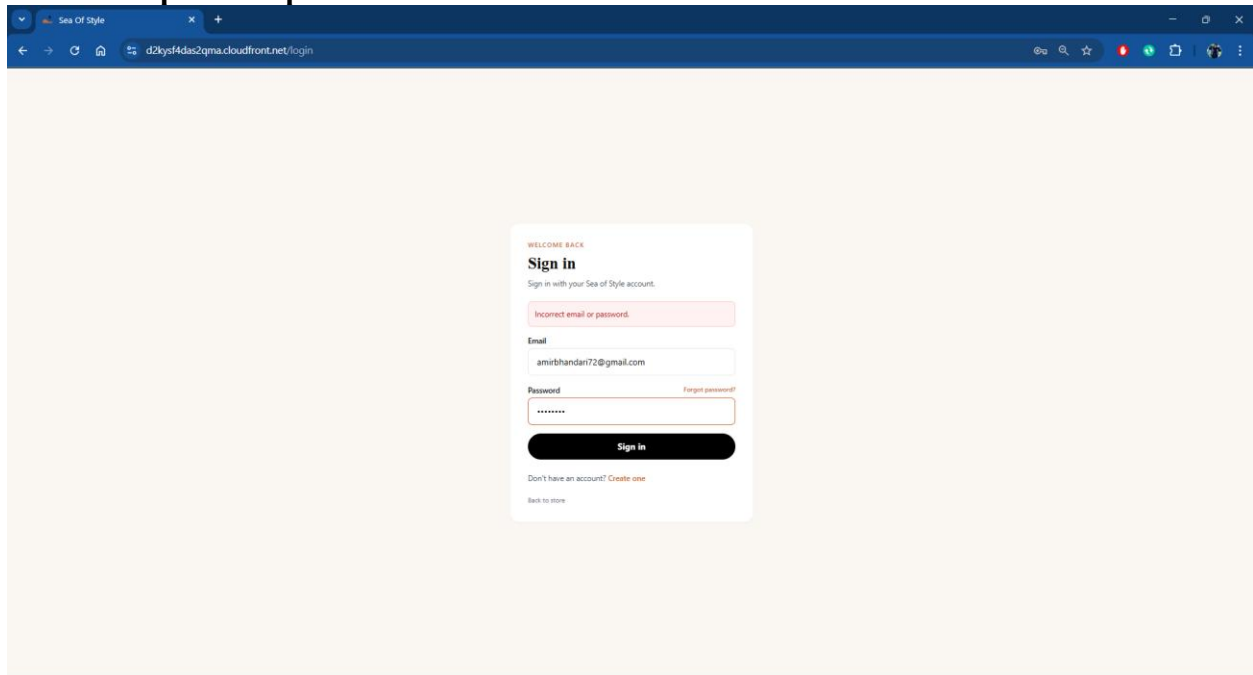


Figure 25: Invalid Username or Password

Password Policy should match

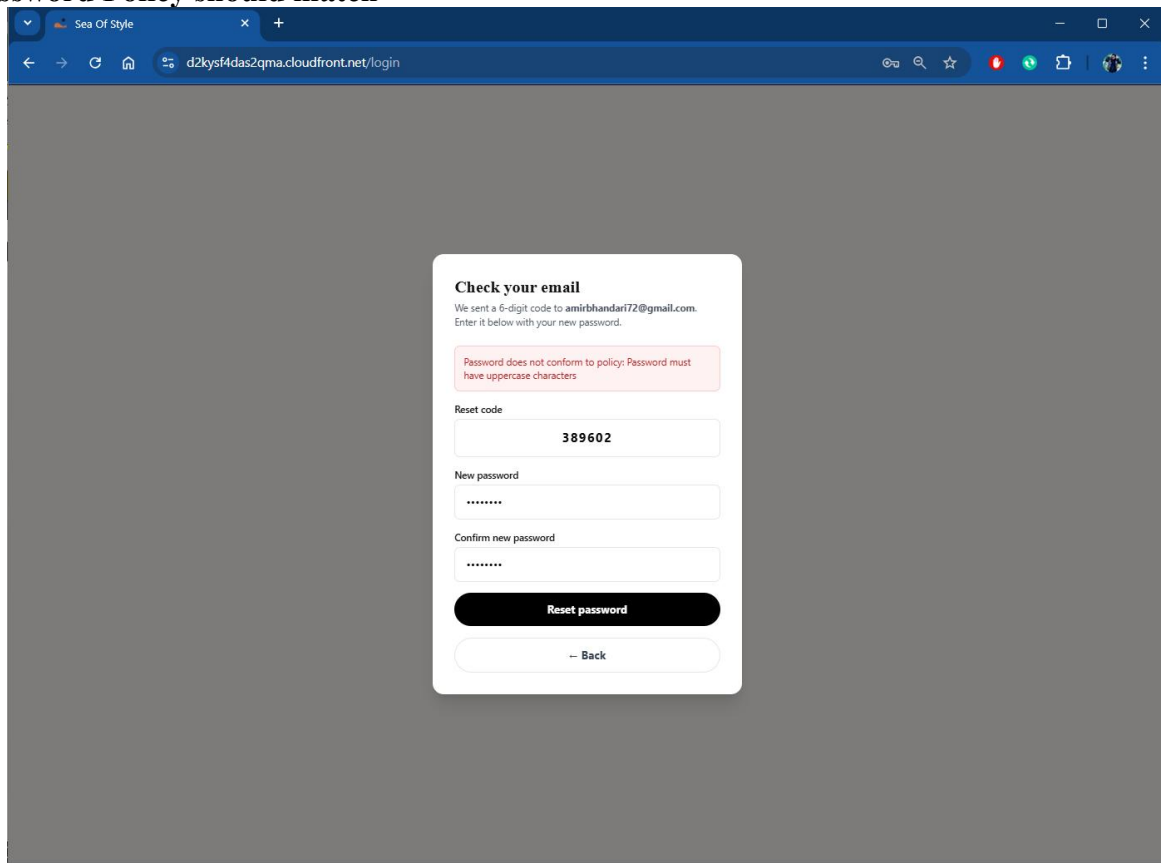


Figure 26: Password Policy Unmatched

Admin Workflow (Management Operations)

Admin Access and Authentication

Administrators log into the system using Amazon Cognito. Based on assigned roles, admins are granted elevated permissions to access management features.

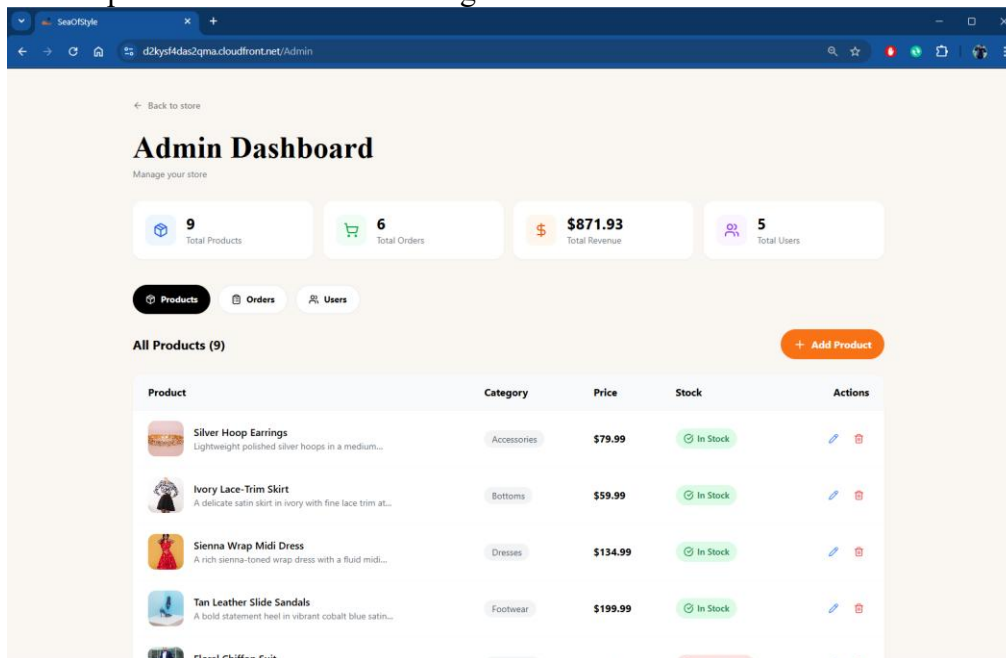


Figure 27: Admin Dashboard

Product Management (Add, Edit, Delete)

Admins can add new products, update existing product details, or delete products from the system. These operations are performed through API Gateway and Lambda functions, which update the corresponding data in DynamoDB. This allows real-time management of the product catalog

Add Product

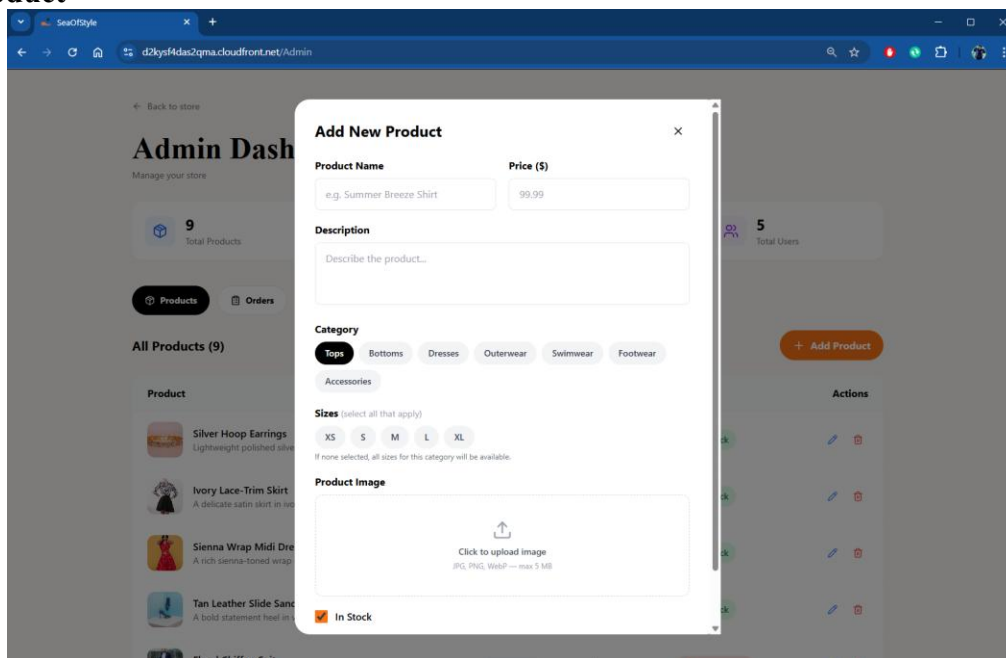


Figure 28: Add Product

Edit Product

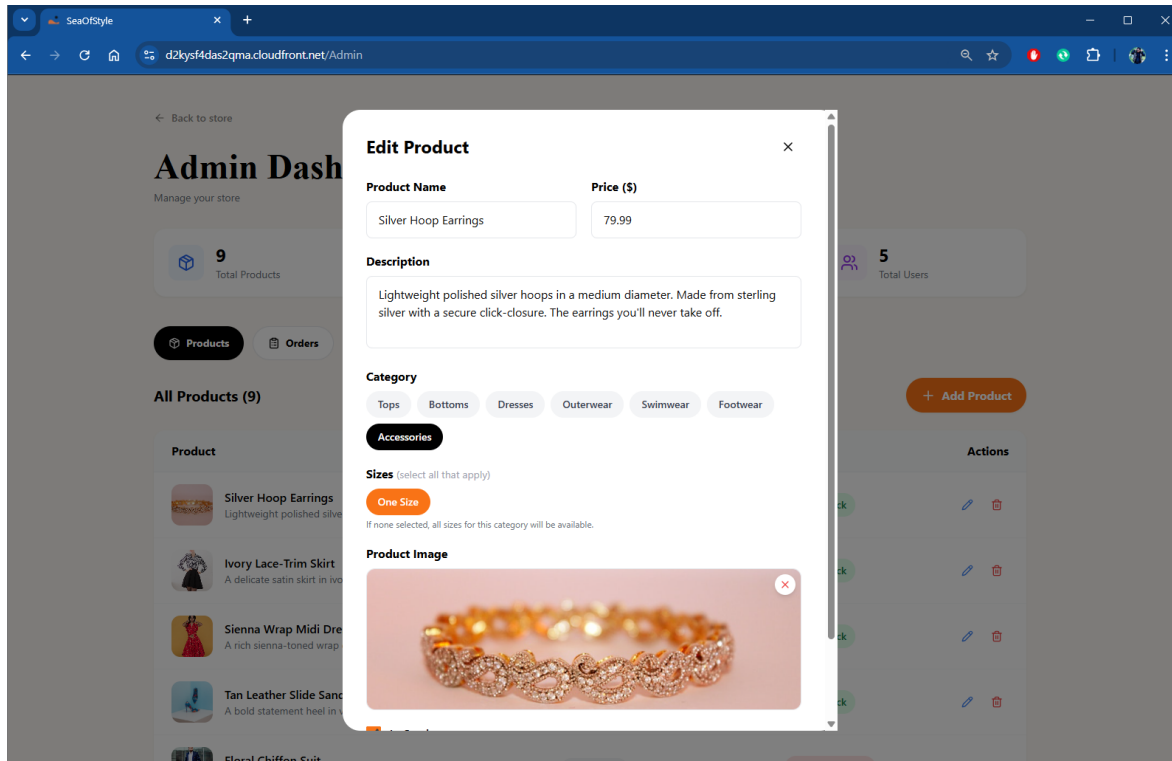


Figure 29: Edit Product

Delete Product

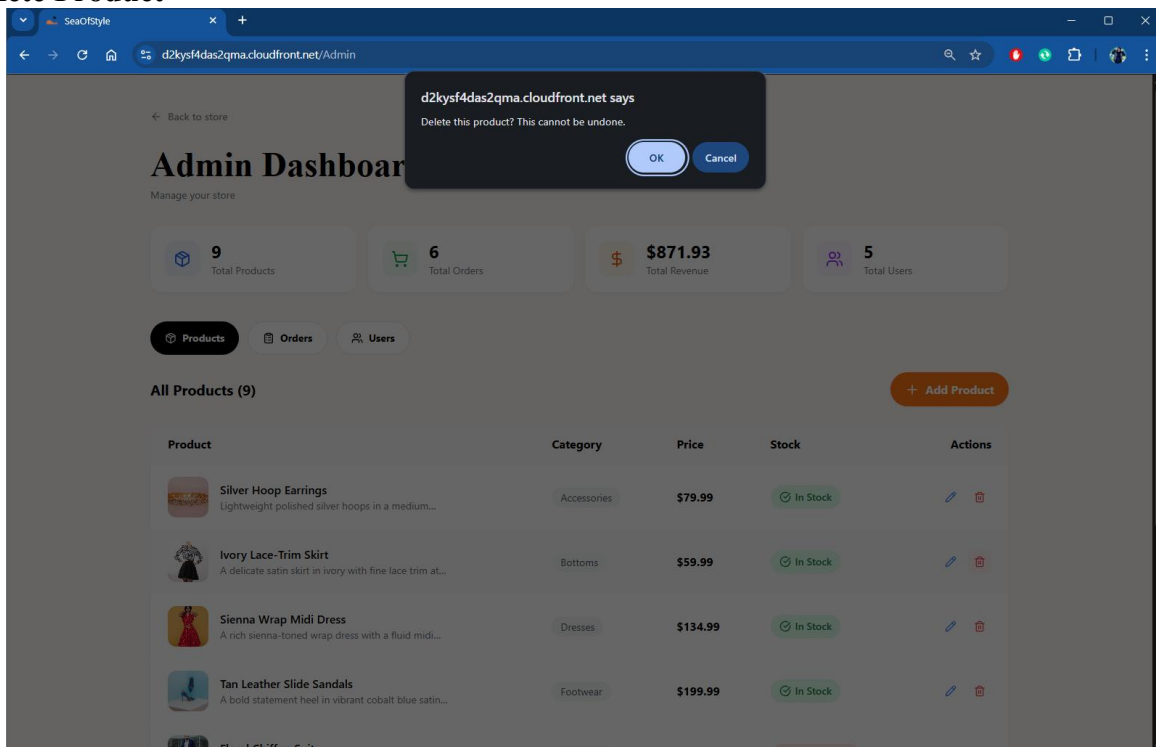


Figure 30: Delete the Product

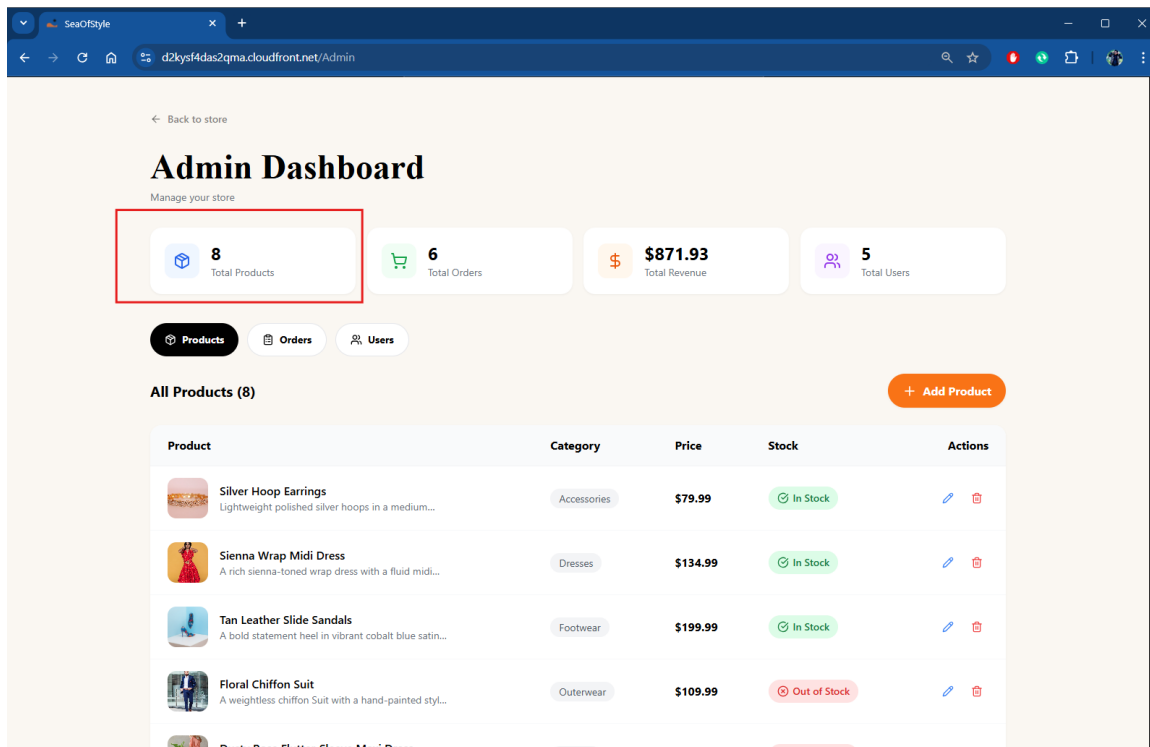


Figure 31: Delete Successful

Figure 29 shows, after deleting, products become from 9 to 8.

User Role Management

Admins can modify user roles, such as promoting a regular user to an admin role. This is handled through secure API calls and controlled using IAM and Cognito-based authorization.

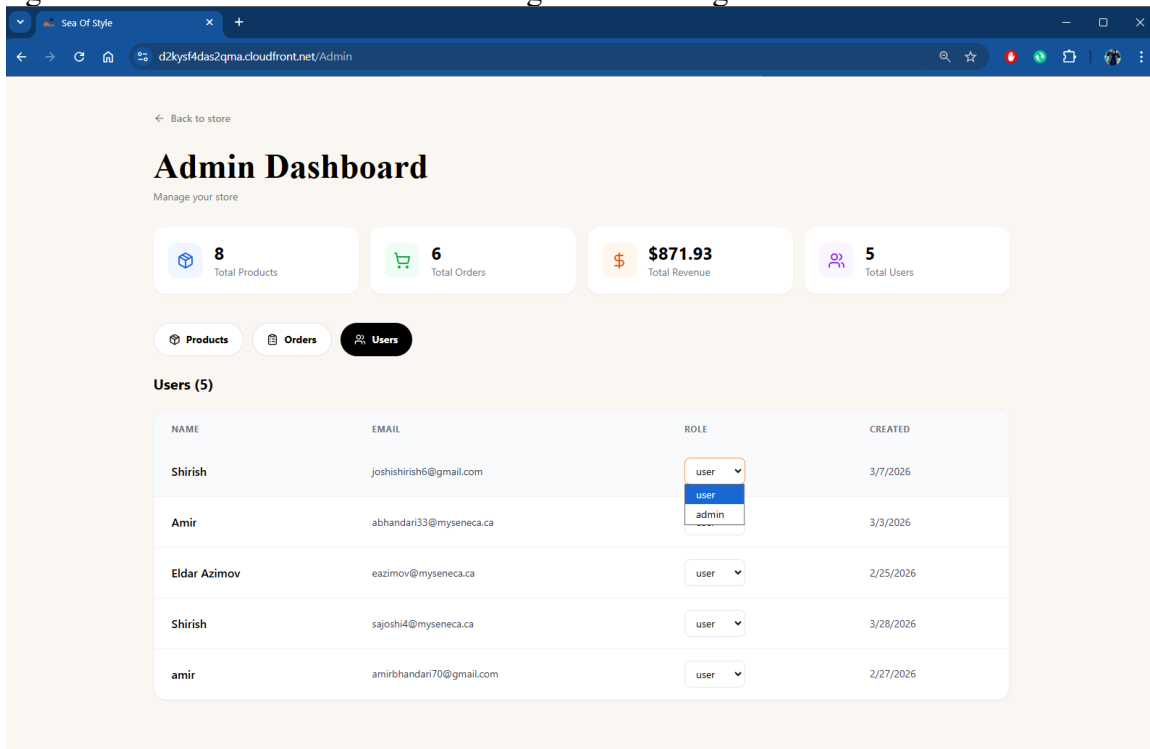


Figure 32: Modify User's Role

Admin User's Page

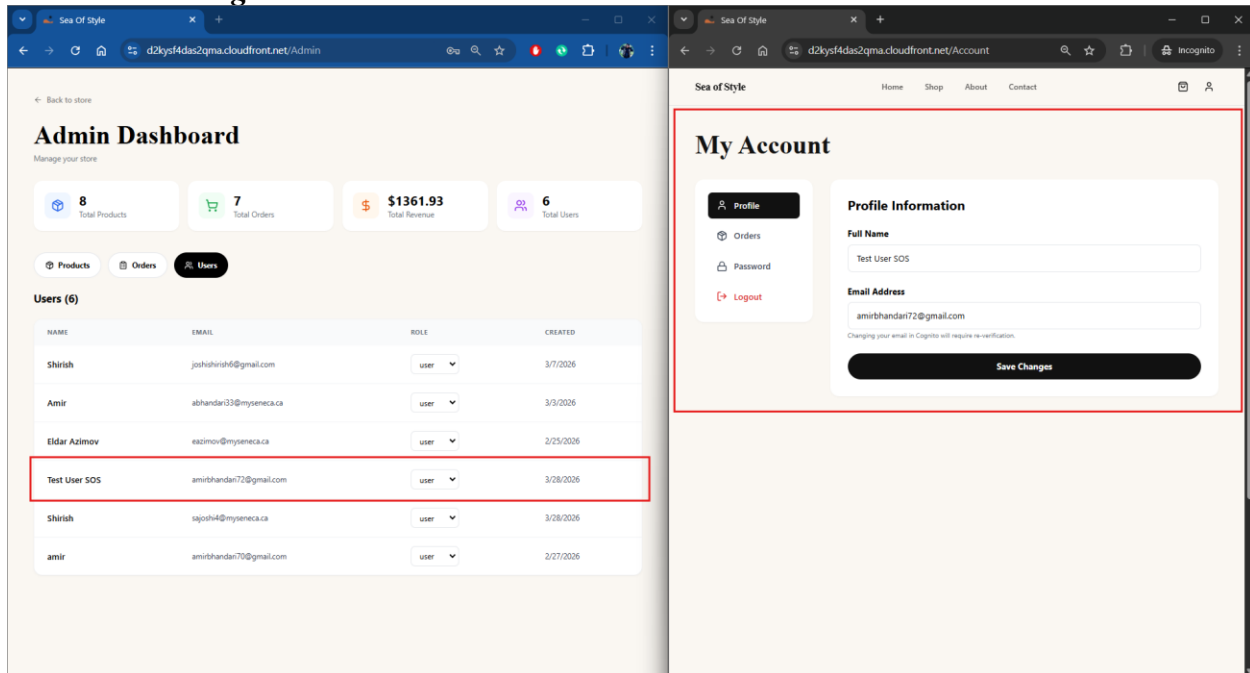


Figure 33: User status on Admin Panel

Order Status Management

Admins can update order statuses, such as processing, confirmed, shipped, Delivered, Cancelled or Refunded orders. These updates are processed through Lambda functions and stored in DynamoDB, ensuring consistency across both admin and user views.

Admin's Order Page

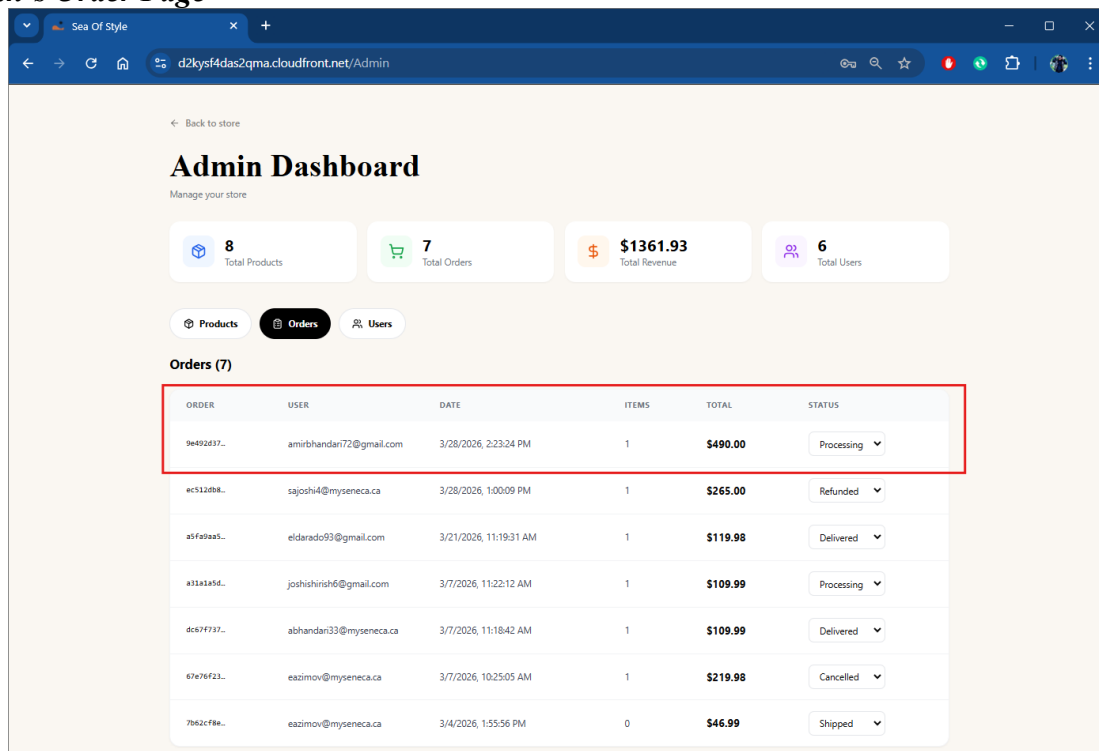


Figure 34: Admin's Order Page

Order Status

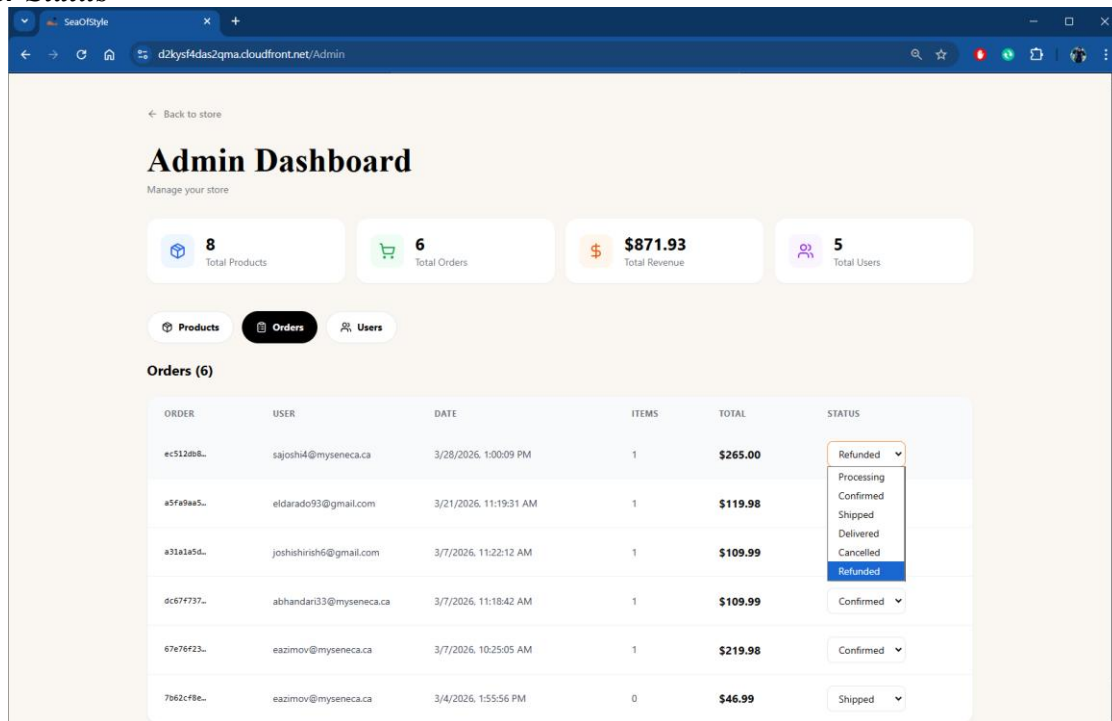


Figure 35: Order Status

Order Confirmed

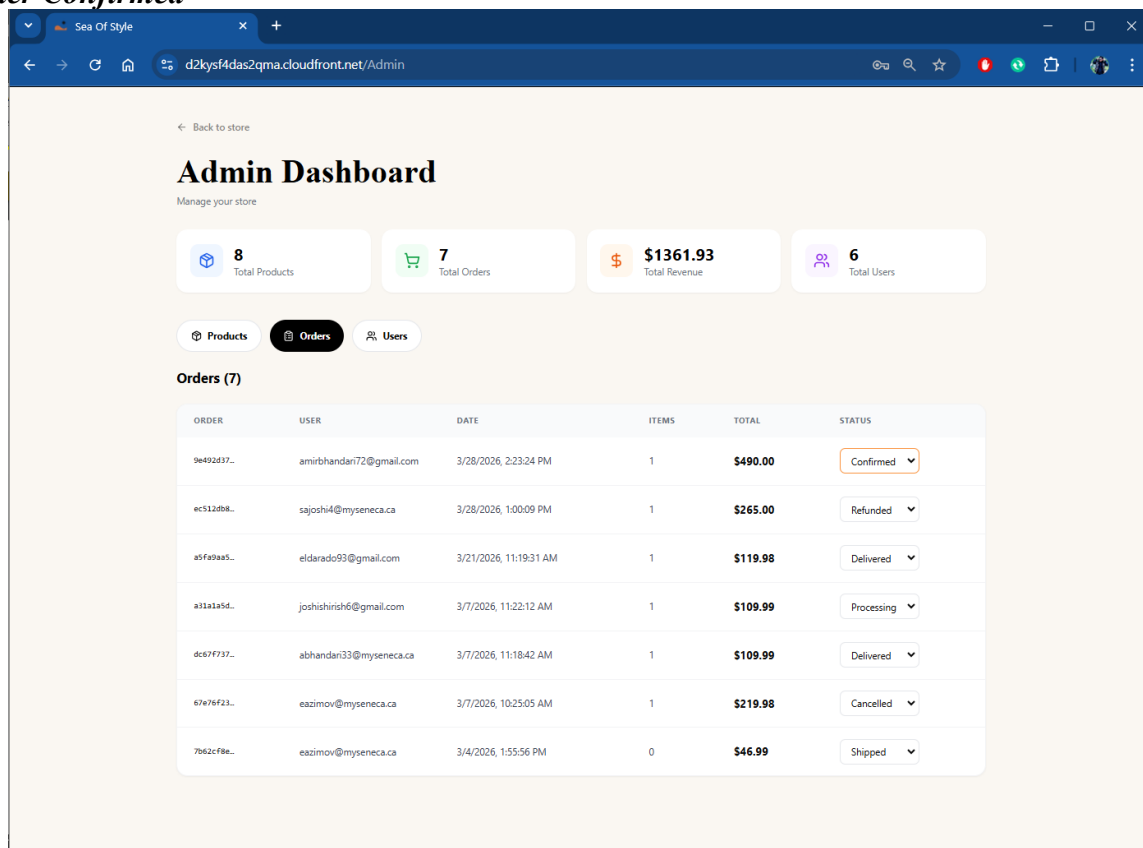


Figure 36: Order Confirmed

Modify Order status from confirmed to delivered

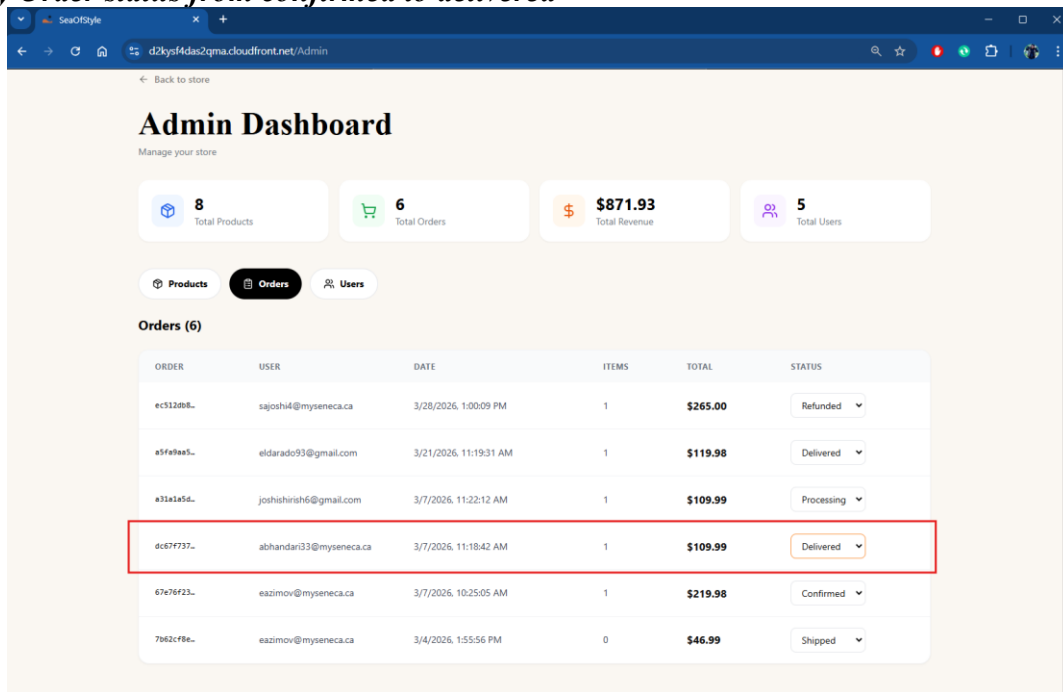


Figure 37: Order Delivered

Modify Order status from confirmed to cancellation

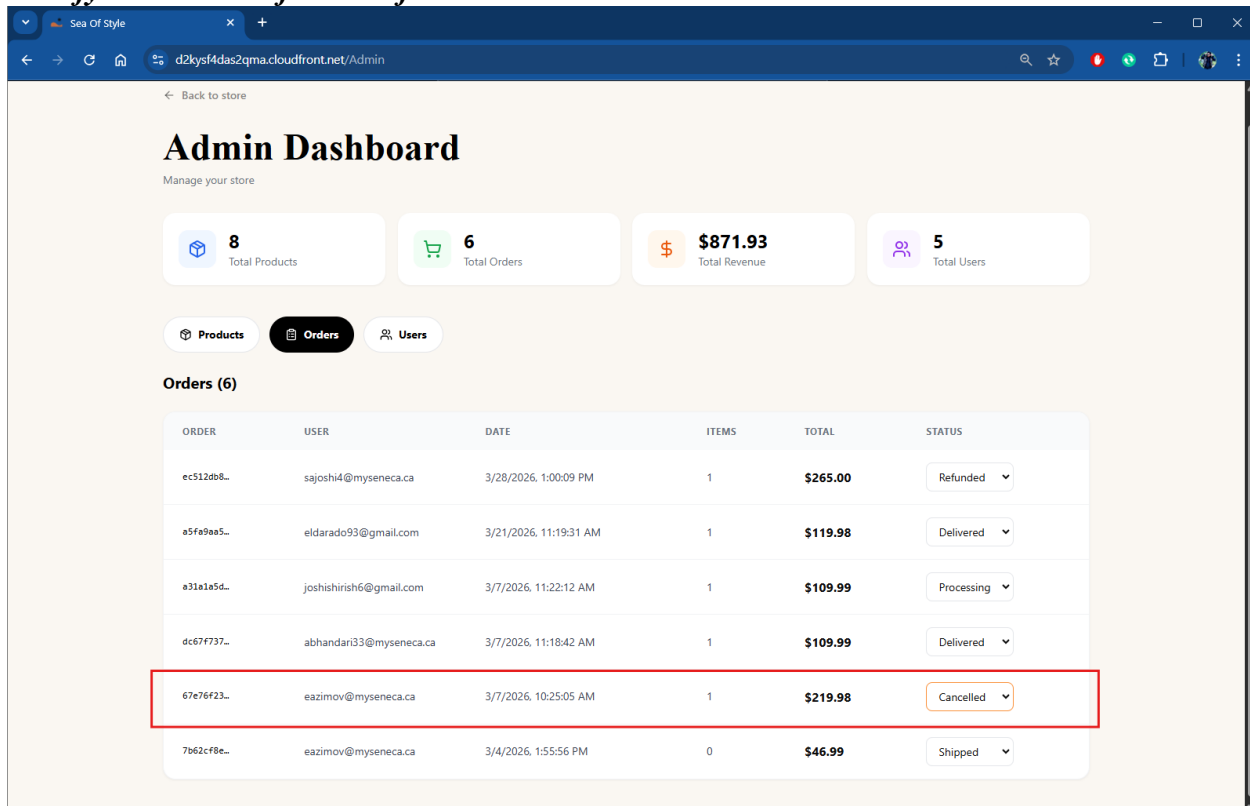


Figure 38: Order Cancelled

Order Delivered on admin side and user side

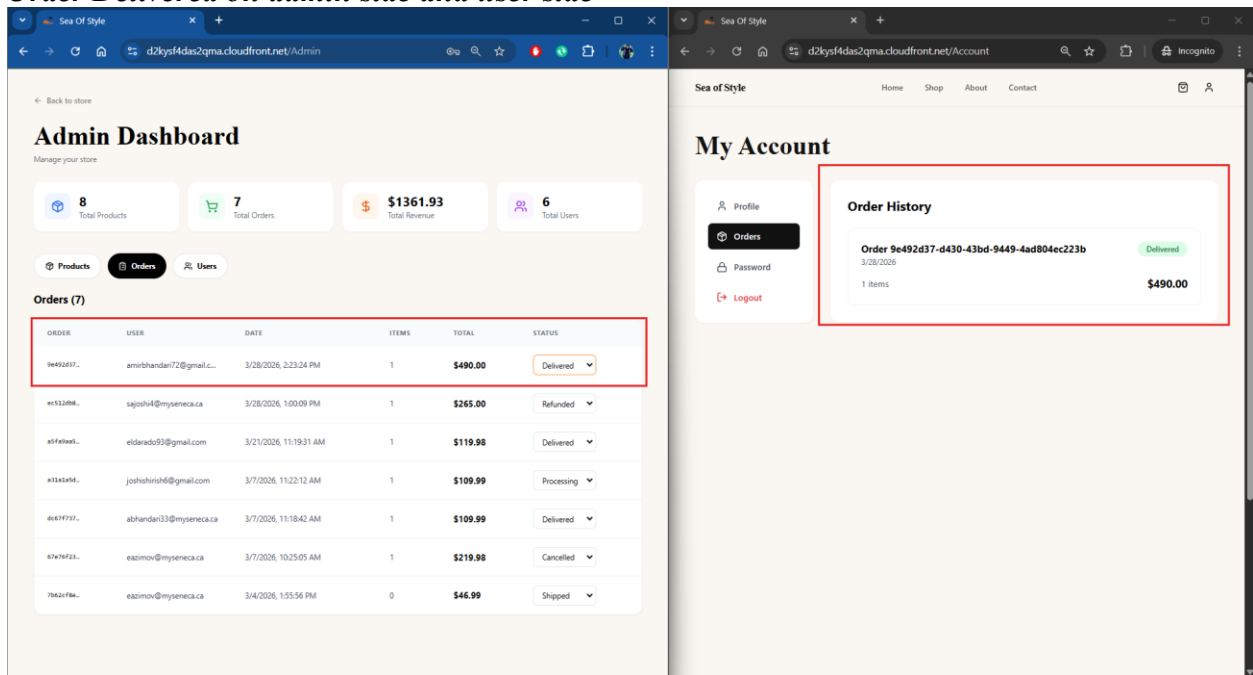


Figure 39: Order status confirmed on both Admin and User side

Admin Password Update Password Length

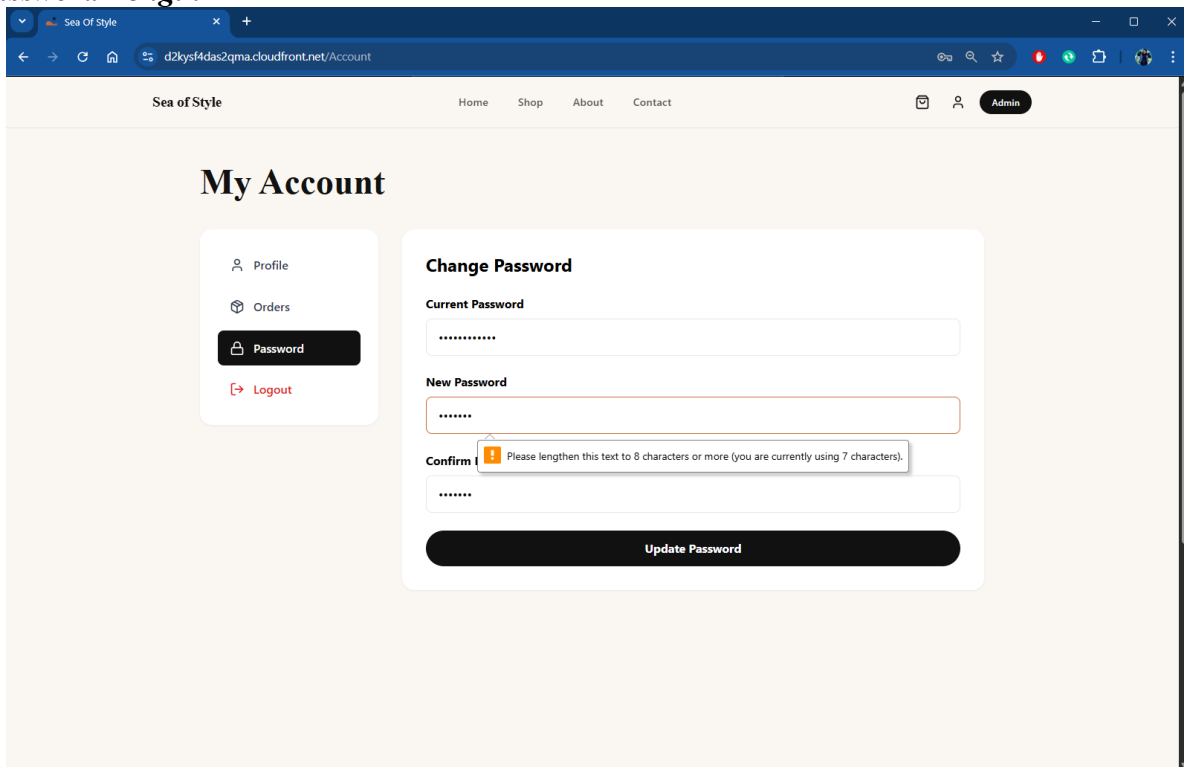
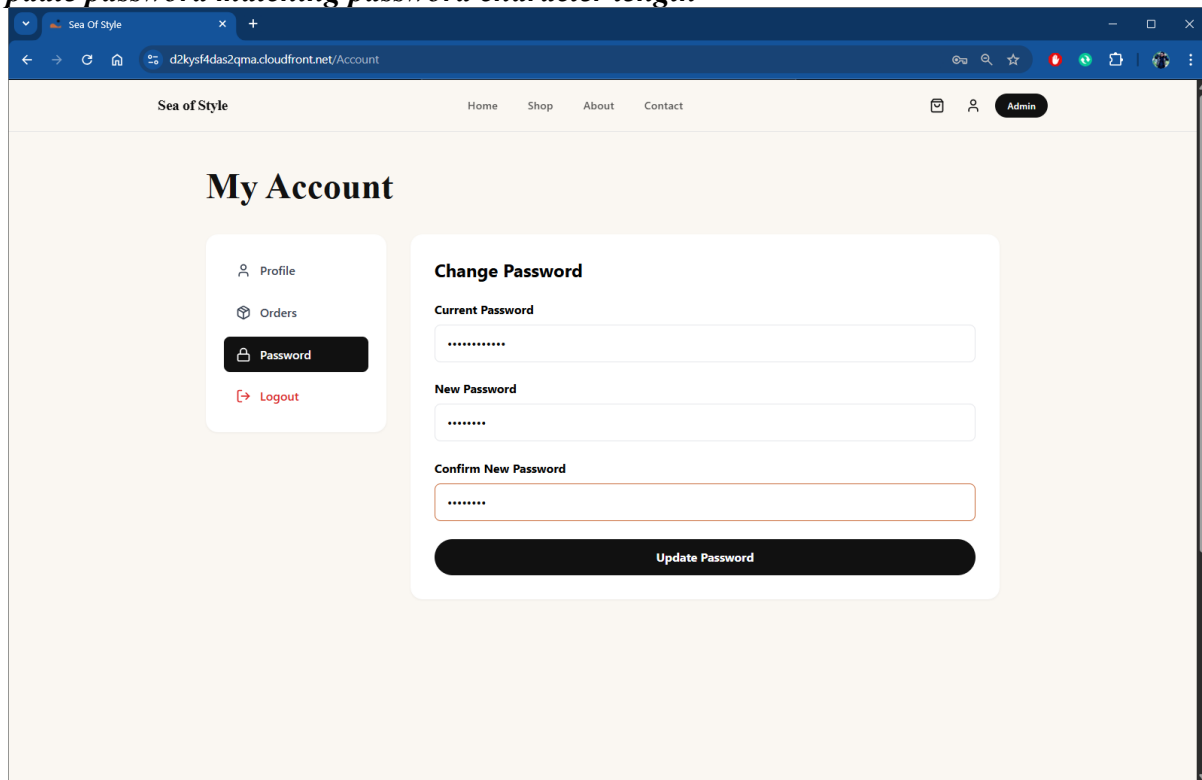


Figure 40: Password Length Error

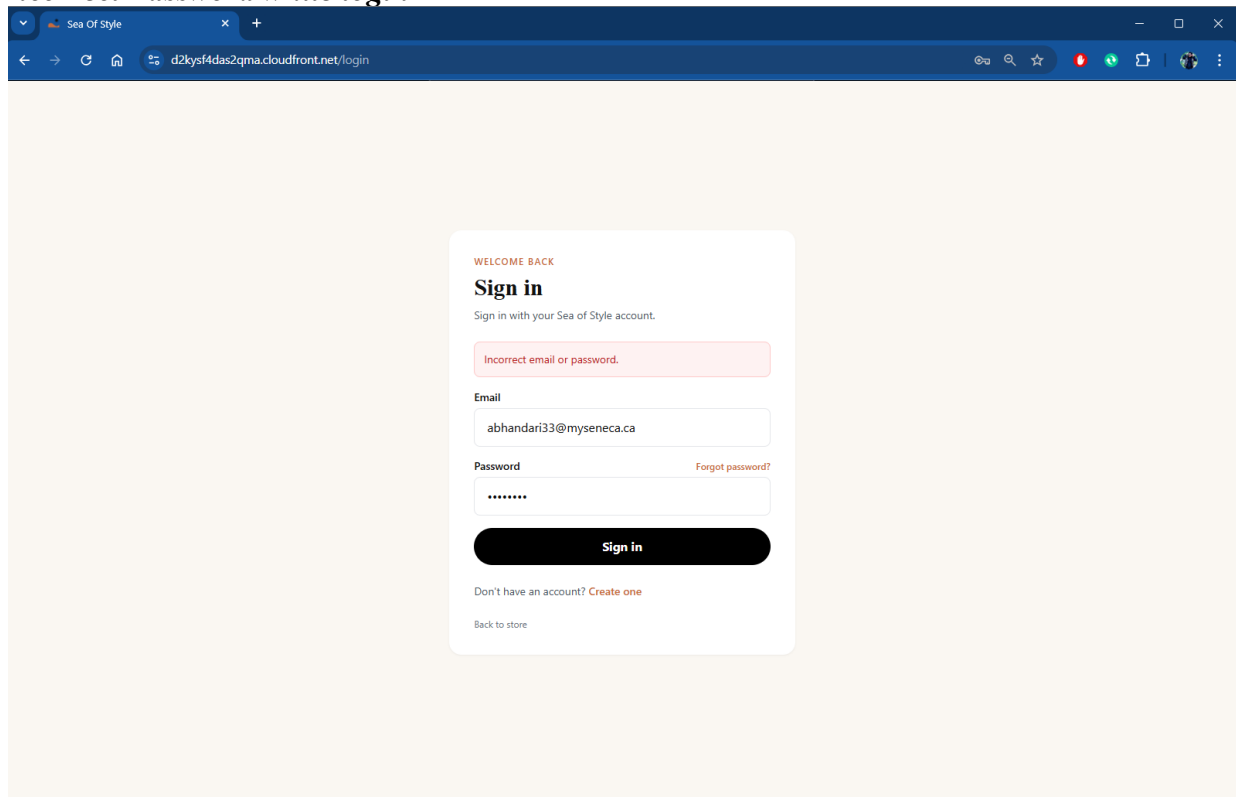
Update password matching password character length



The screenshot shows a web browser window with the URL `d2kysf4das2qma.cloudfront.net/Account`. The page is titled "My Account" and features a navigation menu with "Home", "Shop", "About", and "Contact". A user profile icon and an "Admin" button are visible in the top right. The main content area is divided into two sections. On the left, a sidebar contains links for "Profile", "Orders", "Password" (highlighted), and "Logout". The right section is titled "Change Password" and contains three input fields: "Current Password", "New Password", and "Confirm New Password". All fields are filled with asterisks. A black "Update Password" button is at the bottom of the form.

Figure 41: Update Password with valid

Incorrect Password while login



The screenshot shows a web browser window with the URL `d2kysf4das2qma.cloudfront.net/login`. The page is titled "Sign in" and features a "WELCOME BACK" message. Below the title, it says "Sign in with your Sea of Style account." A red error message "Incorrect email or password." is displayed. The form contains two input fields: "Email" (filled with `abhandari33@myseneca.ca`) and "Password" (filled with asterisks). A "Forgot password?" link is next to the password field. A black "Sign in" button is at the bottom of the form. Below the button, there is a link "Don't have an account? Create one" and a link "Back to store".

Figure 42: Verifying with the updated password

AWS Services Used

The SOS platform is built using a collection of AWS managed services that together provide a scalable, secure, and serverless architecture. Each service is responsible for a specific function, including frontend delivery, authentication, backend processing, data storage, communication, monitoring, and infrastructure automation.

Table: AWS Services and Their Purpose

Service	Purpose for the Project
Amazon S3	Hosts static frontend files and stores Terraform state
Amazon CloudFront	Delivers content globally with low latency
Amazon Cognito	Manages user authentication and authorization
Amazon API Gateway	Acts as entry point for backend APIs
AWS Lambda	Executes backend business logic
Amazon DynamoDB	Stores application data (products, orders, users)
Amazon SNS	Enables event-driven notifications
Amazon SES	Provisioned in infrastructure (password reset handled by Cognito)
Amazon Lex	Provides chatbot-based interaction
Amazon CloudWatch	Monitors logs and system performance
AWS CloudTrail	Tracks API and account activity
AWS IAM	Manages roles and permissions
AWS KMS	Encrypts sensitive data
AWS Secrets Manager	Stores sensitive credentials securely
Terraform	Automates infrastructure provisioning
GitHub Actions	Implements CI/CD pipeline

Core Application Services

These services form the foundation of the SOS platform and handle the main application functionality.

Amazon S3

Amazon S3 is used to host the static frontend of the application and store Terraform state files for infrastructure management.

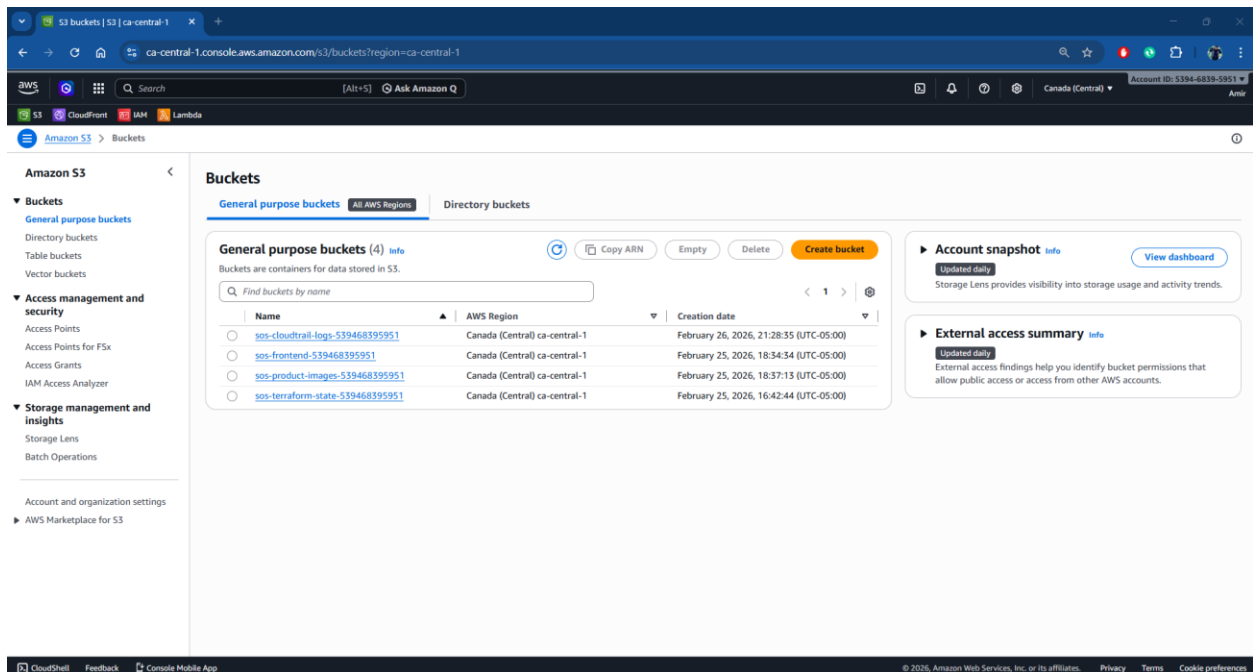


Figure 43: S3 Bucket List

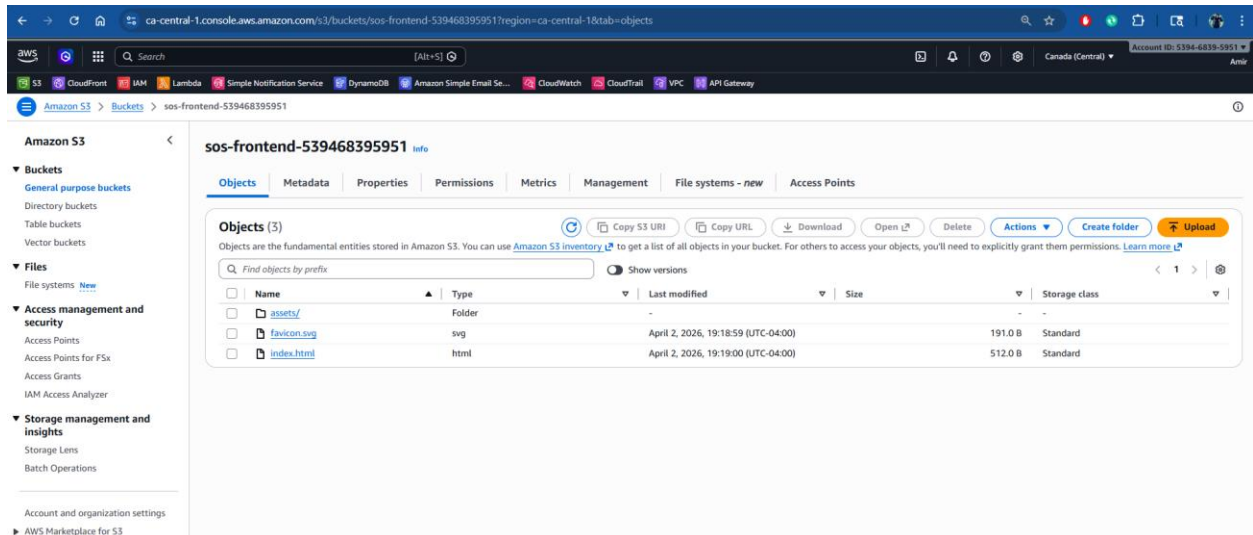


Figure 44: S3 Bucket (Frontend Hosting)

Amazon CloudFront

CloudFront improves performance by caching frontend content at edge locations and delivering it securely over HTTPS.

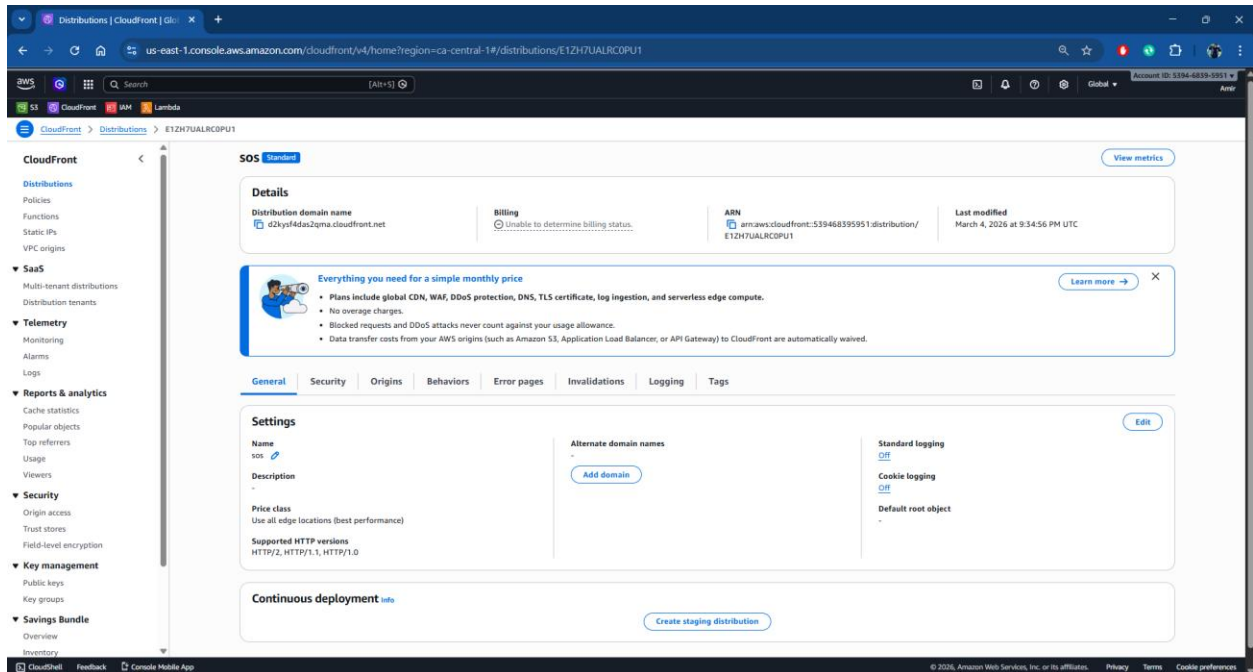


Figure 45: CloudFront Distribution

Amazon Cognito

Cognito manages user authentication, including signup, login, and password recovery (Forgot Password).

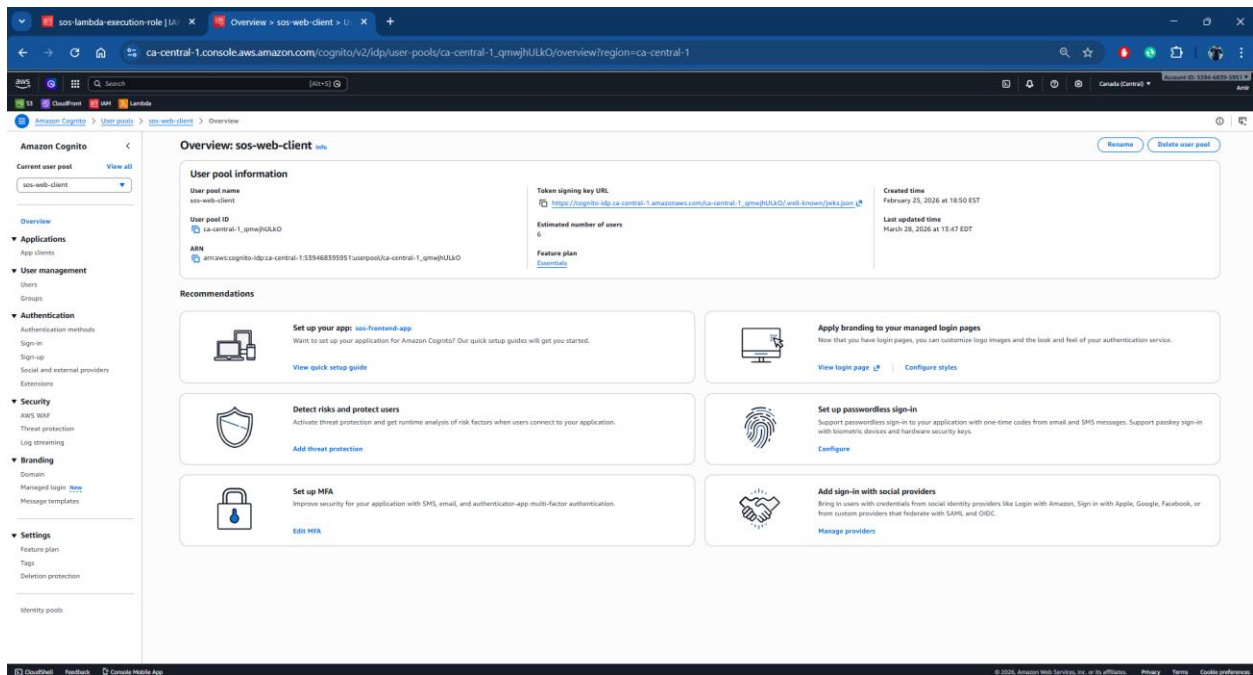


Figure 46: Amazon Cognito Overview

Amazon Cognito – App Client

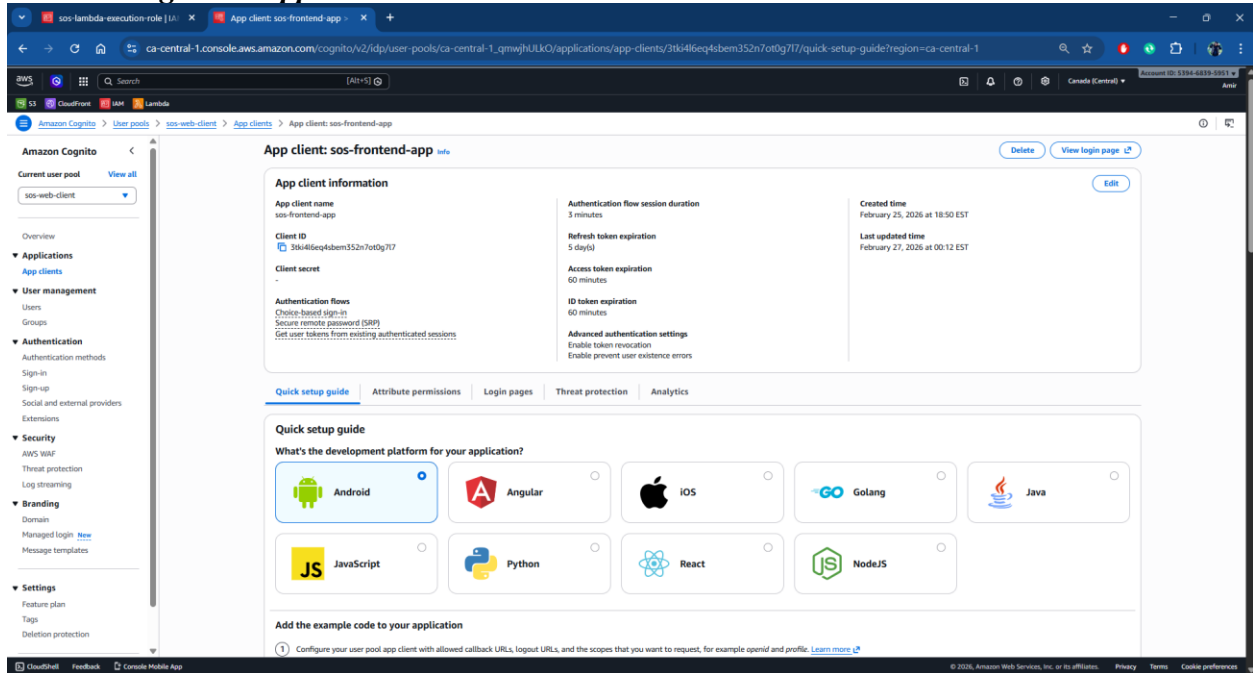


Figure 47: Amazon Cognito - App Client

Amazon Cognito – Users

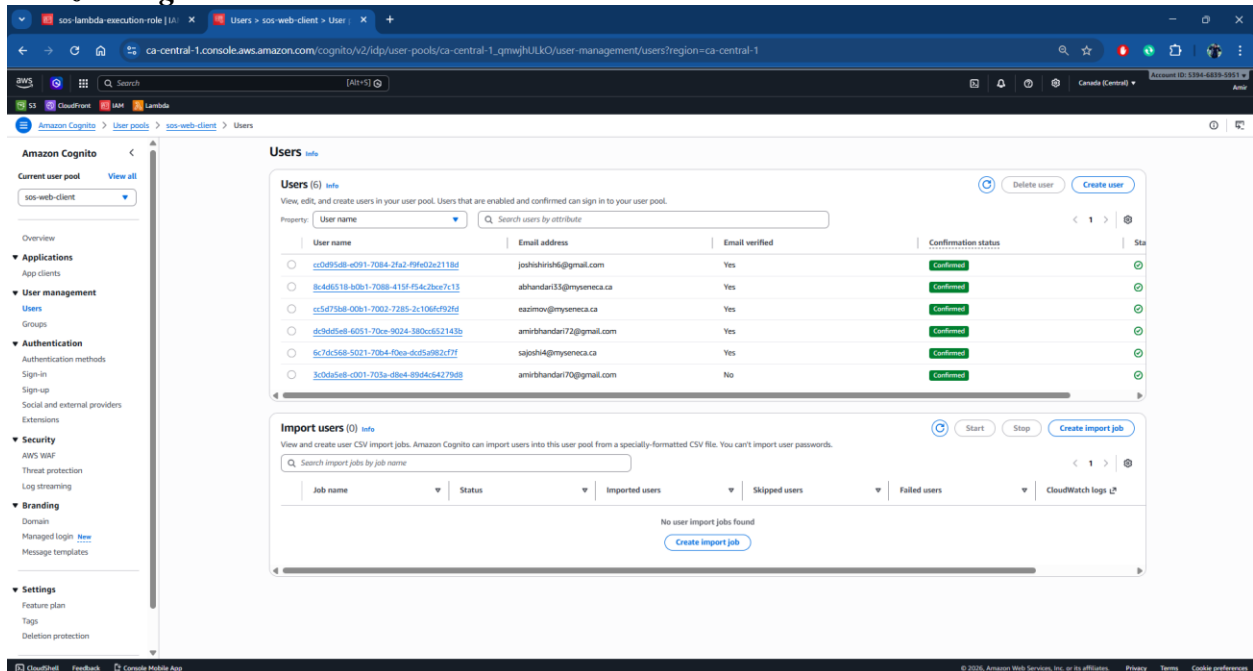


Figure 48: Amazon Cognito – Users

Amazon Cognito – Groups

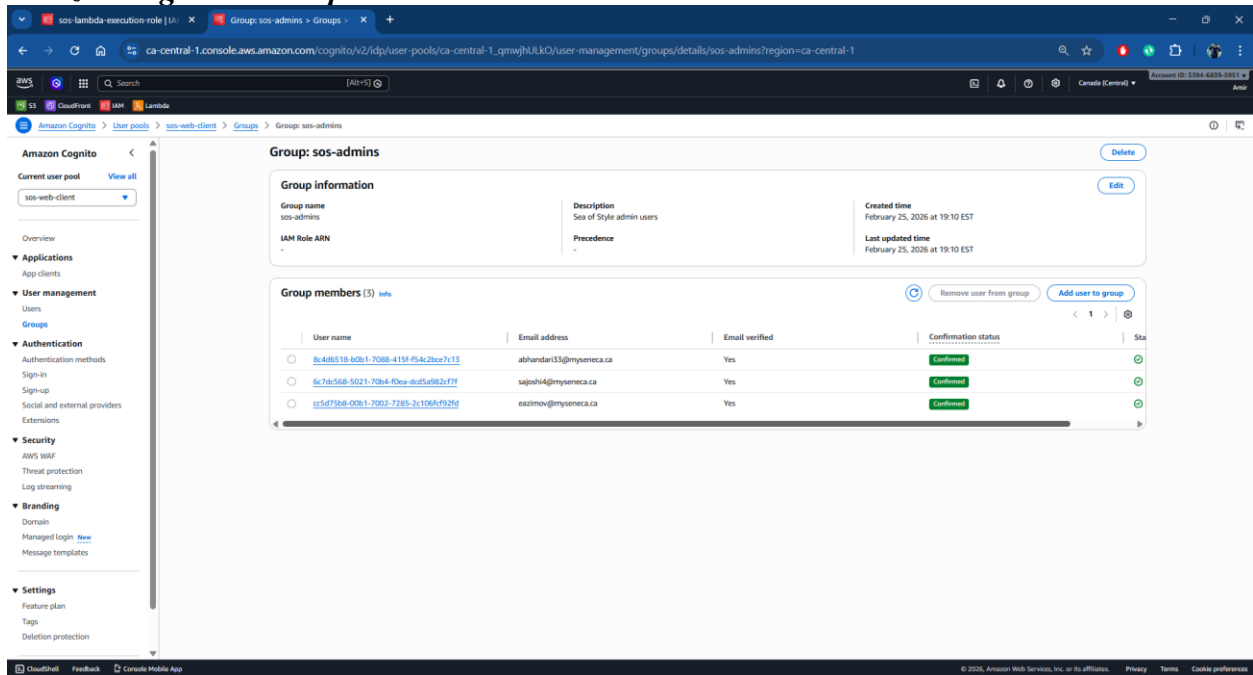


Figure 49: Amazon Cognito - Groups

Amazon API Gateway

API Gateway acts as the entry point for all backend requests and routes them to Lambda functions.

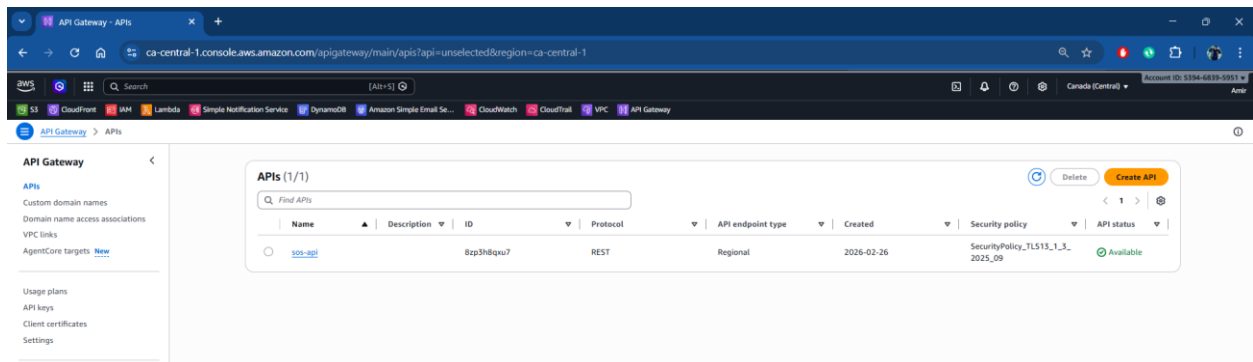


Figure 50: Amazon API Gateway

Resources

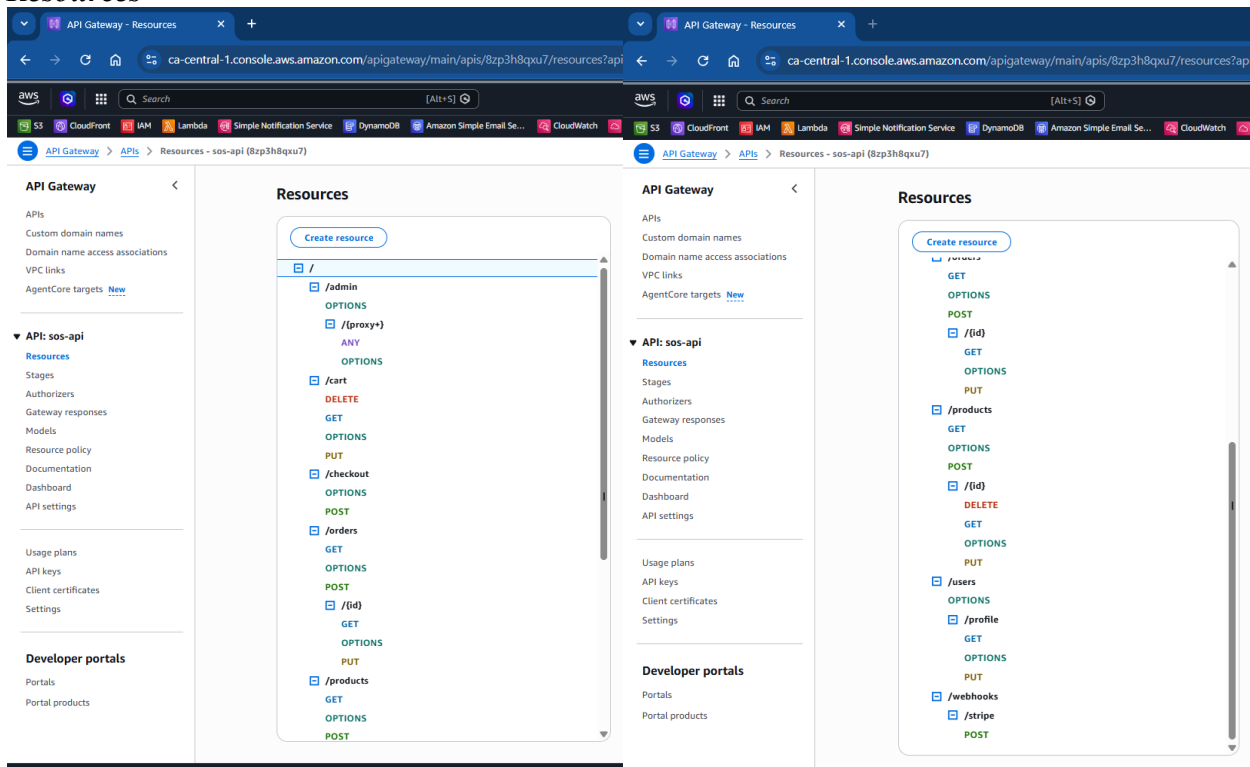


Figure 51: API Gateway - Resources

Stages

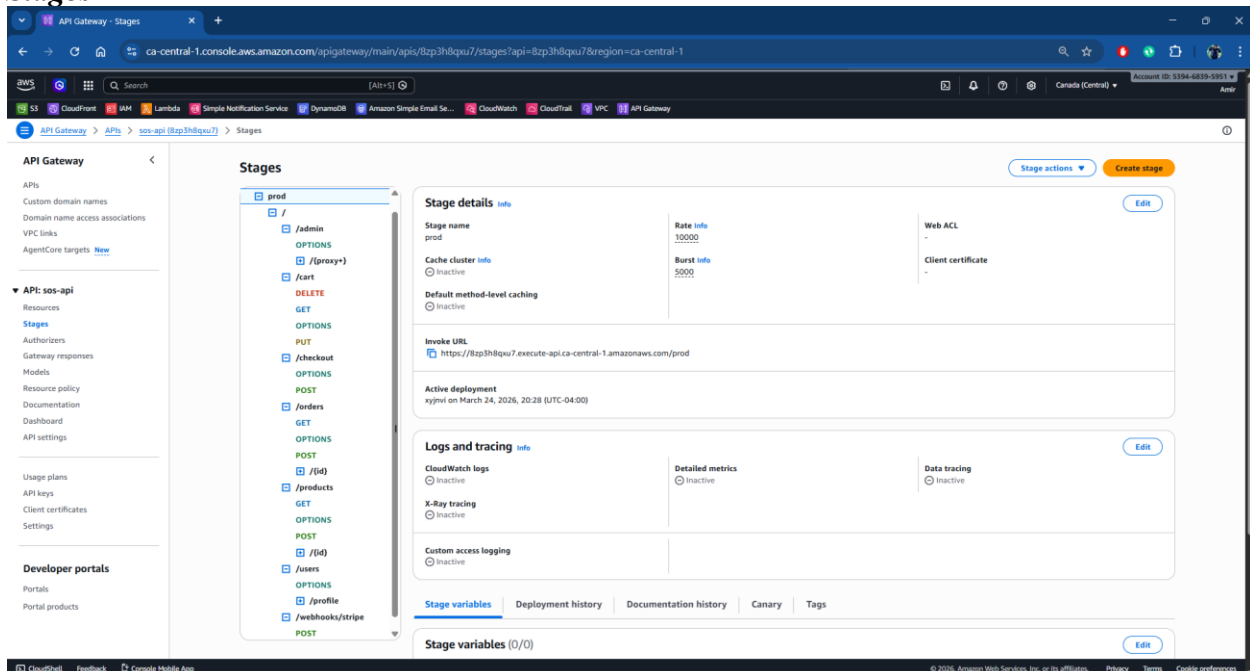


Figure 52: API Gateway – Stages

API Dashboard

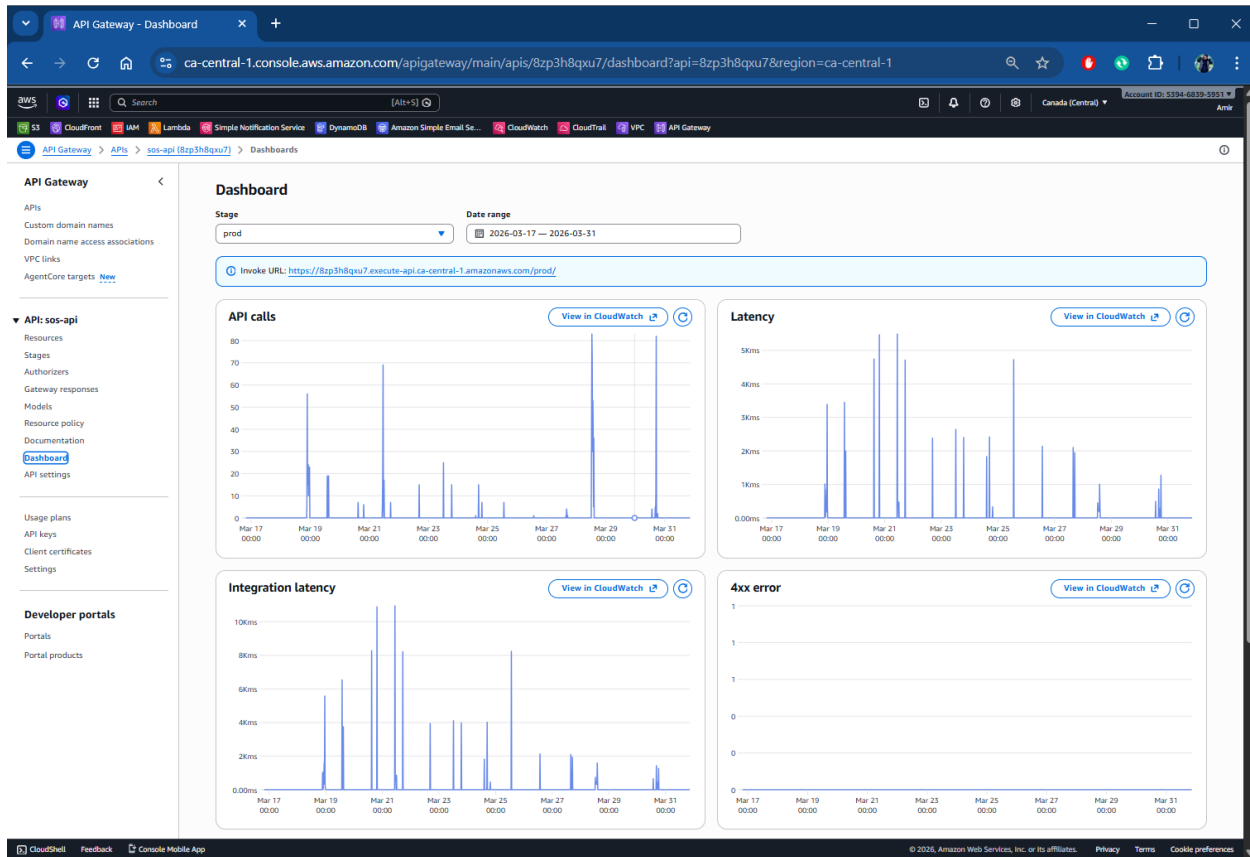


Figure 53: API Gateway - Dashboard

AWS Lambda

Lambda executes backend logic such as product retrieval, cart operations, and order processing.

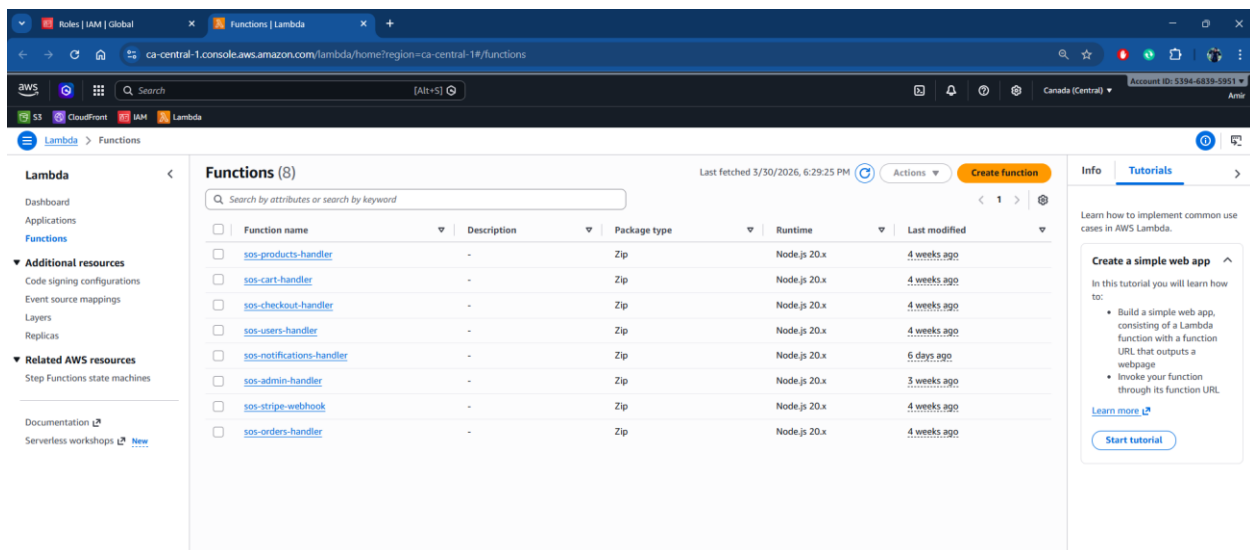


Figure 54: AWS Lambda Functions List

SOS-Products-handler

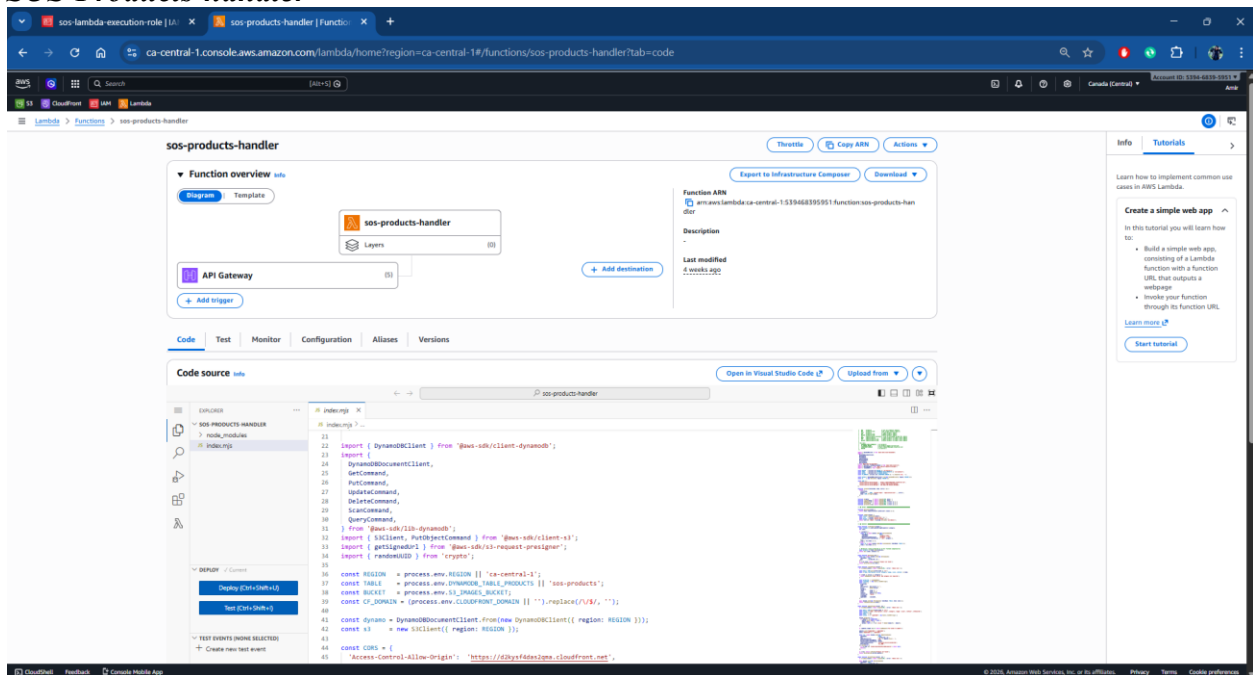


Figure 55: AWS Lambda Functions (sos-products-handler)

SOS-Cart-Handler

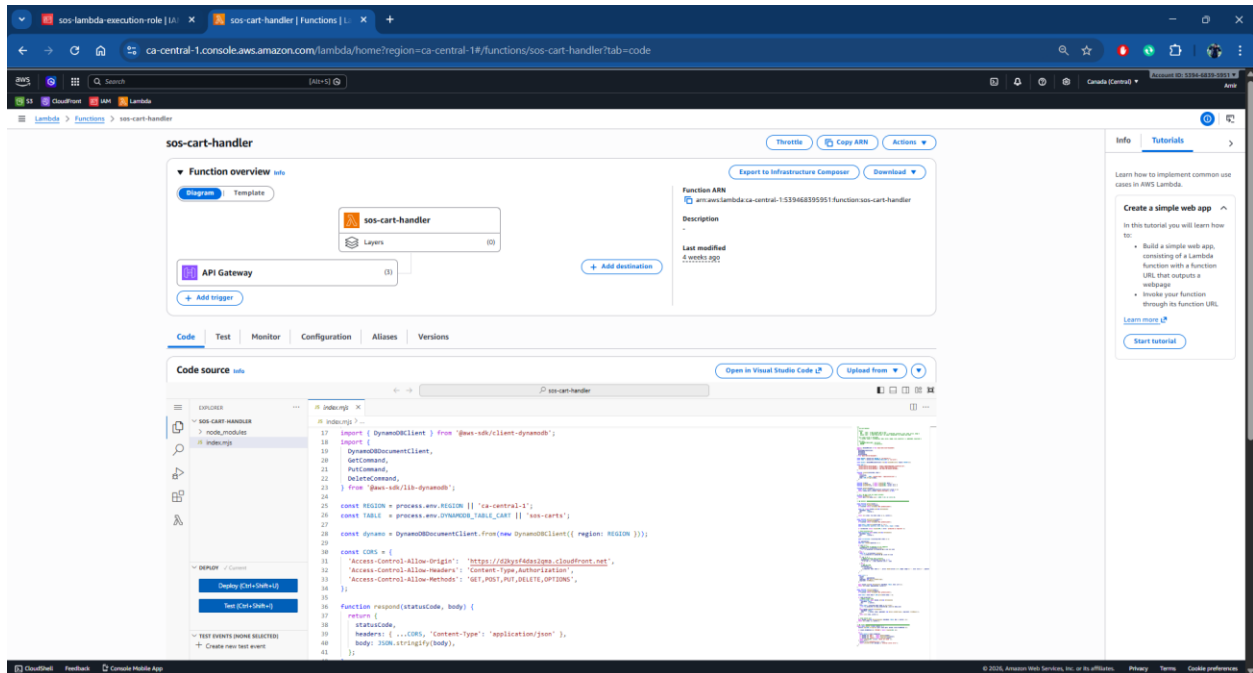


Figure 56: AWS Lambda Functions (sos-cart-handler)

sos-checkout-handler

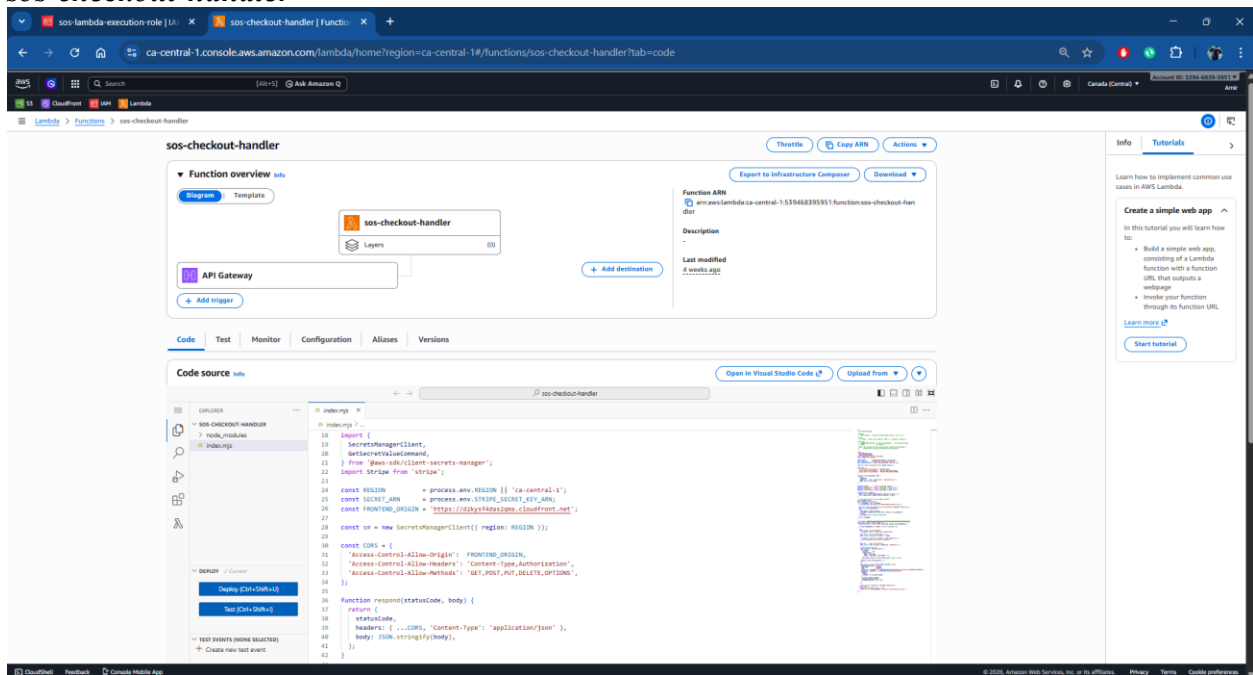


Figure 57: AWS Lambda Functions (sos-checkout-handler)

sos-users-handler

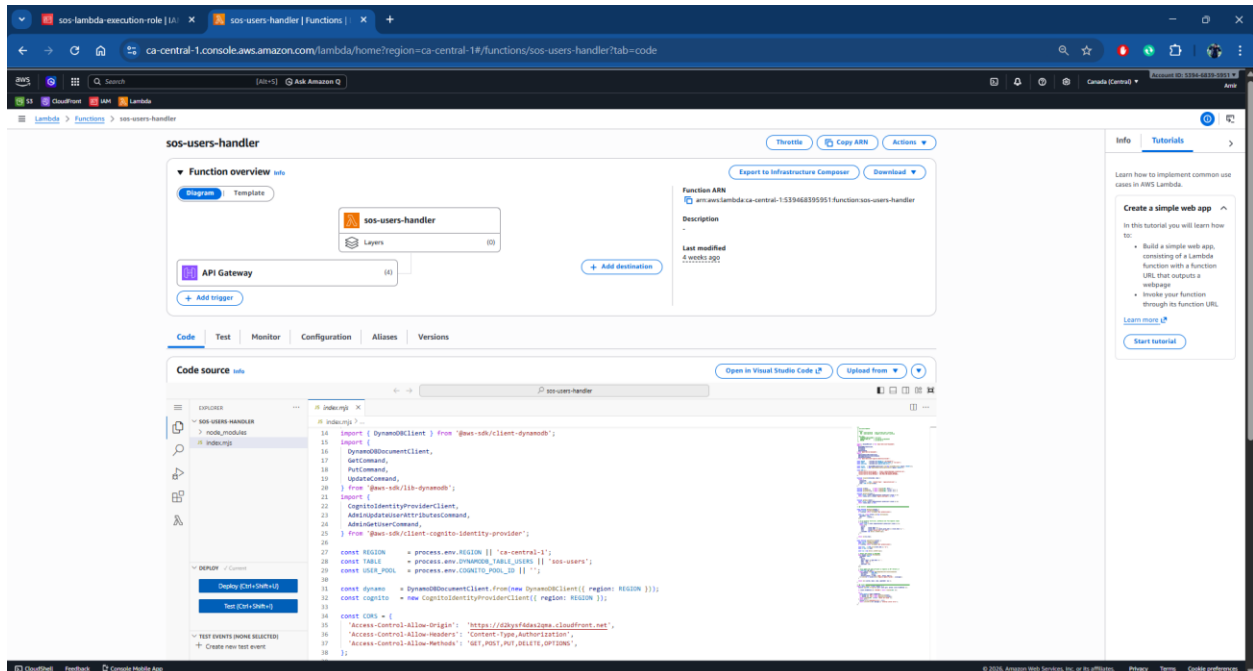


Figure 58: AWS Lambda Functions (sos-users-handler)

sos-notifications-handler

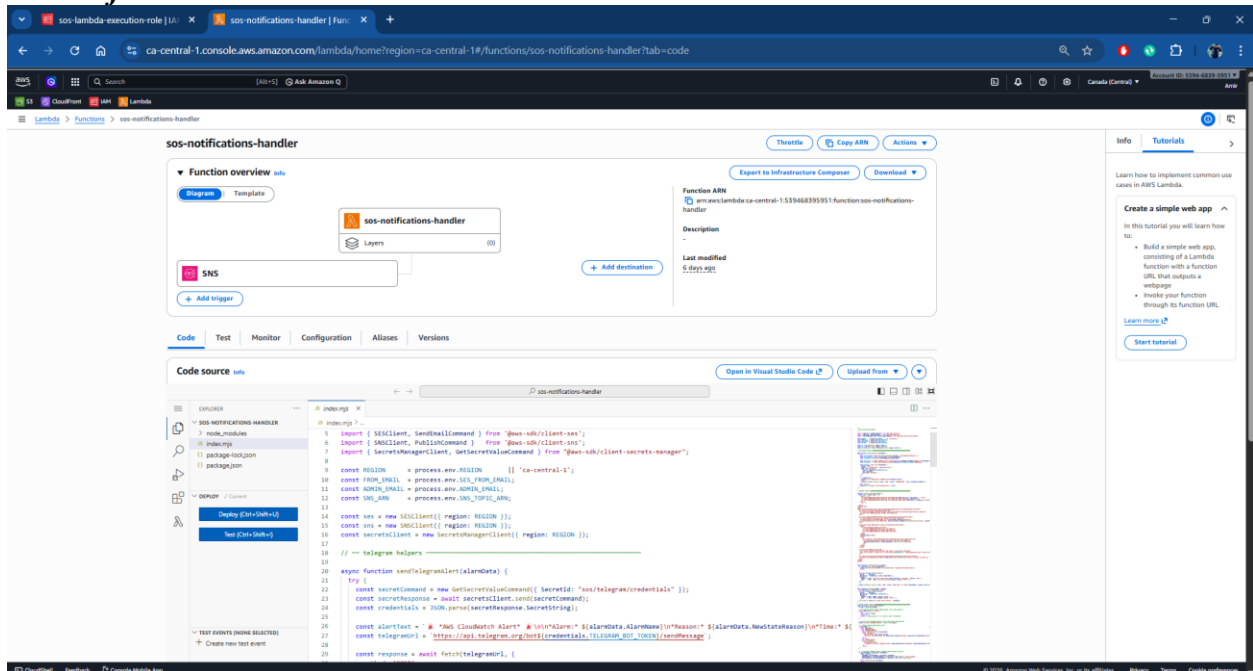


Figure 59: AWS Lambda Functions (sos-notifications-handler)

sos-admin-handler

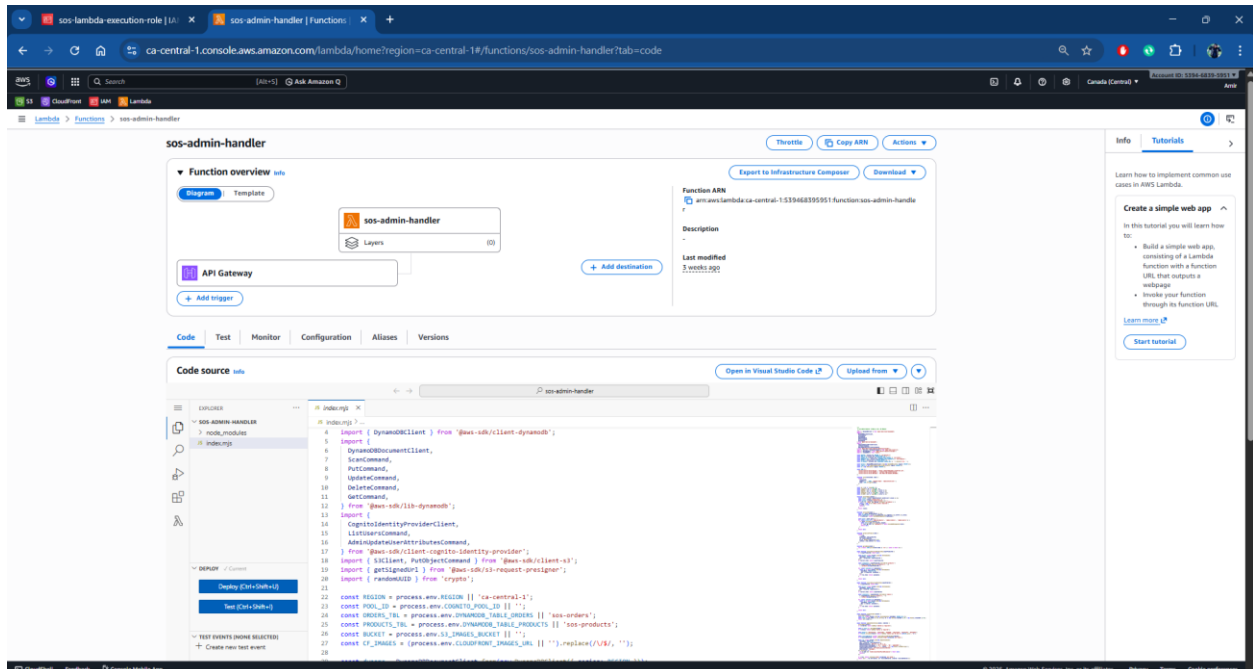


Figure 60: AWS Lambda Functions (sos-admin-handler)

sos-stripe-webhook

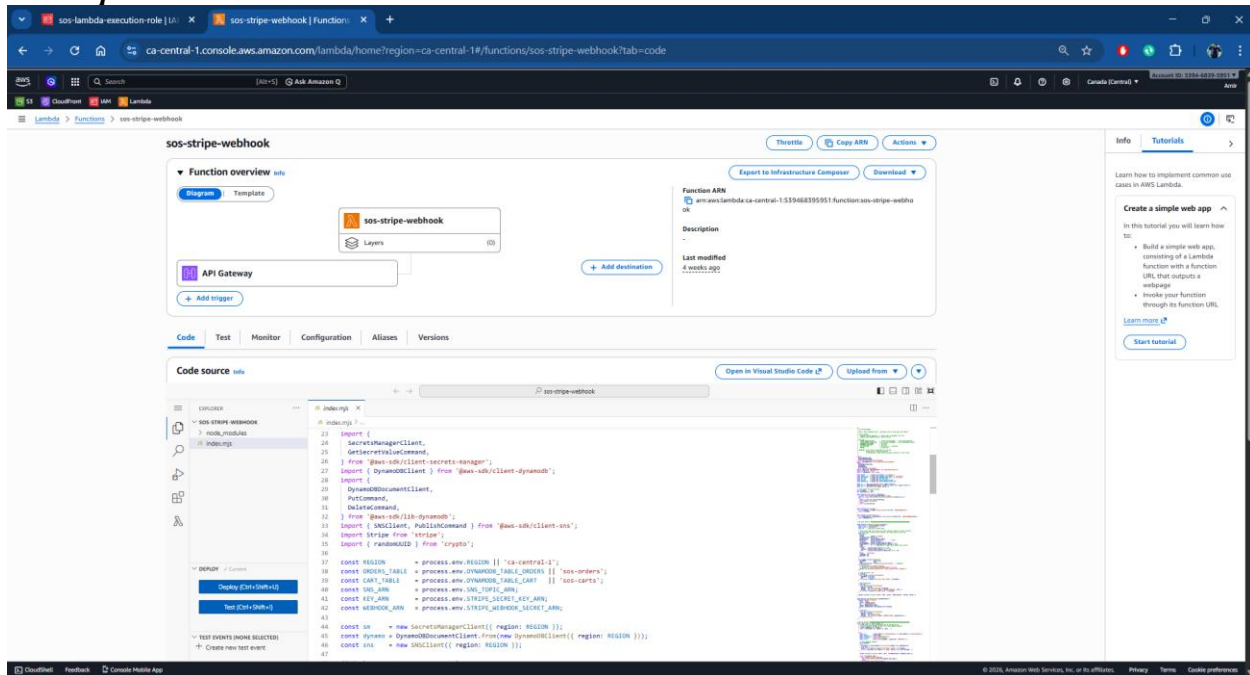


Figure 61: AWS Lambda Functions (sos-stripe-webhook)

sos-orders-handler

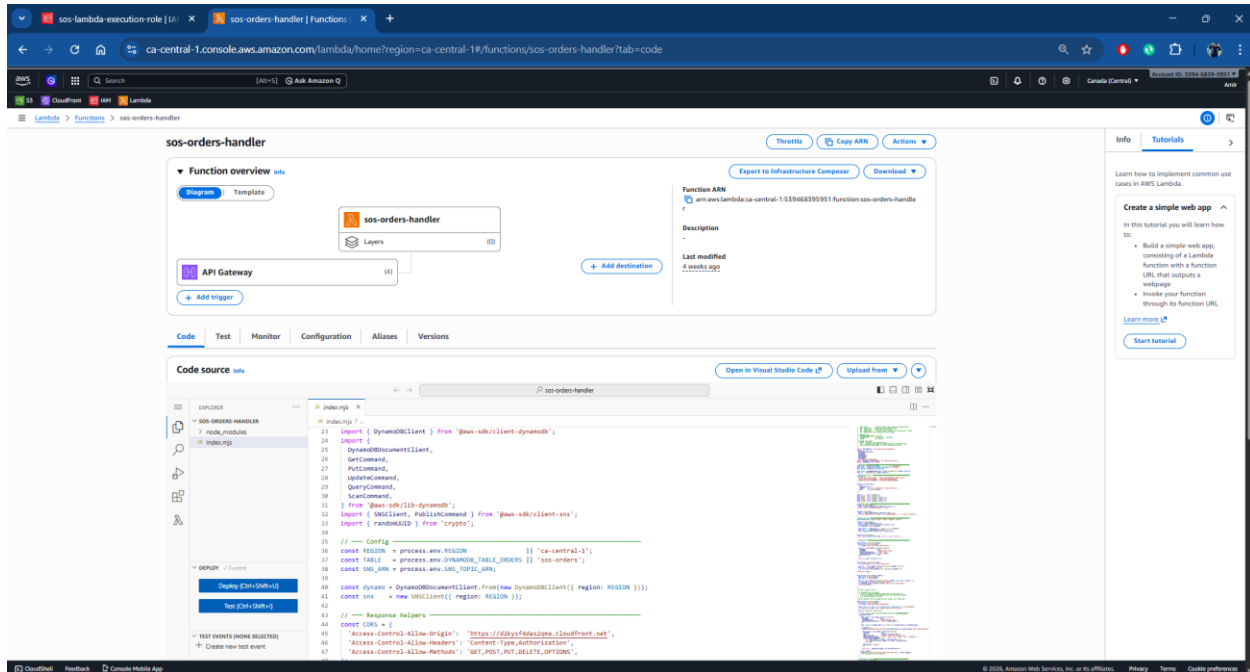


Figure 62: AWS Lambda Functions (sos-orders-handler)

Amazon DynamoDB

DynamoDB stores application data with high scalability and low latency.

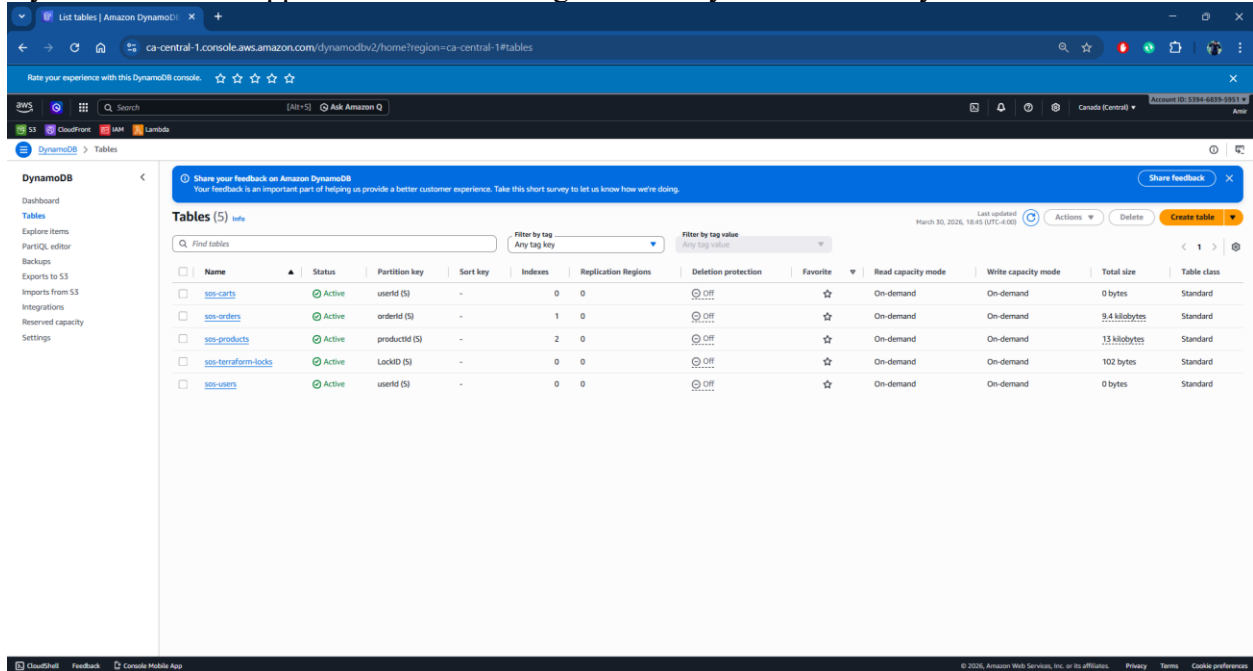


Figure 63: Amazon DynamoDB Tables

DynamoDB – sos-orders

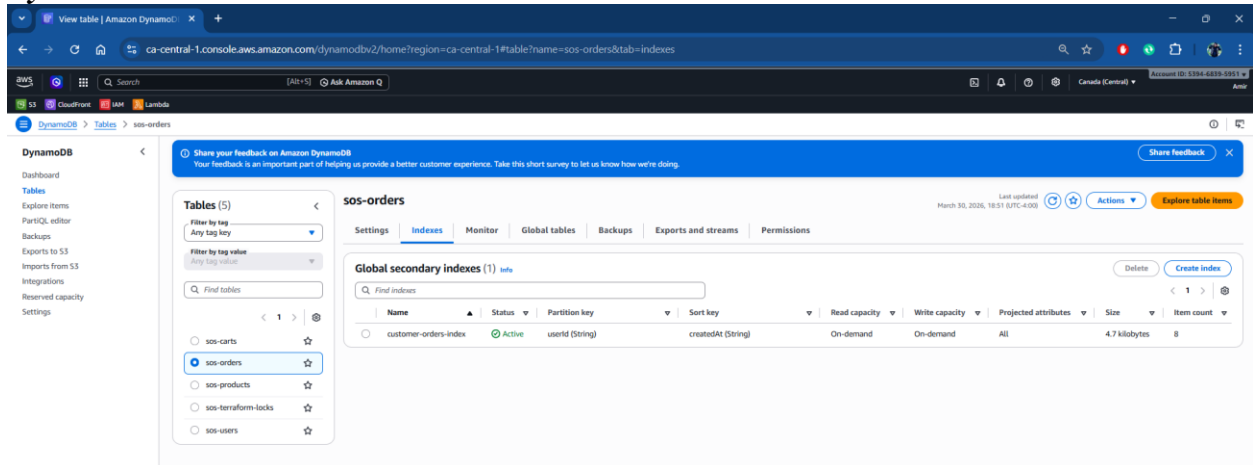


Figure 64: Amazon DynamoDB (sos-orders)

DynamoDB – sos-products

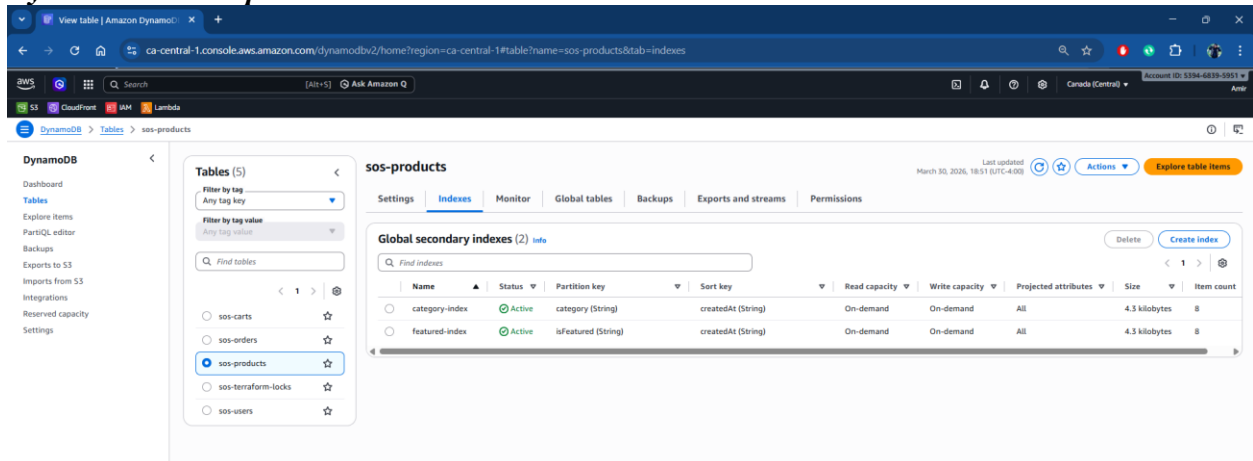


Figure 65: Amazon DynamoDB (sos-products)

DynamoDB – sos-orders – explore items

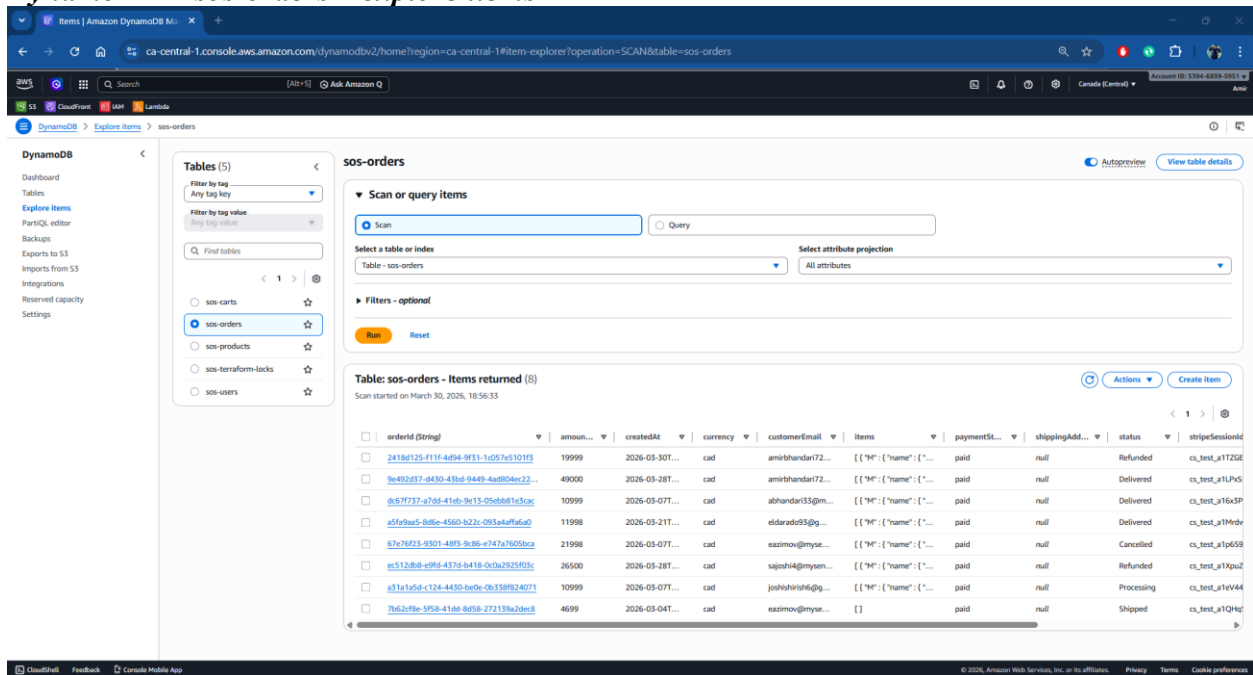


Figure 66: Amazon DynamoDB (sos-orders-explore-item)

DynamoDB – sos-products – explore items

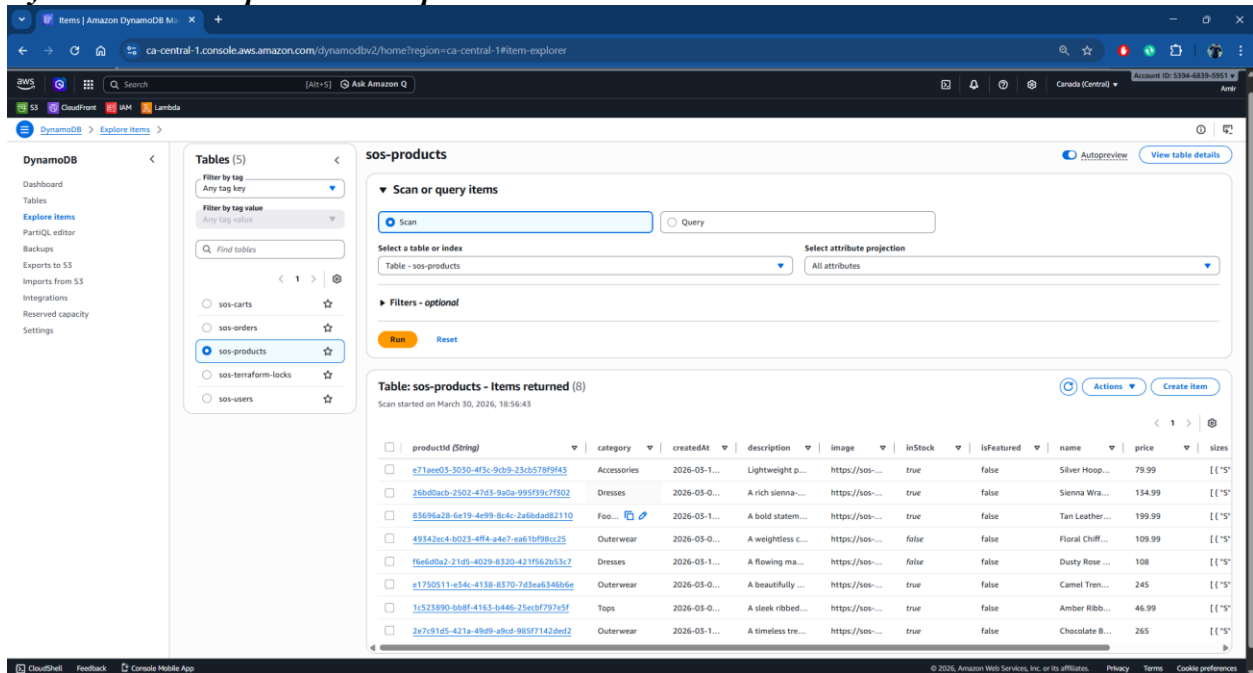


Figure 67: Amazon DynamoDB (sos-products-explore-items)

Integration and Communication Services

These services enable interaction and communication within the system.

Amazon SNS

SNS is used for event-driven communication, triggering workflows like notifications.

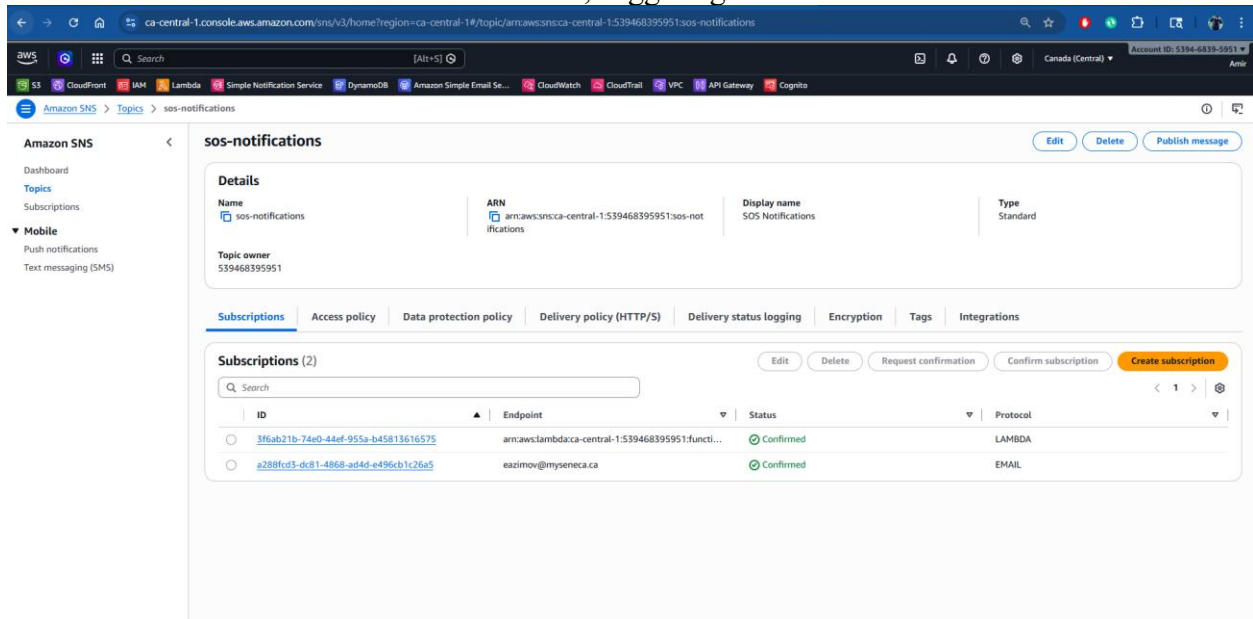


Figure 68: Amazon SNS Topic

Amazon SES

Email-based communication for password recovery is handled by Amazon Cognito. Order-related notifications are delivered through Telegram integration instead of email.

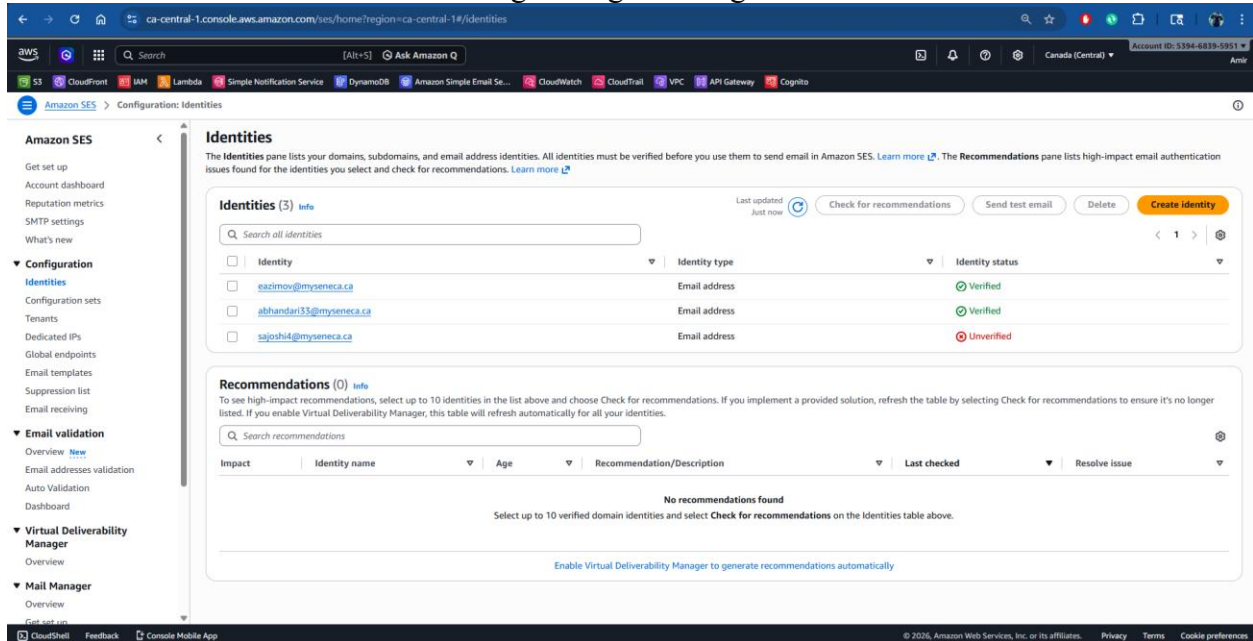


Figure 69: Amazon SES Identities

Amazon Lex

Lex provides a chatbot interface that allows users to interact with the application using natural language.

Amazon LEX - Overview

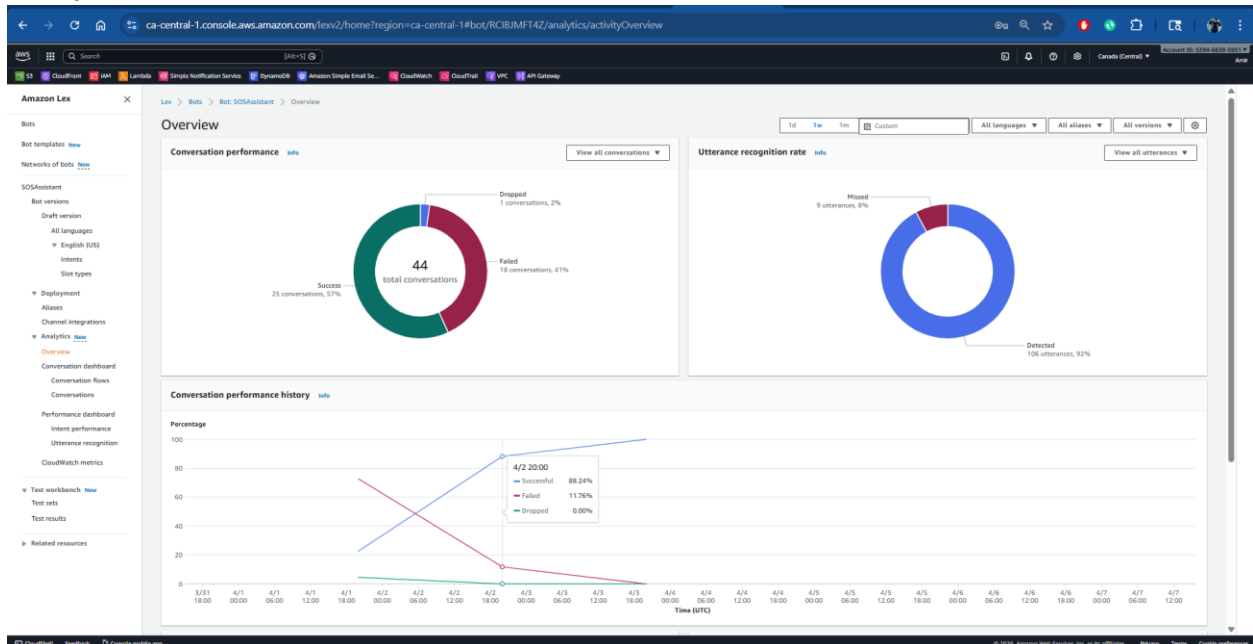


Figure 70: Amazon Lex - Overview

Amazon Lex – Intents

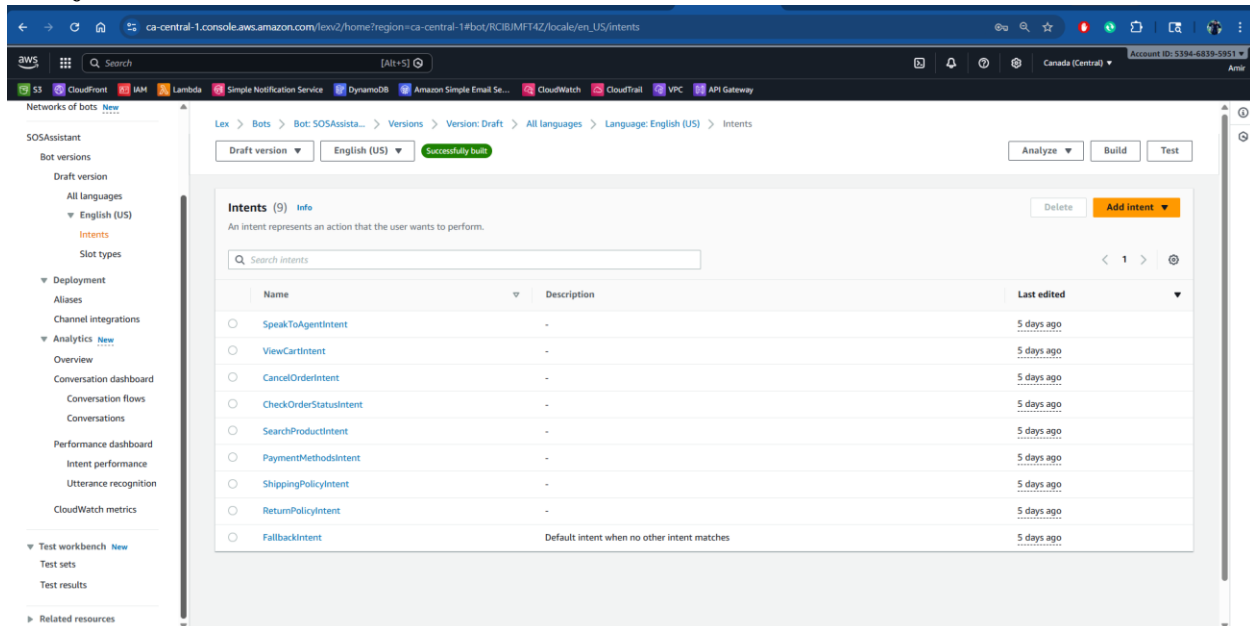


Figure 71: Amazon Lex - Intents

Amazon Lex – Conversation Flows

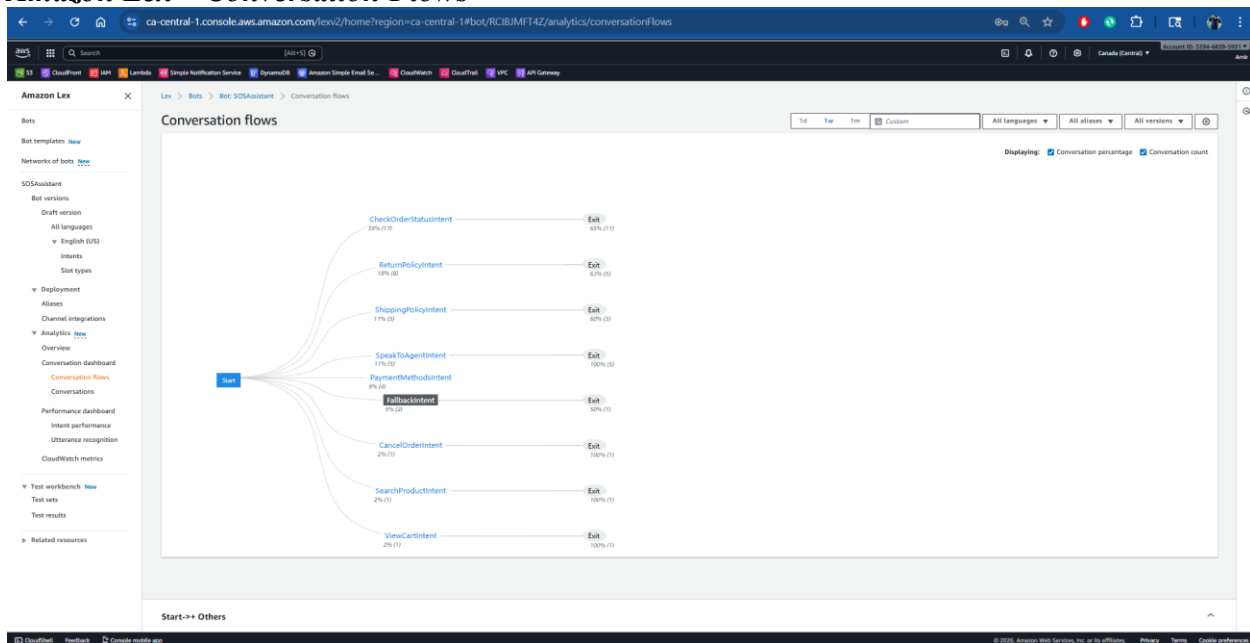


Figure 72: Amazon Lex – Conversation Flows

Amazon Lex – Conversations

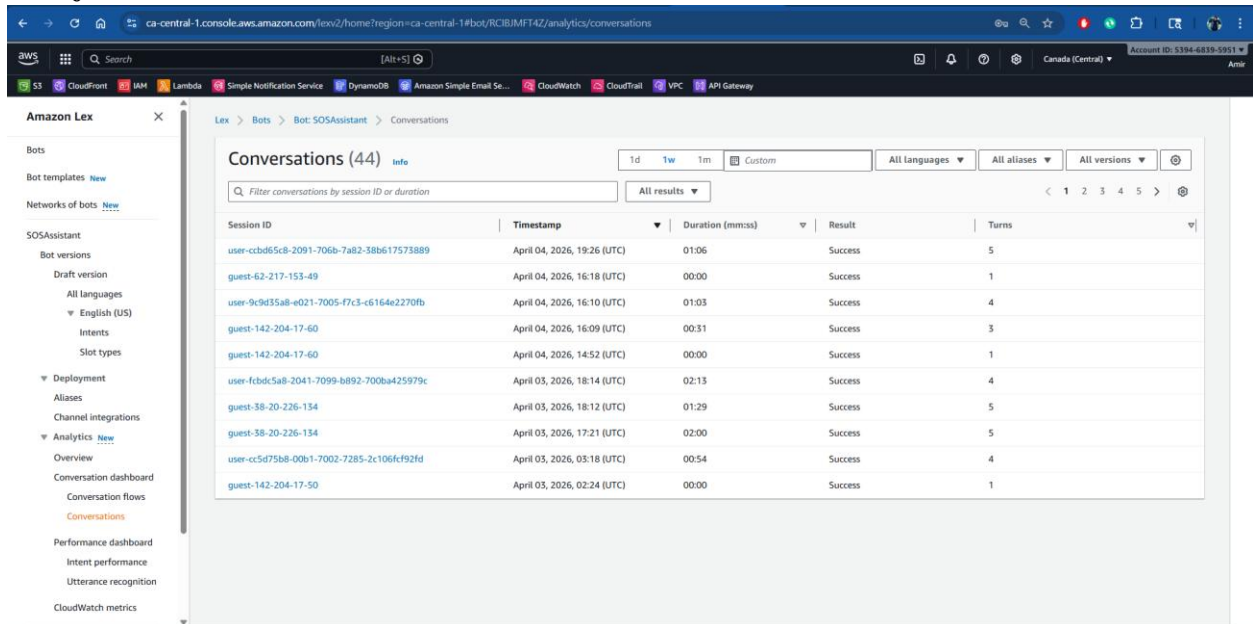


Figure 73: Amazon Lex – Conversations

Monitoring, Logging and Alerting Services

These services ensure system observability and tracking. In addition, real-time alerting is implemented using CloudWatch and SNS, which sends error notifications to a Telegram group using bot integration.

Amazon CloudWatch

CloudWatch collects logs and monitors performance across services.

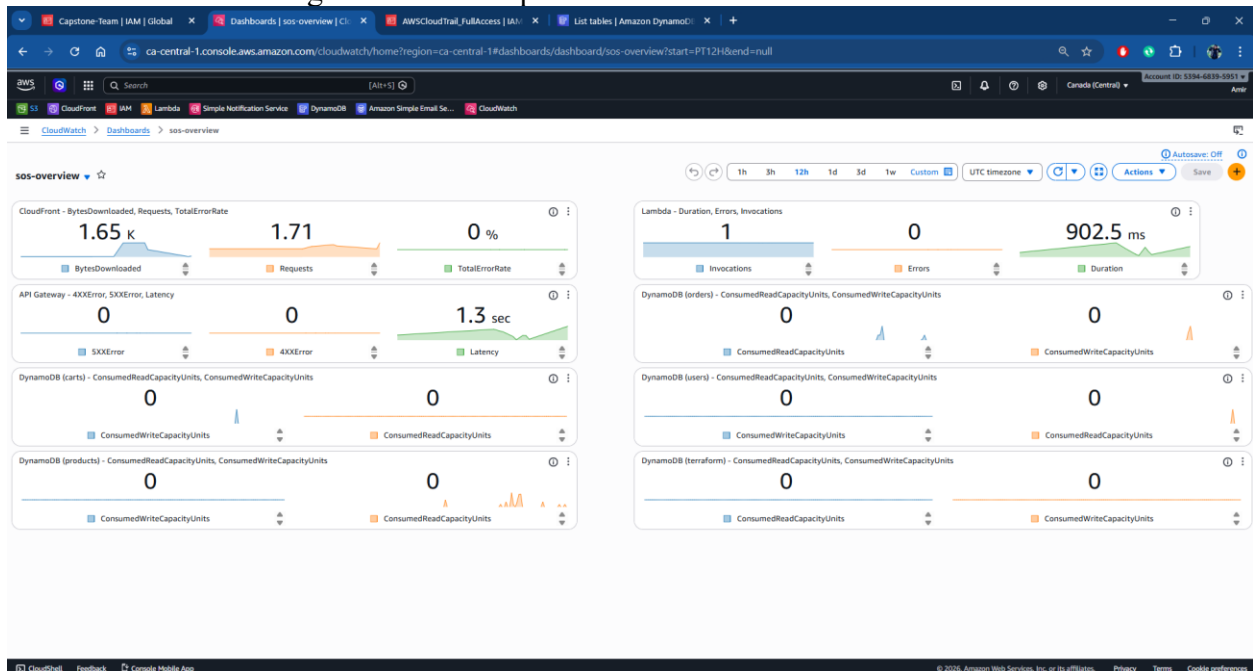


Figure 74: CloudWatch Overview

Cloudwatch – Log Management

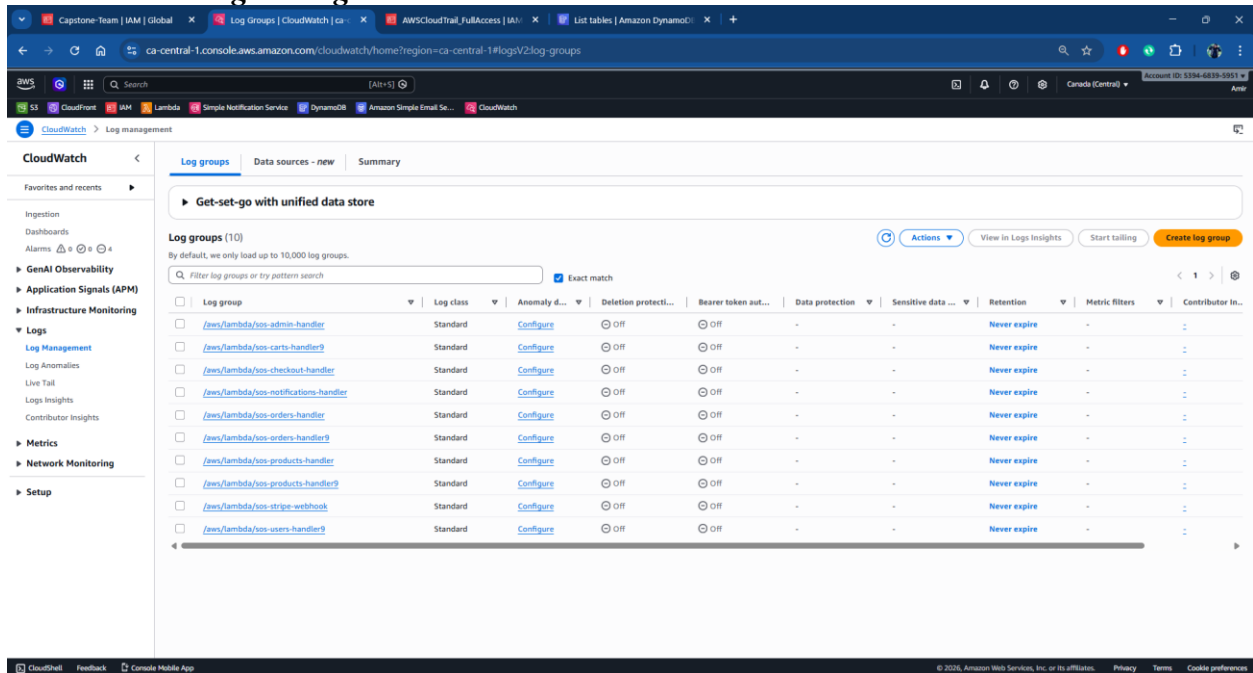


Figure 75: CloudWatch - Log Management

AWS CloudTrail

CloudTrail records all account activities for auditing and compliance.

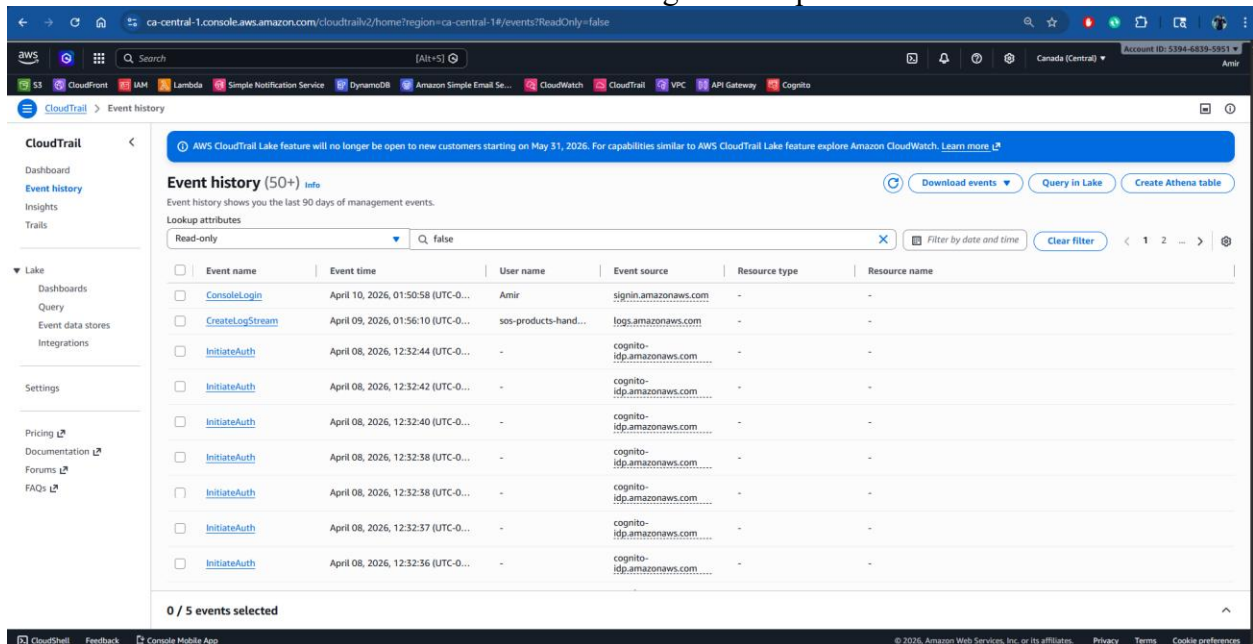


Figure 76: CloudTrail Event History

Telegram Notification Output

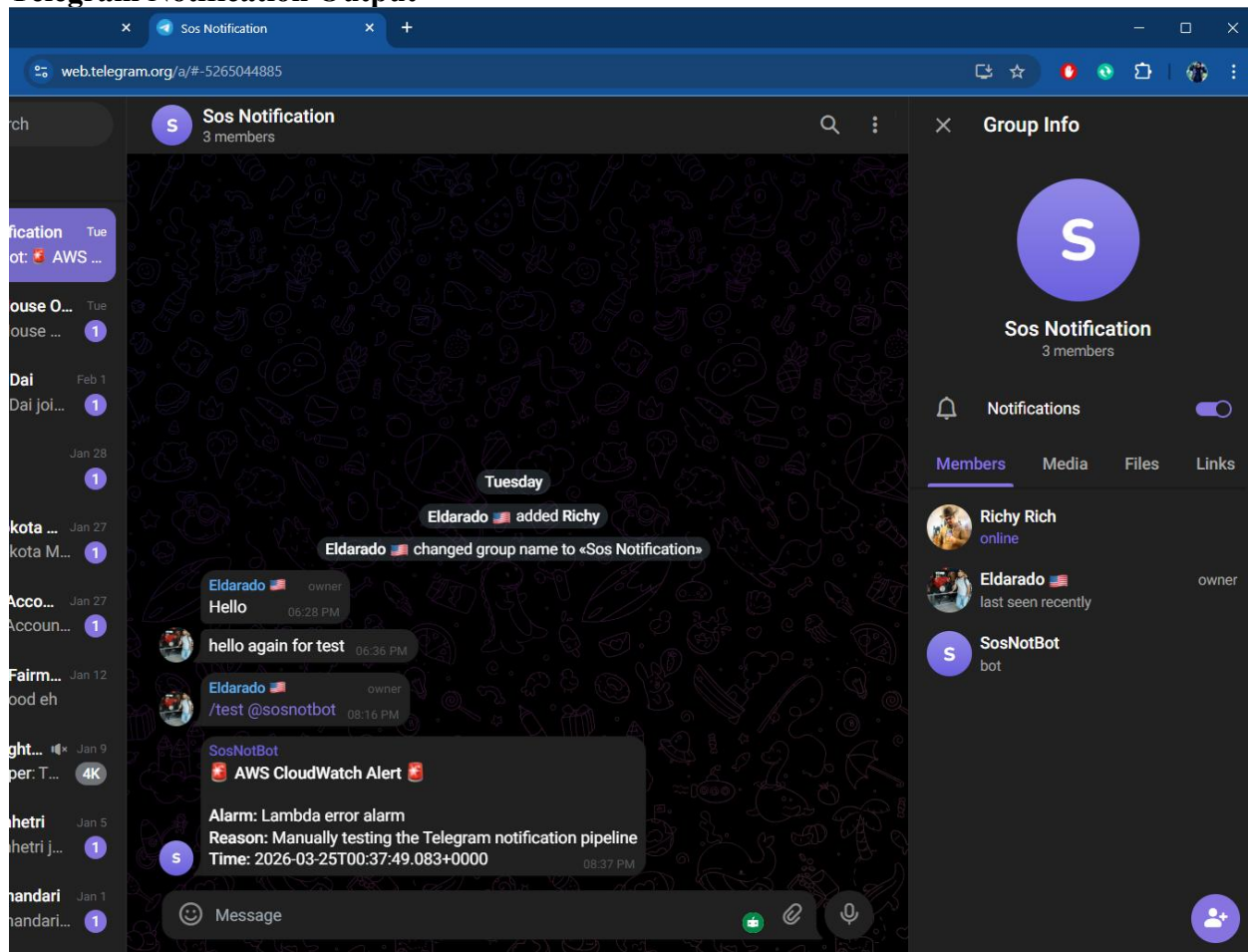


Figure 77: Telegram Notification Output

Security Services

These services ensure the system remains secure and follows best practices.

AWS IAM

IAM controls access permissions between AWS services.

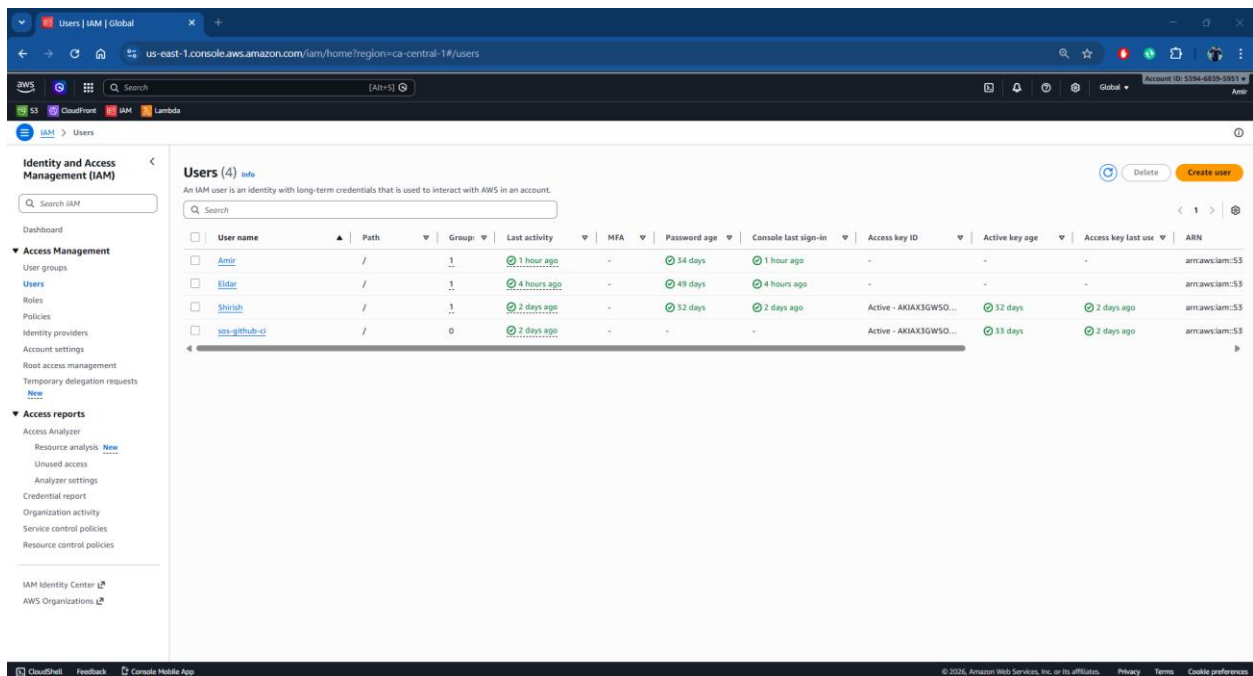


Figure 78: AWS IAM - Users

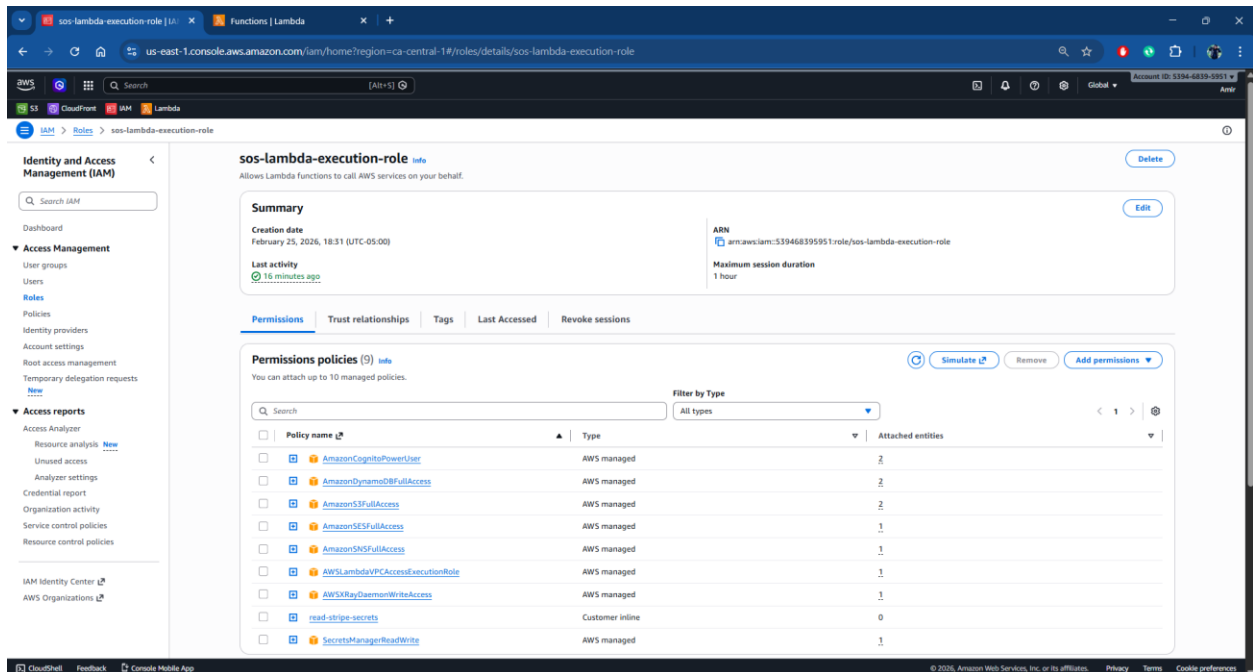


Figure 79: AWS IAM Roles (sos-lambda-execution-role)

AWS KMS

KMS is used to encrypt sensitive data stored in AWS services.

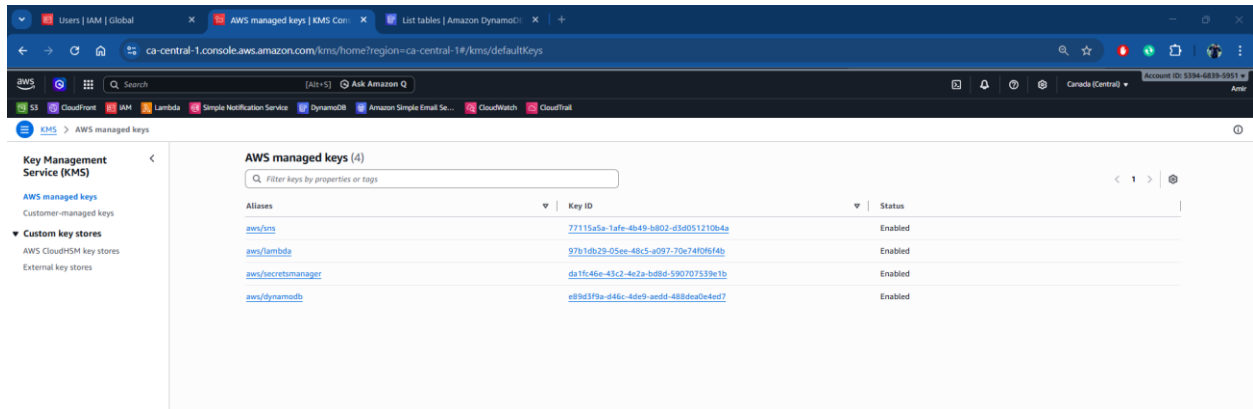


Figure 80: Amazon KMS – AWS Managed Keys

KMS – Customer Managed Keys

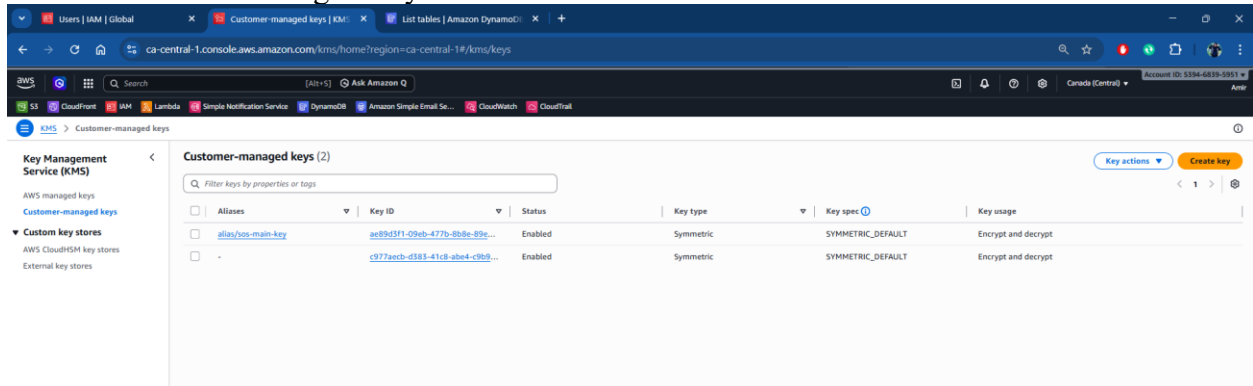


Figure 81: Amazon KMS – Customer Managed Keys

AWS Secrets Manager

Secrets Manager securely stores sensitive credentials such as API keys.

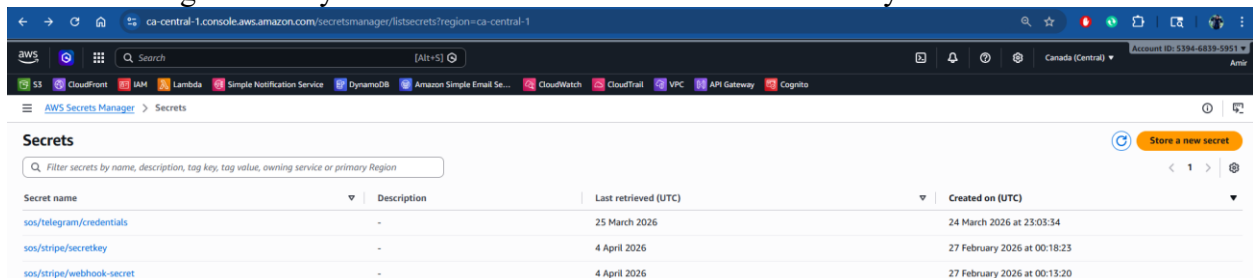


Figure 82: AWS Secrets Manager - Stored Secrets

DevOps and Infrastructure Management

Terraform is used as an Infrastructure as Code (IaC) tool to define and provision AWS resources in a repeatable and consistent manner. It enables automated infrastructure deployment and reduces manual configuration errors.

Terraform Implementation

Terraform is used to provision and manage AWS infrastructure in a consistent and automated manner. Instead of manually configuring resources through the AWS Console, infrastructure components such as IAM roles, backend storage, and supporting services are defined in code. The Terraform setup is integrated with a CI/CD pipeline using GitHub Actions, which securely authenticates to AWS using OpenID Connect (OIDC). This eliminates the need for storing long-term access keys. Terraform state is stored in Amazon S3, while DynamoDB is used for state locking to prevent conflicts during concurrent deployments. This ensures that infrastructure changes are tracked, version-controlled, and applied reliably.

```

github/workflows
infra
  terraform
  modules
  api_gateway
  gkeek
  main.tf
  outputs.tf
  variables.tf
  versions.tf
  terraform.tfstate
  backend.tf
  local.tf
  main.tf
  modules
  providers.tf
  variables.tf
  versions.tf
  terraform.tfstate

Plan: 155 to add, 0 to change, 0 to destroy.

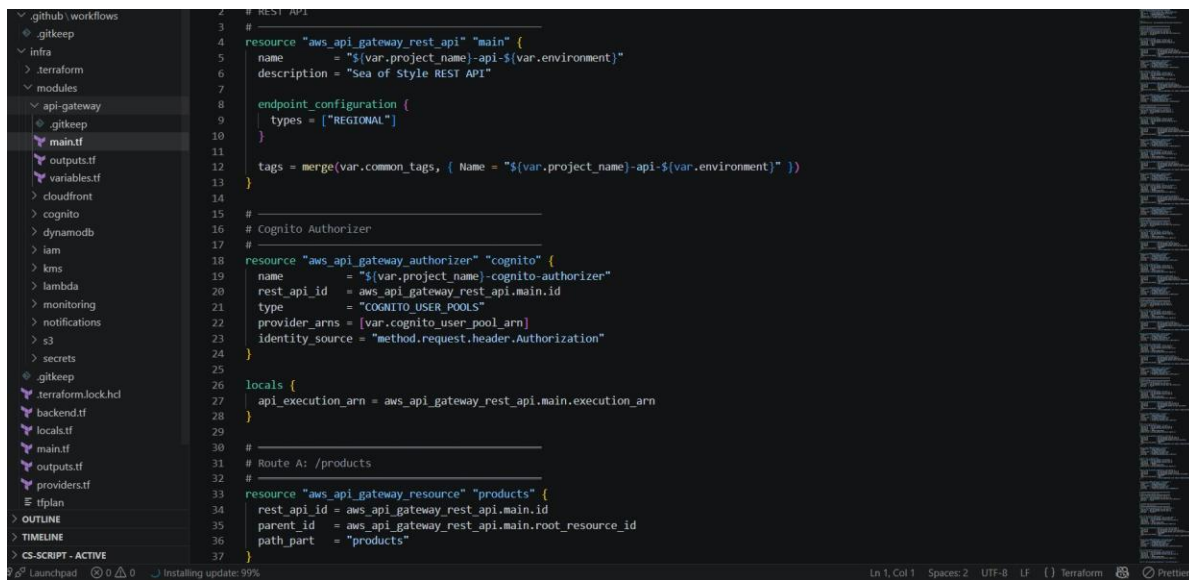
Changes to Outputs:
  + api_gateway_api_id      = (known after apply)
  + api_gateway_execution_arn = (known after apply)
  + api_gateway_invoke_url   = (known after apply)
  + chat_bot_api            = (known after apply)
  + cloudfront_distribution_id = (known after apply)
  + cloudfront_domain_name   = (known after apply)
  + cloudtrail_arn           = (known after apply)
  + cloudtrail_bucket        = "s3-cloudtrail-logs-dev-539468395951"
  + cognito_client_id        = (known after apply)
  + cognito_client_name      = "sso-auth-dev"
  + cognito_pool_arn         = (known after apply)
  + cognito_user_pool_id     = (known after apply)
  + dashboard_name          = "sso-overview-dev"
  + dynamodb_cart_table_arn = (known after apply)
  + dynamodb_cart_table_name = "sso-cart-dev"
  + dynamodb_orders_table_arn = (known after apply)
  + dynamodb_orders_table_name = "sso-orders-dev"
  + dynamodb_products_table_arn = (known after apply)
  + dynamodb_products_table_name = "sso-products-dev"
  + dynamodb_users_table_arn = (known after apply)
  + dynamodb_users_table_name = "sso-users-dev"
  + frontend_bucket_name     = "sso-frontend-dev-539468395951"
  + iam_lambda_role_arn      = (known after apply)
  + iam_lambda_role_name     = "sso-lambda-role-dev"
  + iam_alias                = "alias/sso-main-key-dev"
  + kms_key_arn              = (known after apply)
  + kms_key_id               = (known after apply)
  + lambda_admin_handler_arn = (known after apply)
  + lambda_cart_handler_arn  = (known after apply)
  + lambda_checkout_handler_arn = (known after apply)
  + lambda_notification_handler_arn = (known after apply)
  + lambda_orders_handler_arn = (known after apply)
  + lambda_products_handler_arn = (known after apply)
  + lambda_stripe_webhook_arn = (known after apply)
  + lambda_users_handler_arn = (known after apply)
  + s3_assets_bucket_arn     = (known after apply)
  + s3_assets_bucket_name    = "sso-assets-dev-539468395951"
  + s3_logs_bucket_arn      = (known after apply)
  + s3_logs_bucket_name     = "sso-logs-dev-539468395951"
  + s3_notifications_bucket_arn = (known after apply)
  + s3_notifications_bucket_name = "sso-notifications-dev-539468395951"
  + ses_from_email           = "noreply@seneca.com"
  + ses_topic_arn            = (known after apply)
  + sns_topic_name           = "sso-notifications-dev"
  + stripe_secret_key_arn    = (known after apply)
  + stripe_webhook_secret_arn = (known after apply)

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
PS C:\Projects\use-of-style-infra\infra>
  
```

Figure 83: Terraform plan output

API Gateway Module

The api-gateway module defines the REST API that serves as the entry point for all client requests. The API is configured with a REGIONAL endpoint type, meaning it is deployed to a specific AWS region rather than distributed globally through CloudFront edge locations. A Cognito authorizer is wired directly to the Cognito user pool ARN, so every protected route validates the incoming JWT token against the user pool before the request reaches a Lambda function. API routes (such as /products) are defined as resources with method integrations pointing to their respective Lambda handlers.



```

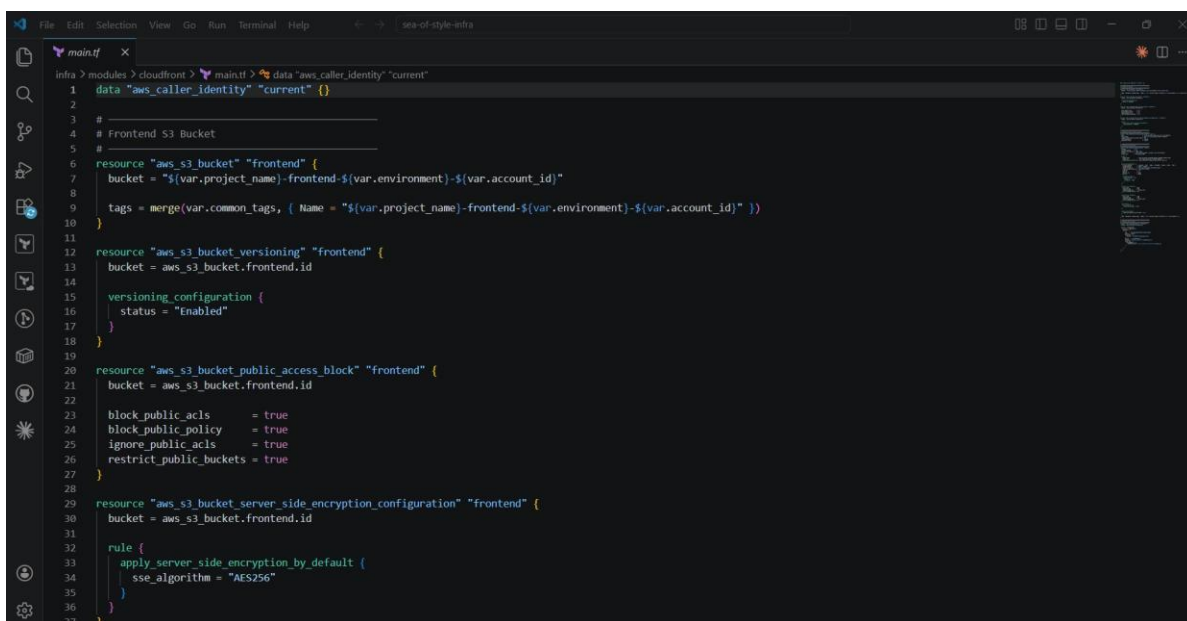
2 # REST API
3 #
4 resource "aws_api_gateway_rest_api" "main" {
5   name       = "${var.project_name}-api-${var.environment}"
6   description = "Sea of Style REST API"
7
8   endpoint_configuration {
9     types = ["REGIONAL"]
10  }
11
12  tags = merge(var.common_tags, { Name = "${var.project_name}-api-${var.environment}" })
13 }
14
15 #
16 # Cognito Authorizer
17 #
18 resource "aws_api_gateway_authorizer" "cognito" {
19   name       = "${var.project_name}-cognito-authorizer"
20   rest_api_id = aws_api_gateway_rest_api.main.id
21   type       = "COGNITO_USER_POOLS"
22   provider_arns = [var.cognito_user_pool_arn]
23   identity_source = "method.request.header.Authorization"
24 }
25
26 locals {
27   api_execution_arn = aws_api_gateway_rest_api.main.execution_arn
28 }
29
30 #
31 # Route A: /products
32 #
33 resource "aws_api_gateway_resource" "products" {
34   rest_api_id = aws_api_gateway_rest_api.main.id
35   parent_id   = aws_api_gateway_rest_api.main.root_resource_id
36   path_part   = "products"
37 }

```

Figure 84: api-gateway - main.tf file

CloudFront Module: Frontend Hosting

The CloudFront module provisions the S3 bucket that hosts the SOS frontend, alongside all security and CDN configuration. The bucket name is built dynamically from the project name, environment, and AWS account ID to ensure global uniqueness. Versioning is enabled so every deployed build is preserved and can be rolled back if needed. All four public access block settings are set to true, meaning no object can ever be made publicly accessible directly through S3. Server-side encryption is applied with AES-256 so all stored assets are encrypted at rest. Public traffic reaches the frontend exclusively through the CloudFront distribution, which enforces HTTPS and provides edge caching.



```

1 data "aws_caller_identity" "current" {}
2
3 #
4 # Frontend S3 Bucket
5 #
6 resource "aws_s3_bucket" "frontend" {
7   bucket = "${var.project_name}-frontend-${var.environment}-${var.account_id}"
8
9   tags = merge(var.common_tags, { Name = "${var.project_name}-frontend-${var.environment}-${var.account_id}" })
10 }
11
12 resource "aws_s3_bucket_versioning" "frontend" {
13   bucket = aws_s3_bucket.frontend.id
14
15   versioning_configuration {
16     status = "Enabled"
17   }
18 }
19
20 resource "aws_s3_bucket_public_access_block" "frontend" {
21   bucket = aws_s3_bucket.frontend.id
22
23   block_public_acls       = true
24   block_public_policy     = true
25   ignore_public_acls     = true
26   restrict_public_buckets = true
27 }
28
29 resource "aws_s3_bucket_server_side_encryption_configuration" "frontend" {
30   bucket = aws_s3_bucket.frontend.id
31
32   rule {
33     apply_server_side_encryption_by_default {
34       sse_algorithm = "AES256"
35     }
36   }
37 }

```

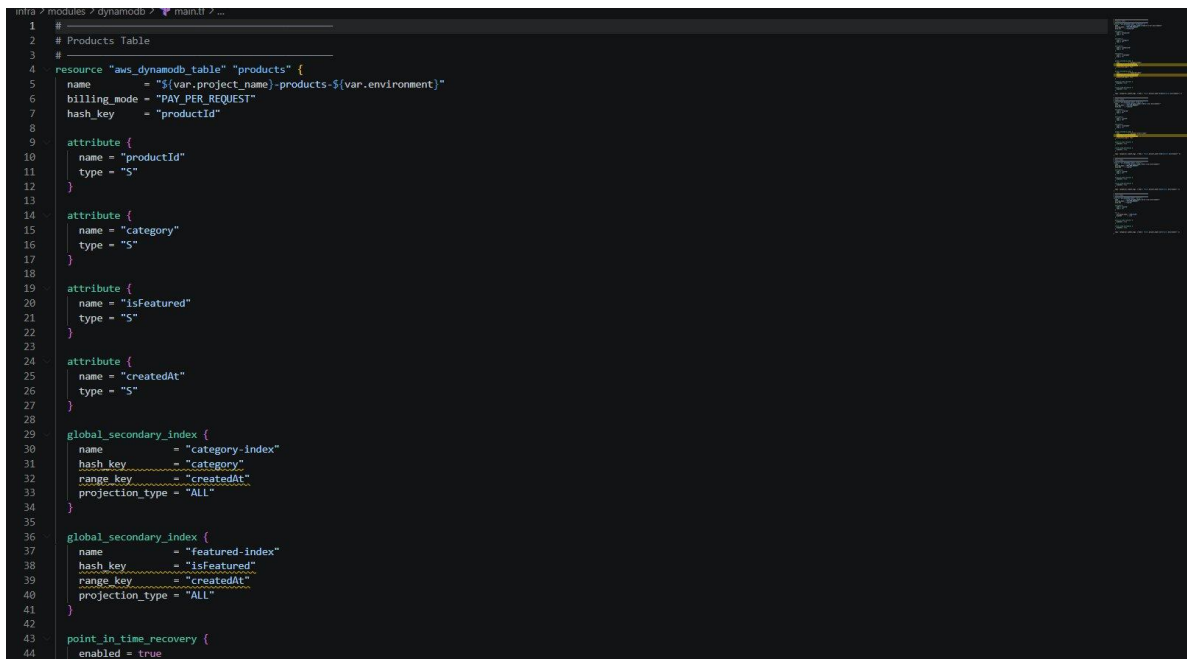
Figure 85: CloudFront Module - S3 Bucket, Versioning, and Encryption

```

main.tf
infra > modules > cloudfront > main.tf > ...
1 data "aws_caller_identity" "current" {}
2
3 # Frontend S3 Bucket
4
5 resource "aws_s3_bucket" "frontend" {
6   bucket = "${var.project_name}-frontend-${var.environment}-${var.account_id}"
7   tags = merge(var.common_tags, { Name = "${var.project_name}-frontend-${var.environment}-${var.account_id}" })
8 }
9
10 resource "aws_s3_bucket_versioning" "frontend" {
11   bucket = aws_s3_bucket.frontend.id
12   versioning_configuration {
13     status = "Enabled"
14   }
15 }
16
17 resource "aws_s3_bucket_public_access_block" "frontend" {
18   bucket = aws_s3_bucket.frontend.id
19   block_public_acls = true
20   block_public_policy = true
21   ignore_public_acls = true
22   restrict_public_buckets = true
23 }
24
25 resource "aws_s3_bucket_server_side_encryption_configuration" "frontend" {
26   bucket = aws_s3_bucket.frontend.id
27   server_side_encryption_configuration {
28     rules {
29       rule {
30         apply_server_side_encryption_by_default {
31           sse_algorithm = "AES256"
32         }
33       }
34     }
35   }
36 }
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
26
```

DynamoDB Module: Application Tables

All application data tables are defined in the DynamoDB module. Each table uses PAY_PER_REQUEST billing so capacity scales automatically with traffic and there is no need to manage provisioned throughput. Point-in-time recovery is enabled on every table, providing a continuous backup window without manual snapshots. The products table illustrates the design pattern used across all tables. The primary key is productId. Two Global Secondary Indexes are defined: category-index (partitioned by category, sorted by createdAt) and featured-index (partitioned by isFeatured, sorted by createdAt). These GSIs enable efficient filtered queries for product listings and the homepage without performing full table scans.



```
1 #
2 # Products Table
3 #
4 resource "aws_dynamodb_table" "products" {
5   name           = "${var.project_name}-products-${var.environment}"
6   billing_mode    = "PAY_PER_REQUEST"
7   hash_key       = "productId"
8
9   attribute {
10    name = "productId"
11    type = "S"
12  }
13
14  attribute {
15    name = "category"
16    type = "S"
17  }
18
19  attribute {
20    name = "isFeatured"
21    type = "S"
22  }
23
24  attribute {
25    name = "createdAt"
26    type = "S"
27  }
28
29  global_secondary_index {
30    name           = "category-index"
31    hash_key       = "category"
32    range_key      = "createdAt"
33    projection_type = "ALL"
34  }
35
36  global_secondary_index {
37    name           = "featured-index"
38    hash_key       = "isFeatured"
39    range_key      = "createdAt"
40    projection_type = "ALL"
41  }
42
43  point_in_time_recovery {
44    enabled = true
45  }
46 }
```

Figure 88: DynamoDB Module - Products Table with GSIs

IAM Module: Execution Role and Policies

All Lambda functions share a single execution role provisioned by the IAM module. The trust policy allows only the lambda.amazonaws.com service principal to assume the role via sts:AssumeRole, ensuring no other service can use these permissions. A least privilege inline policy is attached to the role and broken into named permission statements. The CloudWatchLogs statement grants only the three log actions needed for Lambda to write execution logs, scoped to the project log group prefix. The S3BucketAccess statement grants only s3:GetObject and s3:PutObject, limited to the relevant buckets. Additional statements cover DynamoDB, Cognito, KMS, Secrets Manager, SNS, and SES, each scoped to the minimum actions required. Every permission change goes through code review before reaching AWS.

```

infra > modules > iam > main.tf > resource "aws_iam_policy" "lambda"
1
2 # Lambda Execution Role
3
4 resource "aws_iam_role" "lambda" {
5   name = "${var.project_name}-lambda-role-${var.environment}"
6
7   assume_role_policy = jsonencode({
8     Version = "2012-10-17"
9     Statement = [
10       {
11         Effect = "Allow"
12         Principal = { Service = "lambda.amazonaws.com" }
13         Action = "sts:AssumeRole"
14       }
15     ]
16   })
17
18   tags = merge(var.common_tags, { Name = "${var.project_name}-lambda-role-${var.environment}" })
19 }
20
21 # Lambda Policy
22
23 resource "aws_iam_policy" "lambda" {
24   name = "${var.project_name}-lambda-policy-${var.environment}"
25
26   policy = jsonencode({
27     Version = "2012-10-17"
28     Statement = [
29       {
30         Sid = "CloudWatchLogs"
31         Effect = "Allow"
32         Action = [
33           "logs:CreateLogGroup",
34           "logs:CreateLogStream",
35           "logs:PutLogEvents"
36         ]
37         Resource = "arn:aws:logs:*:*:log-group:/aws/lambda/${var.project_name}-*"
38       },
39       {
40         Sid = "S3BucketAccess"
41         Effect = "Allow"
42         Action = [
43           "s3:GetObject",
44           "s3:PutObject"
45         ]

```

Figure 89: IAM Module - Lambda Role and Policy

Lambda Module: Serverless Functions

The Lambda module provisions all 8 serverless functions. A shared locals block defines the common configuration applied to every function: Node.js 20.x runtime, ARM64 (Graviton) architecture for cost efficiency, 512 MB memory, a 30-second timeout, the shared IAM execution role, and X-Ray active tracing. This avoids repeating the same settings across every function resource. A placeholder ZIP archive is generated at plan time using the `archive_file` data source. This allows Terraform to create the Lambda functions on the first apply before any real deployment artifact exists. The CI/CD pipeline for application code then updates the function code separately via a deploy workflow. Each function resource inherits all settings from the locals block and adds only its unique function name, description, and handler path.

```

infra > modules > lambda > main.tf > data "aws_caller_identity" "current"
1
2 data "aws_caller_identity" "current" {}
3
4 # Placeholder ZIP
5
6 data "archive_file" "placeholder" {
7   type = "zip"
8   output_path = "${path.module}/placeholder.zip"
9   source {
10     content = "exports.handler = async () => ({ statusCode: 200 });"
11     filename = "index.js"
12   }
13 }
14
15 resource "null_resource" "placeholder" {}
16
17 # Shared config
18
19 locals {
20   common_lambda_config = {
21     runtime = "nodejs20.x"
22     architectures = ["arm64"]
23     memory_size = 512
24     timeout = 30
25     role = var.lambda_role_arn
26     tracing_config_mode = "Active"
27   }
28 }
29
30 # Products Handler
31
32 resource "aws_lambda_function" "products_handler" {
33   function_name = "${var.project_name}-products-handler-${var.environment}"
34   description = "Products CRUD and image upload URLs"
35   filename = data.archive_file.placeholder.output_path
36   handler = "index.handler"
37   runtime = local.common_lambda_config.runtime
38   architectures = local.common_lambda_config.architectures
39   memory_size = local.common_lambda_config.memory_size
40   timeout = local.common_lambda_config.timeout
41   role = local.common_lambda_config.role
42   publish = false
43 }
44
45

```

Figure 90: Lambda Module - Shared Config and Function Definition

Monitoring Module: CloudTrail and Audit Logging

The monitoring module configures AWS CloudTrail to capture a complete, tamper-resistant audit log of all API activity across the account. CloudTrail writes logs to a dedicated S3 bucket with its own specific bucket policy. The public access block on this bucket is set to fully private. The bucket policy defines two statements: `AWSCloudTrailAclCheck` allows CloudTrail to call `s3:GetBucketAcl`, and `AWSCloudTrailWrite` allows `s3:PutObject` for the `AWSLogs` path scoped to the account ID. A `StringEquals` condition on the `PutObject` statement restricts writes to this account only, preventing cross-account log injection. CloudWatch log groups for each Lambda function are also provisioned here, ensuring log retention policies are set at the infrastructure level.

```
1 # CloudTrail S3 Bucket
2 #
3 resource "aws_s3_bucket" "cloudtrail" {
4   bucket = "${var.project_name}-cloudtrail-logs-${var.environment}-539468395951"
5   tags = merge(var.common_tags, { Name = "${var.project_name}-cloudtrail-logs-${var.environment}" })
6 }
7
8 resource "aws_s3_bucket_public_access_block" "cloudtrail" {
9   bucket = aws_s3_bucket.cloudtrail.id
10   block_public_acls = true
11   block_public_policy = true
12   ignore_public_acls = true
13   restrict_public_buckets = true
14 }
15
16 resource "aws_s3_bucket_policy" "cloudtrail" {
17   bucket = aws_s3_bucket.cloudtrail.id
18   depends_on = [aws_s3_bucket_public_access_block.cloudtrail]
19   policy = jsonencode({
20     Version = "2012-10-17"
21     Statement = [
22       {
23         Sid = "AWSCloudTrailAclCheck"
24         Effect = "Allow"
25         Principal = {
26           Service = "cloudtrail.amazonaws.com"
27         }
28         Action = "s3:GetBucketAcl"
29         Resource = aws_s3_bucket.cloudtrail.arn
30       },
31       {
32         Sid = "AWSCloudTrailWrite"
33         Effect = "Allow"
34         Principal = {
35           Service = "cloudtrail.amazonaws.com"
36         }
37         Action = "s3:PutObject"
38         Resource = "${aws_s3_bucket.cloudtrail.arn}/AWSLogs/${var.account_id}/*"
39         Condition = {
40           StringEquals = {
41             "aws:PrincipalAccount" = var.account_id
42           }
43         }
44       }
45     ]
46   })
47 }
```

Figure 91: Monitoring Module - CloudTrail S3 Bucket and Policy

Notifications Module

The SOS platform uses Amazon SNS to enable event-driven communication within the system. When a user completes a purchase, an event is generated and processed through SNS, which triggers a notification workflow. Instead of sending email confirmations, the system forwards these notifications to a Telegram bot using a webhook integration, allowing administrators to receive real-time purchase alerts. For user account-related actions such as password recovery, Amazon Cognito is used. Cognito automatically sends verification emails containing a reset code, ensuring a secure and managed password reset process without requiring custom email handling.

```

infra > modules > notifications > main.tf > ...
1 #
2 # SNS Topic
3 #
4 resource "aws_sns_topic" "notifications" {
5   name = "${var.project_name}-notifications-${var.environment}"
6   display_name = "SNS Notifications"
7   tags = merge(var.common_tags, { Name = "${var.project_name}-notifications-${var.environment}" })
8 }
9
10
11 resource "aws_sns_topic_subscription" "admin_email" {
12   topic_arn = aws_sns_topic.notifications.arn
13   protocol = "email"
14   endpoint = var.admin_email
15 }
16
17 resource "aws_sns_topic_subscription" "lambda" {
18   topic_arn = aws_sns_topic.notifications.arn
19   protocol = "lambda"
20   endpoint = var.lambda_notifications_arn
21 }
22
23 resource "aws_lambda_permission" "sns_invoke" {
24   statement_id = "AllowSNSInvoke"
25   action = "lambda:InvokeFunction"
26   function_name = var.lambda_notifications_arn
27   principal = "sns.amazonaws.com"
28   source_arn = aws_sns_topic.notifications.arn
29 }
30
31 #
32 # SES Email Identities
33 # NOTE: AWS will send verification emails to both addresses.
34 # Both must be verified before SES can send in sandbox mode.
35 #
36 resource "aws_ses_email_identity" "from" {
37   email = var.ses_from_email
38 }
39
40 resource "aws_ses_email_identity" "admin" {
41   email = var.admin_email
42 }
43
44 #

```

Figure 92: Notifications Module - SNS Topic and SES Identities

S3 Module: Data Buckets

The S3 module provisions three runtime data buckets: assets (product images and media), uploads (user-uploaded content), and logs (application and access logs). Each bucket follows the same security baseline as the frontend bucket: versioning enabled, full public access block applied, and AES-256 server-side encryption by default. Bucket names are constructed from the project name, environment, and AWS account ID for global uniqueness. Separating these buckets from the CloudFront module keeps concerns isolated. The frontend hosting lifecycle is independent of the data storage lifecycle, and each bucket can have different retention policies, lifecycle rules, or access controls applied without affecting the others.

```

infra > modules > s3 > main.tf > %data.aws_caller_identity.current
1 data "aws_caller_identity" "current" {}
2
3 #
4 # Assets Bucket
5 #
6 resource "aws_s3_bucket" "assets" {
7   bucket = "${var.project_name}-assets-${var.environment}-${data.aws_caller_identity.current.account_id}"
8   tags = merge(var.common_tags, { Name = "${var.project_name}-assets-${var.environment}-${data.aws_caller_identity.current.account_id}" })
9 }
10
11
12 resource "aws_s3_bucket_versioning" "assets" {
13   bucket = aws_s3_bucket.assets.id
14   versioning_configuration {
15     status = "Enabled"
16   }
17 }
18
19
20 resource "aws_s3_bucket_public_access_block" "assets" {
21   bucket = aws_s3_bucket.assets.id
22   block_public_acls = true
23   block_public_policy = true
24   ignore_public_acls = true
25   restrict_public_buckets = true
26 }
27
28
29 resource "aws_s3_bucket_server_side_encryption_configuration" "assets" {
30   bucket = aws_s3_bucket.assets.id
31   rule {
32     apply_server_side_encryption_by_default {
33       sse_algorithm = "AES256"
34     }
35   }
36 }
37
38 #
39 # Uploads Bucket
40 #
41
42 resource "aws_s3_bucket" "uploads" {
43   bucket = "${var.project_name}-uploads-${var.environment}-${data.aws_caller_identity.current.account_id}"
44 }

```

Figure 93: S3 Module - Assets and Uploads Buckets

Secrets Module: KMS and Secrets Manager

The secrets module provisions the customer-managed KMS key used to encrypt sensitive data across the platform. Automatic key rotation is enabled, meaning AWS generates a new backing key every year without changing the key ARN or alias. The key policy defines three explicit permission statements: the root account receives full kms:* administrative access; the CloudTrail service receives kms:GenerateDataKey* and kms:Decrypt with a StringLike condition scoped to this account's trails; and the Lambda execution role receives kms:Decrypt so functions can read secrets from Secrets Manager at runtime. Stripe API keys are stored as secrets in AWS Secrets Manager and encrypted with this KMS key. By defining the key policy in code, every change to who can encrypt or decrypt data goes through the same pull request review process as all other infrastructure changes.

```
infra > modules > secrets > main.tf > ...
1 # =====
2 # KMS Key
3 # =====
4 resource "aws_kms_key" "main" {
5   description = "KMS key for ${var.project_name} ${var.environment}"
6   deletion_window_in_days = 7
7   enable_key_rotation = true
8 }
9
10 policy = jsonencode({
11   Version = "2012-10-17"
12   Statement = [
13     {
14       Sid = "Enable IAM User Permissions"
15       Effect = "Allow"
16       Principal = {
17         AWS = "arn:aws:iam:${var.account_id}:root"
18       }
19       Action = "kms:*"
20       Resource = "*"
21     },
22     {
23       Sid = "Allow CloudTrail to encrypt logs"
24       Effect = "Allow"
25       Principal = {
26         Service = "cloudtrail.amazonaws.com"
27       }
28       Action = [
29         "kms:GenerateDataKey*",
30         "kms:Decrypt"
31       ]
32       Resource = "*"
33       Condition = {
34         StringLike = {
35           "kms:EncryptionContext:aws:cloudtrail:arn" = "arn:aws:cloudtrail:*:${var.account_id}:trail/*"
36         }
37       }
38     },
39     {
40       Sid = "Allow lambda role to use KMS"
41       Effect = "Allow"
42       Principal = {
43         AWS = var.lambda_role_arn
44       }
45       Action = [
46         "kms:Decrypt",
47       ]
48     }
49   ]
50 })
```

Figure 94: Secrets Module - KMS Key and Policy

CI/CD Pipeline and Automation

All Terraform operations run automatically through GitHub Actions. When a developer opens a pull request, Terraform generates a plan showing exactly what will change. On merge to main, it applies those changes to AWS automatically. No one touches the AWS console to make infrastructure changes.

AWS credentials are never stored as static secrets in GitHub. The pipeline uses an IAM OIDC identity provider, which grants the GitHub Actions runner a short-lived role with the minimum required permissions — only for the duration of the deploy run. Terraform state is stored in an S3 bucket in ca-central-1 with versioning enabled, preserving the full history of every infrastructure change. A DynamoDB table prevents two deployments from running at the same time and conflict with each other.

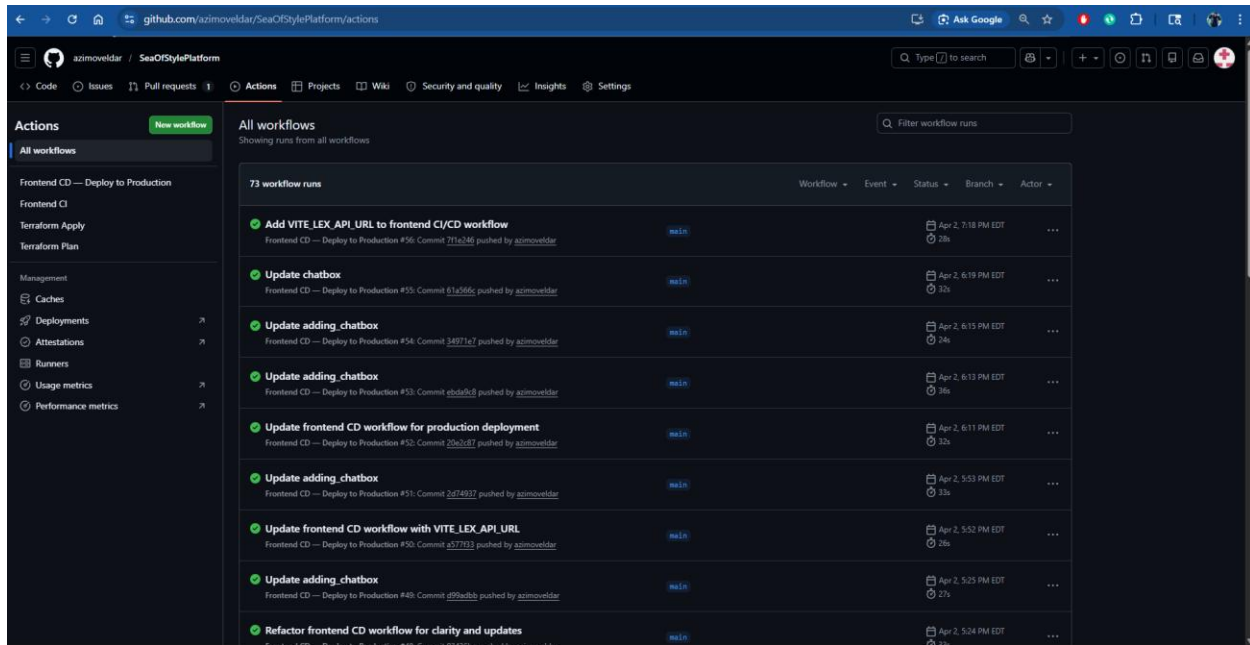


Figure 95: GitHub Actions Workflow

Why have we used these Service?

The selected AWS services were chosen to simplify development while ensuring the system remains scalable and efficient. By using serverless components such as AWS Lambda, API Gateway, and DynamoDB, there is no need to manage servers or infrastructure manually. This allows the application to automatically scale based on user demand and reduces operational complexity.

Amazon CloudFront is used to improve performance by delivering content from edge locations closer to users, resulting in faster load times. Amazon Cognito handles user authentication, removing the need to build and maintain a custom authentication system.

Overall, this approach focuses on reducing maintenance effort while improving performance, scalability, and reliability, making the system easier to manage and more suitable for real-world cloud applications.

AWS Well-Architected Framework Compliance

The SOS platform is designed in alignment with the AWS Well-Architected Framework, ensuring that the system follows industry best practices across all six pillars. The architecture leverages managed and serverless services to achieve a balance between performance, security, reliability, and cost efficiency.

Operational Excellence

The system is designed to be easy to monitor, manage, and continuously improve. Amazon CloudWatch is used to collect logs and track performance metrics, providing visibility into application behavior. In addition to basic monitoring, a real-time alerting mechanism is implemented. When a Lambda function encounters an error, a CloudWatch alarm is triggered,

which sends a notification through Amazon SNS and delivers it to a Telegram group using bot integration. This allows the team to respond quickly to issues as they occur. Furthermore, Terraform is used to manage infrastructure as code, and GitHub Actions is used for automated deployments. This ensures consistent deployment and reduces manual configuration errors.

Security

Security is implemented using a layered approach across the system. Amazon Cognito handles user authentication, including login, registration, and password recovery. The “Forgot Password” feature allows users to securely reset their credentials using email verification. AWS IAM is used to enforce role-based access control, ensuring that services only have the permissions they require. Sensitive data is encrypted at rest using AWS KMS.

Additionally, payment processing is handled through Stripe, which ensures that sensitive financial data is not stored within the system. AWS CloudTrail records all API activity, providing full visibility for auditing and security monitoring.

Reliability

The system is designed to remain available even during failures. AWS Lambda runs across multiple Availability Zones, ensuring that backend services continue to function without interruption. Amazon DynamoDB provides built-in replication and high availability for application data. CloudFront improves reliability by caching content and serving it even if backend services experience temporary issues. This combination ensures that the application remains accessible and stable under different conditions.

Performance Efficiency

Performance is optimized by using scalable and distributed services. CloudFront reduces latency by delivering content from edge locations closer to users. AWS Lambda automatically scales based on incoming requests, allowing the system to handle traffic spikes without performance degradation. DynamoDB provides fast and consistent data access, which is essential for real-time operations such as product browsing and order processing. The serverless design ensures that resources are allocated dynamically based on demand.

Cost Optimization

The architecture follows a pay-as-you-use model, ensuring that costs are aligned with actual usage. Services such as Lambda and API Gateway eliminate the need for continuously running infrastructure, reducing idle costs. DynamoDB on-demand pricing further ensures cost efficiency by scaling automatically based on workload. The current serverless architecture avoids fixed networking costs such as NAT Gateway. In more complex production designs, additional networking components may increase cost.

Sustainability

The serverless architecture contributes to sustainability by reducing unnecessary resource usage. AWS manages infrastructure efficiently, optimizing compute resources behind the scenes. By avoiding always-on servers and relying on event-driven execution, the system minimizes energy consumption and supports environmentally responsible cloud practices.

Cost Estimation

The SOS platform is designed using serverless architecture, which follows a pay-as-you-use pricing model. This ensures that costs are directly aligned with system usage, making the solution cost-efficient, especially for small to medium-sized workloads.

The following table provides an estimated monthly cost breakdown based on moderate usage, assuming approximately 1,000–2,000 users per month.

Estimated Monthly Cost Breakdown

Service	Usage Assumption	Estimated Monthly Cost (USD)
Amazon S3	Static website hosting (~5–10 GB storage + requests)	\$1 – \$3
Amazon CloudFront	Content delivery (~50–100 GB data transfer)	\$5 – \$10
Amazon Cognito	Up to 50,000 monthly active users (free tier applies)	\$0
API Gateway	~1 million API requests	\$3 – \$5
AWS Lambda	~1 million requests (short execution time)	\$1 – \$3
DynamoDB	On demand (~low to moderate read/write traffic)	\$3 – \$6
Amazon SNS	Notification triggers (low usage)	<\$1
Amazon SES	Email sending (within free tier limits)	\$0 – \$1
Amazon CloudWatch	Logs and monitoring	\$2 – \$5
AWS CloudTrail	Basic logging (mostly free tier)	\$0
AWS KMS	Minimal key usage	\$1
AWS Secrets Manager	Few secrets stored	\$1 – \$2
Amazon Lex	Low chatbot usage	\$1 – \$3
Terraform (S3 + DynamoDB backend)	State storage + locking	<\$1
GitHub Actions	Free tier / minimal usage	\$0

Total Estimated Monthly Cost: Approximately \$20 – \$45 USD per month

NOTE: (Cost may vary depending on traffic, usage patterns, and region.)

Cost Analysis

The overall cost of the system remains relatively low due to the use of serverless and managed services. Since services like AWS Lambda, API Gateway, and DynamoDB scale automatically, there is no need to pay for idle resources. This makes the architecture highly cost-efficient for applications with variable or unpredictable traffic. As traffic increases, services such as Lambda and API Gateway scale automatically, causing cost to grow proportionally with usage.

Additionally, several services such as Amazon Cognito and AWS CloudTrail operate within the AWS Free Tier under moderate usage, further reducing expenses. The use of on-demand pricing ensures that costs increase only when the system experiences higher traffic.

However, certain components, such as CloudFront data transfer and CloudWatch logs, can contribute to increased costs as usage grows. In a production environment, cost optimization strategies such as log retention policies, caching improvements, and efficient API design can help control expenses.

Conclusion

In conclusion, the SOS platform demonstrates a complete and practical implementation of a modern cloud-based application using AWS services. By adopting serverless architecture, the system can achieve high scalability, reduced operational overhead, and efficient resource utilization without the need to manage traditional infrastructure.

The integration of services such as Amazon S3, CloudFront, Cognito, API Gateway, Lambda, and DynamoDB enables the application to deliver a seamless user experience from browsing products to completing a purchase. Additional components like Amazon SNS, Telegram Integration and Lex enhance operational visibility and user interaction, while CloudWatch and CloudTrail provide visibility and control over system operations.

The architecture is designed with consideration for the AWS Well-Architected Framework, ensuring that best practices are followed in terms of security, reliability, performance, cost efficiency, and sustainability. Features such as automated deployments using Terraform, real-time monitoring with Telegram alerts, and secure authentication mechanisms further strengthen the overall solution.

Overall, this project reflects a well-structured and production-ready approach to building scalable cloud applications. This project not only demonstrates the ability to build a functional application but also reflects an understanding of how to design scalable, secure, and production-ready cloud solutions.

Questions and Answers:

a) How products are being updated on your website?

⇒ Products are updated through the admin side of the SOS platform. Authorized administrators can add, edit, or delete products using the admin dashboard. When an admin performs any of these actions, the request is sent through Amazon API Gateway to AWS Lambda functions, which process the request and updates the product data stored in Amazon DynamoDB. Once the database is updated, the changes are reflected on the website when users browse the product catalog.

b) Have you considered guest checking out? Or must all customers create their own account/profile first?

⇒ In the current implementation of the SOS platform, users must create an account and log in before placing an order. This approach was chosen to ensure secure access, maintain order history, and allow user-specific features such as profile management, password reset, and order tracking. Guest checkout was considered as a possible future enhancement, but it is not included in the current version of the project.

c) Are you saving customer financial information in your application?

⇒ No, the application does not store any customer financial information. Payment processing is handled securely through Stripe, which manages sensitive card and transaction details on its own platform. The SOS system only receives the payment result and transaction status required to process the order. This design reduces security risk and helps maintain a safer payment workflow.

d) How much is the monthly cost of your web application?

⇒ The estimated monthly cost of the SOS platform is approximately \$20 to \$45 USD under low to moderate usage. Since the application is built using serverless and managed AWS services, most of the cost depends on actual usage rather than fixed infrastructure. Services such as AWS Lambda, API Gateway, DynamoDB, S3, and CloudFront follow a pay-as-you-use model, which keeps the overall cost relatively low for small-scale deployments.